

Hybrid POMDP for Mobile ITS with Co-optimized Pedagogical Effectiveness and Mobile Performance

By

Vimanga Umange

A Thesis

Submitted to the Faculty of Graduate Studies
through the School of Computer Science
in Partial Fulfillment of the Requirements for
the Degree of Master of Science in Computer Science
at the University of Windsor

Windsor, Ontario, Canada

2026

© 2026 Vimanga Umange

Hybrid POMDP for Mobile ITS with Co-optimized Pedagogical Effectiveness and Mobile Performance

by

Vimanga Umange

Date of final oral defence: August 28th 2026

The document has received final approval by the Master's Committee:

Supervisor: Xiaobu Yuan

Program Reader: Jessica Chen

Program Reader 2: Muhammad Asaduzzaman

DECLARATION OF ORIGINALITY

I hereby certify that I am the sole author of this thesis and that no part of this thesis has been published or submitted for publication.

I certify that, to the best of my knowledge, my thesis does not infringe upon anyone's copyright nor violate any proprietary rights and that any ideas, techniques, quotations, or any other material from the work of other people included in my thesis, published or otherwise, are fully acknowledged in accordance with the standard referencing practices. Furthermore, to the extent that I have included copyrighted material that surpasses the bounds of fair dealing within the meaning of the Canada Copyright Act, I certify that I have obtained a written permission from the copyright owner(s) to include such material(s) in my thesis and have included copies of such copyright clearances to my appendix.

I declare that this is a true copy of my thesis, including any final revisions, as approved by my thesis committee and the Graduate Studies office, and that this thesis has not been submitted for a higher degree to any other University or Institution.

ABSTRACT

Mobile ITS deliver engaging avatar-based instruction and personalized instruction with verbal and non-verbal cues while running on heterogeneous mobile phones with strict latency and energy constraints. Recent mobile ITS have made progress on resource-constrained delivery through rule-based adaptation, offline caching, and hybrid local-server processing, and meta-analytic evidence confirms that ITS can improve learning outcomes across diverse contexts. However, existing mobile ITS and mobile edge computing frameworks typically optimize either pedagogical effectiveness or computational efficiency, and rarely treat both as a single learned joint decision problem. This research formulates mobile avatar-based tutoring as a hybrid Partially Observable Markov Decision Process (POMDP) and learns a deep Q-learning policy that simultaneously controls what to teach, where to execute the computation, and how richly to render the avatar interface. The state space factors a Bayesian Knowledge Space Theory (KST) mastery model, engagement and affect estimates, and detailed device telemetry including battery level, CPU load, memory pressure, and network quality. The action space combines micro-pedagogical moves with compute-venue choices across local, edge, and cloud tiers and four avatar modalities ranging from full 3D to text-only. A multi-objective reward function balances learning gain, latency, energy consumption, and modality appropriateness, and is optimized using a modified Deep Q-Network trained inside the Dart/Flutter runtime of the ECA Tutor research prototype. In large-scale simulations spanning three device classes, three network scenarios, and multiple trade-off settings, the learned policy achieves a substantial 44% relative improvement in normalized learning gain over a pedagogy-only baseline. Simultaneously, it achieves a 498.5 ms reduction in average per-step latency compared to the baseline, satisfying a strict 500 ms real-time SLA threshold across 98.68% of all tutoring steps. These results show that a single Q-learning-based controller can preserve pedagogical effectiveness and drastically reduce latency, though perfectly balancing energy consumption and strict modality compliance under extreme hardware stress remains an open challenge for future refinement. Ultimately, this framework makes a novel contribution at the intersection of ITS, reinforcement learning, and mobile edge computing.

DEDICATION

I dedicate this thesis to my beloved parents (Ammi and Thathi).

My father is my hero and someone I have looked up to every day of my life. His incredible love, unwavering support, steadfast dedication and selfless hard work to provide for our family have been my greatest inspiration. My mother's nurturing care, wisdom, and constant encouragement have been the source of my strength and comfort throughout this journey. My brother's strength and resolve have also inspired me to persevere through every challenge that has come my way. Everything I am today is because of them, and they are, and will always be, my greatest pillars of strength.

I also dedicate this work to the memory of my grandfather and grandmother, whom I believe are watching over me from above.

To my friends both back in the United States and here in Canada, thank you for your unwavering support and encouragement. Your belief in me has meant more than words can express.

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Dr. Xiaobu Yuan, for his guidance and support throughout this research. I am also grateful to my committee members, Dr. Jessica Chen and Dr. Muhammad Asaduzzaman, for their constructive feedback.

TABLE OF CONTENTS

DECLARATION OF ORIGINALITY.....	III
ABSTRACT.....	IV
DEDICATION	V
ACKNOWLEDGEMENTS.....	VI
1 INTRODUCTION.....	1
2 LITERATURE REVIEW.....	4
2.1 Intelligent Tutoring Systems and Engagement.....	4
2.2 ITS on Mobile Platforms	5
2.3 Domain Ontology and Knowledge Space Theory	6
2.3.1 Domain Ontology	6
2.3.2 Knowledge Space Theory and the Outer Fringe	7
2.4 Student Modeling and Knowledge Tracing.....	8
2.5 Interface Design and Avatar Presentation.....	9
2.6 Reinforcement Learning for Tutoring Model	9
2.7 POMDP-Based Tutoring.....	10
2.8 Mobile Edge Computing and Computation Offloading	12
2.9 Transfer Learning and Policy Reuse in Reinforcement Learning	13
2.10 Research Gap.....	13
3 RELATED WORK	16
3.1 Embodied Conversational Agents in Intelligent Tutoring Systems.....	16
3.2 Reinforcement Learning and Knowledge Tracing in ITS	18
3.3 Resource-Aware and Mobile Intelligent Tutoring Systems.....	20
3.4 Transfer Learning and Cross-Context Generalization in Education.....	22
3.5 Motivation for the Proposed Framework.....	24
4 PROBLEM STATEMENT AND CONTRIBUTIONS	26
5 PROPOSED HYBRID POMDP-DQN FRAMEWORK.....	29
5.1 Proposed Framework.....	29
5.1.1 Architectural overview	29
5.2 Domain Ontology with Knowledge Space Theory.....	32

5.2.1 Component structure.....	32
5.2.2 KST graph and outer fringe	33
5.3 Hybrid Decision Layer	35
5.3.1 Control Flow and Belief Update.....	35
5.3.2 Belief states and mastery estimation	39
5.3.3 Reward function	40
5.4 POMDP Action Policy with Q-Learning.....	46
5.4.1 Action representation and abstraction.....	46
5.4.2 Q-learning and policy training.....	47
5.5 Resource Management and Scaffolding	50
6 EXPERIMENTS AND ANALYSES	54
6.1 Prototype Implementation	54
6.2 Experimental Setup.....	58
6.2.1 Software and Implementation Platform.....	58
6.2.2 Student Simulation Model	61
6.2.3 Per-Task Cost Model	61
6.2.4 Evaluation Metrics.....	62
6.3 Experimental Design	62
6.3.1 Factor Grid.....	62
6.3.2 Network and Device Parameters	63
6.3.3 Baseline Policies.....	63
6.3.4 Episode Budget and Scale	64
6.3.5 Pre-Specified Hypotheses.....	64
6.4 Statistical Methodology	65
6.4.1 Aggregation and Pseudoreplication Prevention.....	65
6.4.2 Hypothesis Tests	66
6.4.3 Effect Size: Cohen's d.....	66
6.4.4 Confidence Intervals.....	66
6.4.5 Justification for the Selected Methods	67
6.5 Experimental Results	67
6.5.1 Learning Preservation and Curriculum Adherence (H1a, H1b)	67

6.5.2 Latency and Energy Reduction (H2a, H2b)	69
6.5.3 Robustness Across Device and Network Conditions (H3a, H3b)	71
6.5.4 User Experience Consistency (H4a, H4b)	71
6.5.5 Transfer Learning (H5a, H5b)	73
6.5.6 Summary Scorecard.....	74
6.6 Analysis and Discussion	75
6.6.1 Key Findings	75
6.6.2 Why the α Curve Flattened.....	77
6.6.3 Why Modality Appropriateness Declined	78
6.6.4 Threats to Validity and Limitations	79
7 CONCLUSIONS	81
7.1 Summary of the Research.....	81
7.2 Achievements and Contributions.....	81
7.3 Limitations.....	82
7.4 Future Work	83
7.5 Closing Remarks.....	84
REFERENCES/BIBLIOGRAPHY	85
APPENDICES	93
Appendix A	93
Appendix B	96
VITA AUCTORIS	102

LIST OF TABLES

3.1: Comparison of ECA-Based ITS Approaches.....	17
3.2: Comparison of RL and Knowledge Tracing Approaches in ITS.....	19
3.3: Comparison of Resource-Aware and Mobile Computing Approaches.....	20
3.4: Transfer Learning Approaches Relevant to Mobile ITS Deployment.....	22
5.1: Belief Value Update and Skill Mastery Determination Example.....	39
5.2: Reward Function and Belief-Update Parameters and Their Roles	45
5.3: Example initial Q-values for fringe-restricted S6.....	49
5.4: Updated Q-values for state 5 after one training step.....	50
6.1: Experimental factor grid.....	62
6.2: Network scenario parameters.....	63
6.3: Device class parameters.....	63
6.4: Pre-specified hypotheses	64
6.5: Hypothesis testing summary.....	74

LIST OF FIGURES

2.1: Classic architecture of an ITS.....	4
2.2: KST diagram.....	8
2.3. The POMDP tutoring interaction loop.....	11
5.1: System Architecture Overview.....	31
5.2: KST DAG.....	34
5.3: Per-step control flow of the hybrid decision layer.....	38
5.4: Reward function decomposition.....	44
5.5: Worked-solution view in ECA Tutor.....	53
6.1: ECA Tutor interface modality tiers (text-only through full 3D).....	56
6.2: ECA Tutor home screen showing full 3D avatar in active tutoring mode.....	57
6.3: Mastery likelihood evolution for the KST base curriculum (three simulations).....	60
6.4: Episode-level learning gain distributions by policy.....	68
6.5: Mean per-step latency by policy condition.....	69
6.6: Box plots of total energy consumption per policy.....	70
6.7: Learning gain, latency, and energy as functions of fixed α	73
6.8: Pareto trade-off: learning gain vs latency.....	75

LIST OF APPENDICES

Appendix A: Extended Experimental Results.....	93
Appendix B: ECA Tutor System Configuration.....	96

LIST OF ABBREVIATIONS/SYMBOLS

3D	Three-Dimensional
AI	Artificial Intelligence
ALEKS	Assessment and Learning in Knowledge Spaces
BKT	Bayesian Knowledge Tracing
CI	Confidence Interval
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DSP	Digital Signal Processor
ECA	Embodied Conversational Agent
FPS	Frames Per Second
GIFT	Generalized Intelligent Framework for Tutoring
GPU	Graphics Processing Unit
ITS	Intelligent Tutoring System
KC	Knowledge Component
KST	Knowledge Space Theory
KT	Knowledge Tracing
LG	Learning Gain
MEC	Mobile Edge Computing
OCC	Ortony-Clore-Collins (affect model)
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
QoE	Quality of Experience
QoS	Quality of Service
RL	Reinforcement Learning
RQ	Research Question
SLA	Service Level Agreement
STEM	Science, Technology, Engineering and Mathematics
UI	User Interface

NOMENCLATURE

S	State space of the POMDP
A	Action space of the POMDP
T	Transition function $T(s' s, a)$
R	Reward function $R(s, a)$
Ω	Observation set
O	Observation function $O(o s', a)$
γ	Discount factor in $[0, 1]$
b	Belief state - probability distribution over S
α	Pedagogy–efficiency trade-off parameter in $[0, 1]$
θ	DQN network weights
θ^-	Target network weights (synchronized every 8 steps)
ε	Exploration rate for ε -greedy action selection
$Q(s,a)$	State-action value function
$L(\theta)$	DQN Bellman loss function
K	Knowledge state - a subset of domain items Q
$\text{fringe}(K)$	Outer fringe: items not in K whose prerequisites are all in K
Δk_t	Per-step knowledge delta at step t
C_t	Cognitive load estimate at step t
B_t	Battery level at step t , normalized to $[0, 1]$
lat_t	Per-step latency in milliseconds
ΔB_t	Per-step battery drain
k_t	Mastery belief
E_t	Emotional valence
G_t	Engagement level
m_t	Mastery level
N_t	Network quality
P_t	CPU load
μ_t	Memory usage (memory pressure)
T_t	Thermal state
x_t	Factored joint POMDP state vector, $x_t = [b_t; d_t]$

CHAPTER 1

INTRODUCTION

Digital education has become a cornerstone of modern learning, providing scalable, accessible, and personalized educational opportunities to learners worldwide. It plays a critical role in democratizing education by overcoming geographical and temporal barriers, thereby enabling continuous, lifelong learning for diverse student populations. A crucial aspect of this progress is addressing the issue of accessibility for users with different types of internet connections and hardware devices, particularly for those in remote regions and developing countries [70].

The global expansion of this paradigm into mobile-first digital education has transformed the deployment landscape for ITS, linking the need for equitable e-education to mobile platforms. Learners now encounter these systems on smartphones and tablets rather than in computer labs, in settings ranging from quiet study rooms to crowded commutes [45]. These environments impose hardware and connectivity constraints that desktop-era ITS designs never anticipated [55].

Modern mobile devices support animated avatars, speech synthesis, and cloud-hosted student models, with features that approach the interactivity of human tutoring [14]. However, delivering them places substantial computational demands on hardware that was never designed for educational inference workloads. On resource-constrained mobile phones, high-fidelity avatar rendering and large neural inference can easily exceed latency and energy budgets, especially under variable network conditions, leading to sluggish responses, rapid battery drain, and mid-session interaction failures [36].

The deployment of ITS on mobile devices introduces a fundamental tension: pedagogical richness demands computationally expensive avatar rendering, speech synthesis, and cloud inference, while mobile constraints impose strict latency, energy, and memory budgets. Research on interactive learning consistently shows that response latency exceeding approximately 500 milliseconds disrupts cognitive flow, reduces perceived responsiveness, and increases

disengagement [59]. When a tutor’s responses are delayed or interrupted, students abandon tasks or shift attention away from the learning activity, reducing learning gains even when the underlying pedagogical policy is sound [58][60].

Existing approaches address this tension through fixed heuristics, for example by degrading to text-only when the battery falls below a threshold, which decouple instructional decisions from resource management. However, such decoupling is suboptimal. Policies that always select the richest presentation (full 3D avatar, high-quality speech, cloud inference) achieve strong learning outcomes in principle but violate latency and energy constraints on constrained devices. At the other extreme, policies that consistently choose low-cost text-only representations respond quickly and conserve battery, yet sacrifice engagement, curriculum adherence, and learning gain compared to richer interactive experiences.

Furthermore, reinforcement-learning-based tutors demonstrate that policies learned from interaction data can outperform hand-crafted sequencing heuristics [52]. However, these systems almost always treat the execution environment as fixed infrastructure, optimizing learning outcomes while assuming that computational resources and network conditions are stable.

This research investigates whether pedagogical quality and mobile resource efficiency can be addressed within a single decision-making framework on mobile hardware, rather than through separate systems that must later be reconciled. To address this problem, this research proposes a unified, AI-driven framework that treats pedagogical decisions, compute-venue selection, and avatar modality choice as a single joint decision problem. The approach formulates mobile avatar-based tutoring as a hybrid Partially Observable Markov Decision Process (POMDP) in which each tutoring step constitutes a joint decision over what to teach and how to execute and present that instruction. The proposed framework represents a student’s hidden knowledge state and observable device context in a single factored state, and selects composite actions that simultaneously determine what to teach, where to run the computation (locally, on edge servers, or in the cloud), and which avatar modality to present. A Deep Q-Network (DQN) trained within the Dart/Flutter runtime of the ECA Tutor prototype application learns to balance pedagogical richness against latency, energy, and robustness constraints.

The remainder of this research paper is organized as follows. Chapter 2 reviews the literature on ITS, Knowledge Space Theory, reinforcement learning, POMDP-based tutoring, mobile edge computing, and transfer learning. Chapter 3 surveys the most directly related work

and positions the proposed framework against it. Chapter 4 states the problem formally and enumerates the contributions of this research. Chapter 5 details the proposed hybrid POMDP-DQN framework as a general, three-layer architecture, including the KST curriculum structure, factored state representation, multi-objective reward function, and modified Q-learning algorithm. Chapter 6 describes the prototype implementation of this framework and reports the experimental evaluation across device classes, network scenarios, and trade-off configurations, together with the analysis and discussion of results. Chapter 7 concludes the thesis and outlines its limitations and directions for future work.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the core literature underlying the proposed framework. The discussion is organized around ITS components and engagement, ITS on mobile platforms, Knowledge Space Theory, reinforcement learning for tutoring models, POMDP-based tutoring, mobile edge computing and computation offloading, and transfer learning. The chapter concludes by summarizing the research gap that motivates the proposed work.

2.1 Intelligent Tutoring Systems and Engagement

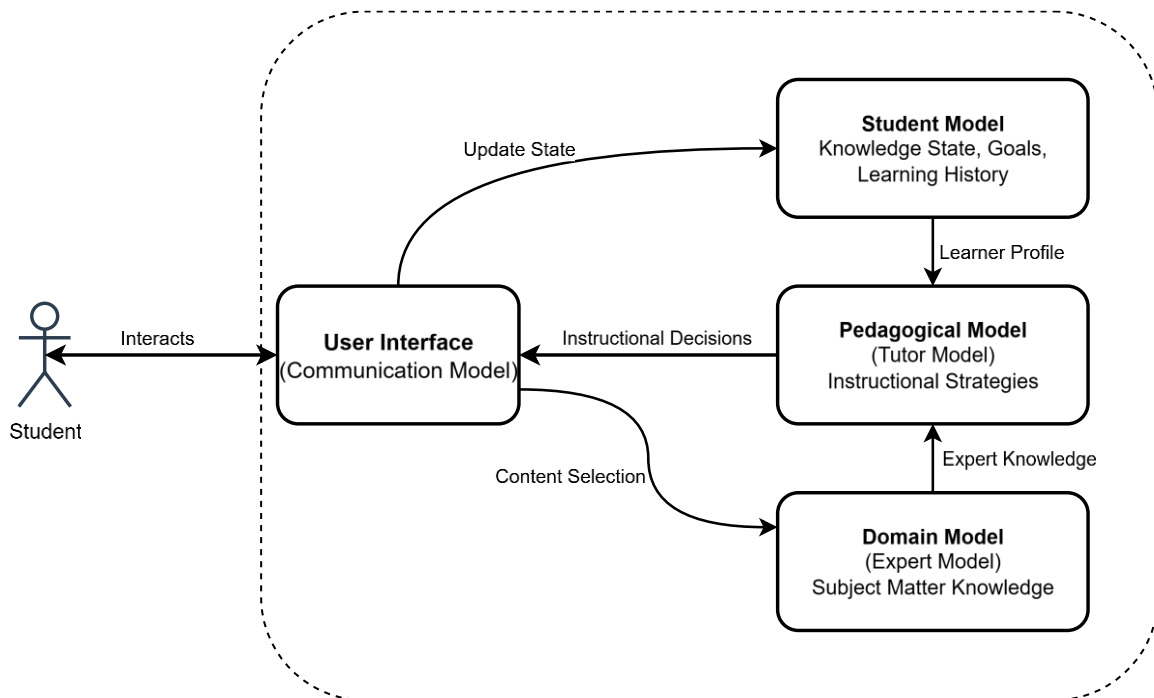


Figure 2.1: Classic architecture of an ITS, showing the four core components (domain model, student model, pedagogical model, and user interface) and their interactions.

As shown in Figure 2.1, an ITS is a computer-based learning environment that aims to provide personalized, adaptive, and interactive instruction comparable to human tutors [12]. An ITS works by integrating four interdependent components. First, the domain model encodes the expert subject matter knowledge and the specific concepts to be learned; this component is discussed in detail in Section 2.3. Second, the student model tracks the learner’s current knowledge state, goals, and learning history, continuously updating based on the learner’s performance; Section 2.4 explores student modeling with a focus on knowledge tracing. Third, the pedagogical model (or tutor model) governs the instructional strategies, utilizing the student model to select appropriate content, hints, and feedback; the literature underlying this component is reviewed in Section 2.6 and Section 2.7. Finally, the interface model (or communication model) controls how content is presented to the learner and handles all interactions, which is the focus of Section 2.5 on interface design with avatars. On desktop platforms, the interface model can render high-fidelity media without consequence, but on constrained platforms, its choices directly impact the computational demand placed on the other components.

2.2 ITS on Mobile Platforms

The deployment of ITS on mobile platforms raises unique issues related to latency, energy consumption, and hardware heterogeneity. While mobile devices enable ubiquitous learning, computationally demanding features like 3D avatars and cloud inference can exceed device budgets, leading to sluggish responses and battery drain. These technical issues directly affect student engagement, which spans behavioral, cognitive, and emotional dimensions. When an interface is unresponsive or a session terminates due to battery failure, behavioral flow is disrupted, cognitive load increases, and students experience frustration. This emotional disengagement suppresses the learning rate and increases dropout [57].

To capture these emotional dimensions computationally, the Ortony-Clore-Collins (OCC) model offers a theoretical foundation for computational affect modeling. In the OCC framework [77], emotions are cognitive appraisals of events, agents, and objects: joy arises from a desirable event, distress from an undesirable one, pride from a praiseworthy self-action, and shame from a blameworthy one. These emotion categories can be operationalized as scalar valence signals updated after each interaction, which in turn modulate a pedagogical reward function, with a

higher engagement scalar amplifying the learning rate and elevated frustration suppressing it and increasing early-termination risk.

Building on this affect-aware foundation, recent research has also made significant progress in addressing mobile deployment constraints, particularly in low-resource and underserved educational contexts [71]. Contemporary mobile AI tutors have successfully employed lightweight, rule-based adaptations, offline caching, and hybrid functionality to deliver adaptive pacing, scaffolded learning, and gamified reinforcement without requiring constant, high-bandwidth connectivity; examples include gamified electrocardiogram tutoring delivered entirely on-device [72], scaffolded verbal-feedback tutoring for pre-school literacy [74], and dialogue-based Arabic-language tutoring apps that combine local and server-side processing [73]. Evidence indicates that when these systems utilize adaptive feedback to address misconceptions, they yield measurable learning gains and increased motivation [71][76]. However, widespread adoption is still limited by systemic barriers, including digital inequities in device access, unstable connectivity, and the challenge of localizing content to diverse cultural contexts [75]. Furthermore, while existing mobile ITS commonly rely on transparent rule-based models or basic natural language processing, the use of advanced, data-driven decision policies like reinforcement learning remains limited due to high computational and data requirements [71].

Together, these lines of work show that mobile ITS research has made real progress on resource-aware delivery and learner engagement but has not yet unified affect-aware tutoring with learned, runtime-adaptive control over latency, energy, and modality.

2.3 Domain Ontology and Knowledge Space Theory

2.3.1 Domain Ontology

A domain ontology is a formal representation of the concepts, subtopics, and skills that make up a subject area, together with the relationships between them; it typically takes the form of a hierarchy or graph in which each node is a topic, subtopic, or individual skill and each edge encodes a relationship such as containment or, where such information is available, prerequisite structure [78]. In ITS design, the domain ontology is the concrete realization of the domain

model shown in Figure 2.1: it is the structure the tutor consults to determine what a topic contains and how topics relate to one another.

Domain ontologies are obtained in one of two ways. In most existing systems, the ontology is curated by hand: instructional designers and subject-matter experts organize a curriculum’s topics and subtopics into a hierarchy based on pedagogical judgment and available syllabus or textbook structure [11]. Because hand curation does not scale well to new subjects, recent work has also explored automatic domain-ontology construction, deriving the topic hierarchy directly from public-domain educational datasets rather than authoring it manually; Jamal [78] proposes such a pipeline for personalized ECA-based tutoring. However, ontologies produced by either method typically capture only a natural, containment-style hierarchy rather than the fine-grained prerequisite relationships between individual skills that a sequencing policy requires. Knowledge Space Theory, introduced next, closes this gap by augmenting the domain ontology with an explicit precedence structure over its elements; Section 5.2 uses this KST-augmented domain ontology directly as the domain model of the proposed framework.

2.3.2 Knowledge Space Theory and the Outer Fringe

Knowledge Space Theory (KST) provides a mathematical framework modeling learner knowledge as a set of mastered concepts constrained by prerequisite relationships [23]. In KST, the domain is represented as a finite set Q of items or problem types, and a knowledge space is a family of subsets $K \subseteq 2^Q$ (called knowledge states) satisfying closure axioms. Large-scale adaptive systems such as ALEKS apply KST-derived learning spaces for diagnostic assessment and personalized problem sequencing [25]. The outer fringe of a knowledge state K is the set of items whose prerequisites are satisfied but not yet mastered:

$$\text{fringe}(K) = \{q \notin K : \forall p \in \text{prerequisite}(q), p \in K\}$$

This outer fringe defines the valid region for selecting the next learning task.

In ITS, this fringe links the domain and student models by restricting sequencing to prerequisite-ready items, so the tutoring policy maintains curriculum adherence. Figure 2.2 illustrates this structure, distinguishing mastered concepts, the outer fringe of skills ready to learn, and concepts that remain unreachable until their prerequisites are satisfied.

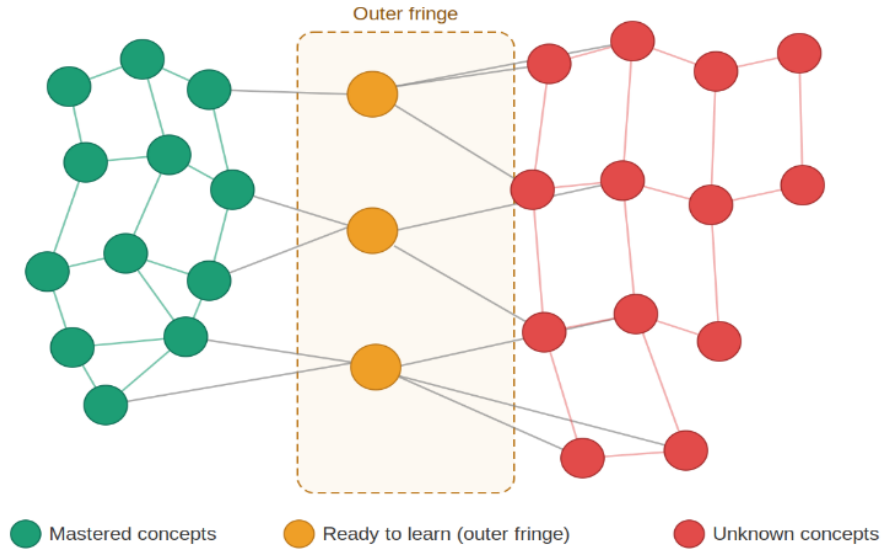


Figure 2.2: KST diagram showing mastered concepts, the outer fringe (skills ready to learn), and unknown concepts.

2.4 Student Modeling and Knowledge Tracing

Student modeling is a fundamental mechanism that empowers an ITS to adapt its instruction to individual learning paces and styles. A comprehensive student model continuously tracks the learner's evolving knowledge state while also considering cognitive and affective factors to personalize the educational experience. Knowledge tracing serves as the core of this model by evaluating a student's mastery of specific concepts over time through their task performance and interactions. For instance, incorporating Knowledge Space Theory (KST) allows the system to analyze the student's knowledge state throughout the learning session to delineate critical learning paths [22][23]. By accurately modeling the student's knowledge level, an ITS can offer timely interventions, balance cognitive load, and maintain emotional engagement. This continuous assessment ensures that learning materials remain appropriately challenging, mitigating boredom and frustration while maximizing long-term learning outcomes.

2.5 Interface Design and Avatar Presentation

The interface model of an ITS serves as the critical communication bridge between the student and the underlying educational materials. To overcome the lack of human-to-human interaction in traditional e-learning, modern interface designs increasingly incorporate Embodied Conversational Agents (ECAs) or animated avatars to simulate face-to-face tutoring. These virtual agents utilize visual and auditory cues such as facial expressions, gestures, and synthesized speech to create a socially immersive learning environment [15]. Implementing an emotionally expressive avatar within the interface design fosters a stronger emotional connection and sense of belonging for the learner. By providing real-time, empathetic feedback and structured guidance, avatar-based interfaces can significantly enhance student motivation and sustain high levels of emotional engagement throughout the learning session [17].

2.6 Reinforcement Learning for Tutoring Model

To optimize instructional sequencing, modern ITS increasingly employ machine learning techniques. Reinforcement Learning (RL) provides a robust paradigm for the tutoring model by learning policies directly from student interaction data. Unlike static heuristics or rule-based models, RL agents can discover non-obvious instructional strategies that maximize long-term learning gains. Within this space, Deep Q-Networks (DQN) offer significant advantages in comparison to traditional tabular Q-learning. Tabular Q-learning relies on a discrete table to store a value for every state-action pair, which becomes computationally intractable as the state space grows. DQN, by contrast, utilizes deep neural networks to approximate the action-value function instead of maintaining a table. This approach allows DQN to naturally interpolate values for unseen states and generalize across large, continuous, high-dimensional state spaces, such as those incorporating detailed device telemetry and continuous affect signals. This capability to handle high-dimensional, factored state representations makes DQN particularly suitable for solving the Partially Observable Markov Decision Process (POMDP) formulation of the mobile tutoring problem.

2.7 POMDP-Based Tutoring

Before discussing the mathematical specifics of Partially Observable Markov Decision Processes (POMDP), it is crucial to link them to the tutoring model of an ITS. In interactive environments such as dialogue systems, POMDP models are extensively used to capture uncertainty over user intentions and speech recognition errors by maintaining a belief distribution over latent states. This concept naturally extends to ITS, where the student’s true knowledge and affective state are never directly observed and must be inferred from their responses. The POMDP model links directly to the tutoring model by formalizing the sequential decision-making process under uncertainty. The tutor maintains a belief state over the student’s mastery and affect, updating it after each interaction. By integrating this framework with reinforcement learning, the tutoring model can optimize pedagogical actions based on an explicit representation of uncertainty.

As illustrated in Figure 2.3, Partially Observable Markov Decision Processes (POMDPs) provide the natural formalism for ITS design: a tutor must make instructional decisions while the student’s true knowledge state is never directly observed [29]. A POMDP is defined as a seven-tuple $\langle S, A, T, R, \Omega, O, \gamma \rangle$, where S is the set of hidden states (e.g., student mastery and true affect), A is the set of pedagogical and offloading actions, T describes the transition probabilities between states, R defines the multi-objective reward function, Ω is the set of observations (e.g., test correctness and detected emotion), O specifies the conditional observation probabilities, and γ is the discount factor. Because the true state s is hidden, the agent maintains a belief state b , a probability distribution over S , updated after each step using Bayes’ rule. The agent selects actions to maximize expected discounted cumulative reward given the belief trajectory.

In dialogue systems, POMDP models capture uncertainty over user intentions and Automatic Speech Recognition (ASR) errors. In the ITS domain, tractable POMDP representations such as state queues and observation chains exploit task structure to make POMDP-based tutoring scalable [32]. Most existing POMDP-based ITS work focuses on pedagogical decisions, choosing which skill or problem to present and how to respond to student actions, without explicitly modeling device-level technical context such as CPU load, network latency, or battery level. As a result, these approaches cannot directly reason about trade-offs between pedagogical richness and mobile performance constraints.

Solving a POMDP exactly is computationally intractable for large state spaces, as the belief space is a continuous high-dimensional simplex [31]. Practical systems use approximate techniques: point-based value iteration, policy gradient methods, or deep Q-networks approximating the action-value function directly from the belief-observation vector [33]. The deep Q-network formulation is especially suited to mobile deployment because it avoids the memory overhead of maintaining a full belief distribution over a large knowledge graph combined with continuous technical context.

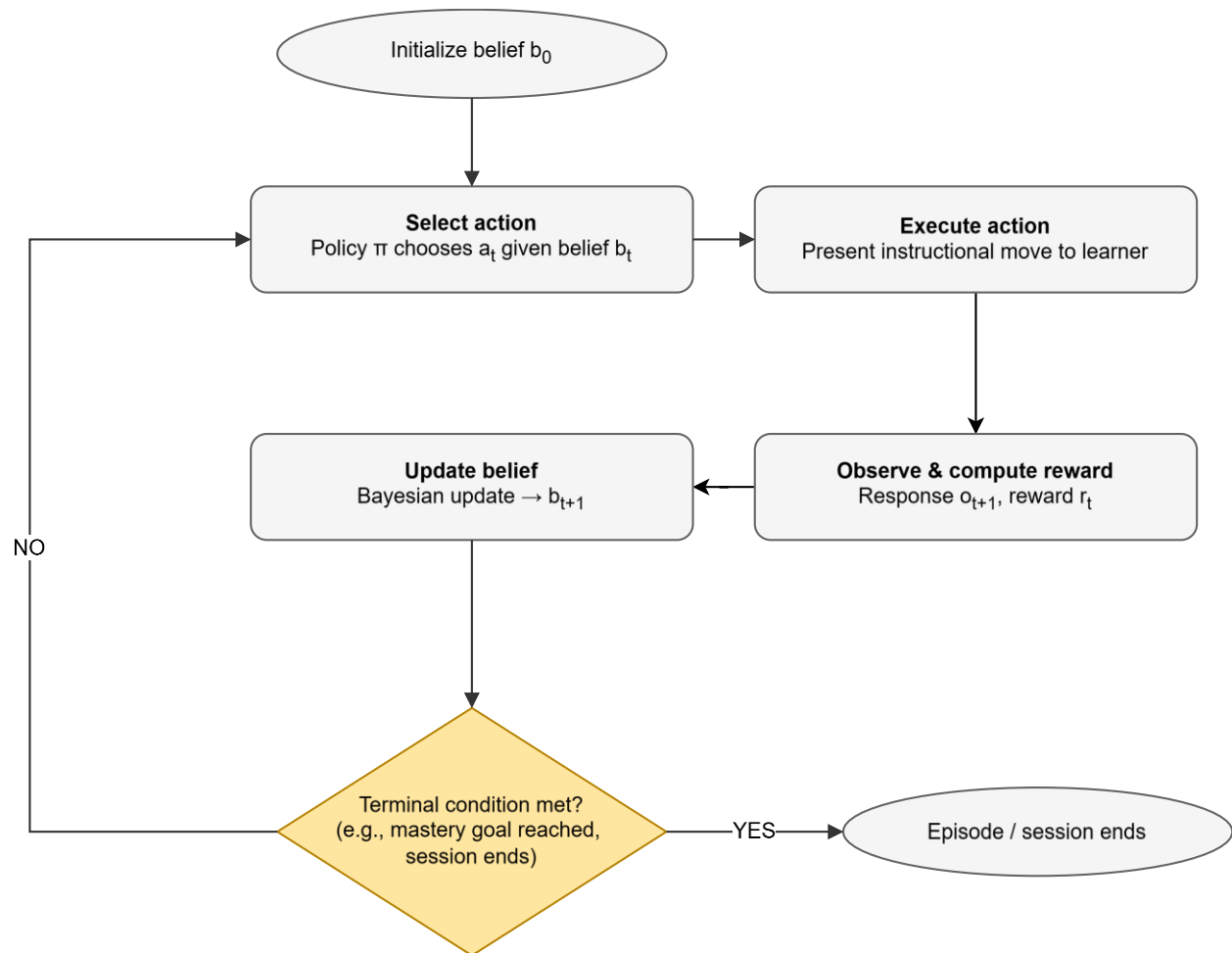


Figure 2.3: The POMDP tutoring interaction loop.

2.8 Mobile Edge Computing and Computation Offloading

Mobile Edge Computing (MEC) and cloudlet architectures reduce latency and energy in mobile applications by offloading computation to nearby servers [39]. By processing tasks closer to users, MEC significantly reduces round-trip times compared to distant cloud data centers, while relieving mobile devices of heavy CPU and GPU workloads and conserving battery life.

A large body of work formulates offloading as an optimization problem balancing latency and energy. Task-assignment schemes allocate tasks across local devices, edge servers, and cloud servers to minimize energy-latency trade-offs under resource constraints [36][37]. Recent studies apply deep reinforcement learning to offloading decisions, modeling the problem as a Markov decision process and using algorithms such as Deep Deterministic Policy Gradient (DDPG) to learn policies dynamically partitioning tasks between local, edge, and cloud sites [35].

Recent work in multi-objective reinforcement learning for computation offloading demonstrates that nonlinear scalarization methods outperform linear weighting for conflicting objectives [66]. Double deep Q-network (DDQN) approaches with adaptive weight learning can simultaneously optimize delay, energy consumption, computing load, and privacy entropy by maintaining separate Q-networks per objective and dynamically adjusting their relative importance through learned scalarization [69]. These techniques establish that multi-objective RL is viable for real-time resource allocation in dynamic network environments.

These approaches, however, are application-agnostic: they optimize generic QoS metrics without modeling educational objectives such as learning gain, curriculum adherence, or engagement. They also assume tasks are fixed units of computation, whereas avatar-based tutoring systems can flexibly change modality and pedagogical strategy in response to context. Within the framework of an ITS, Mobile Edge Computing can be directly linked to the execution of computationally intensive tasks such as 3D avatar rendering, speech synthesis, and deep knowledge tracing. By dynamically offloading these tasks, a mobile ITS can relieve the local CPU and GPU, conserving battery life and reducing response latency. Integrating MEC into the ITS decision-making framework allows the system to balance the quality of the pedagogical presentation with the realities of the mobile execution environment, ensuring that the interface remains responsive and the student remains engaged.

2.9 Transfer Learning and Policy Reuse in Reinforcement Learning

Transfer learning in reinforcement learning accelerates learning on new tasks by initializing from previously learned policies rather than random weights [50]. Policy transfer techniques include action advising, where a teacher policy occasionally overrides or recommends actions to a student learner, and frameworks that adaptively select among multiple source policies based on similarity to the target task [67].

For example, adaptive policy transfer frameworks select an optimal source policy and blend it with ongoing learning, while monitoring for negative transfer and terminating the use of unhelpful policies. These methods have shown that reusing policies can significantly reduce sample complexity and improve initial performance across diverse control domains [50].

While transfer learning has been studied extensively in generic DRL benchmarks, its application to mobile ITS remains limited. In particular, there is little empirical evidence on whether policies trained under one combination of device and network conditions can be reused effectively in other contexts without extensive fine-tuning. In relation to the challenges outlined in the previous subsections, such as managing diverse mobile constraints via MEC and optimizing tutoring policies through reinforcement learning, transfer learning offers a solution for scalability. Once an optimal tutoring policy is learned for a specific mobile environment, transfer learning enables that policy to be reused in different contexts. For example, a policy trained on a high-end device with fast Wi-Fi can be adapted for a low-end device on a 4G network. By leveraging transfer learning, the ITS can generalize its pedagogical and offloading strategies to new deployment scenarios, ensuring robust performance across heterogeneous hardware and networks without the need for extensive retraining.

2.10 Research Gap

Existing literature on ITS, affective pedagogical agents, and Knowledge Space Theory provides strong foundations for modeling learner knowledge and engagement, but largely assumes stable, resource-rich environments. POMDP-based dialogue and tutoring systems demonstrate how decision-theoretic methods can handle uncertainty in student state and dialogue observations, yet

they rarely incorporate detailed device and network context or reason explicitly about latency and energy.

MEC and mobile offloading research offers sophisticated mechanisms for optimizing latency-energy trade-offs across local, edge, and cloud resources, but it typically treats application quality in task-agnostic terms, without modeling educational objectives such as learning gain or curriculum adherence. DRL-based instructional sequencing methods focus on maximizing learning outcomes using simulated students but largely ignore mobile QoE metrics and avatar modality decisions.

Furthermore, while transfer learning has shown promise in generic DRL benchmarks, its application for transferring joint pedagogical and resource-management policies across diverse mobile deployment contexts remains unexplored.

There is therefore a clear gap at the intersection of pedagogically grounded ITS, decision-theoretic dialogue and tutoring, and mobile offloading. As surveyed in Section 2.2 and Section 3.3, contemporary mobile ITS have made real progress on part of this problem: they successfully provide rule-based adaptive pacing, offline-capable scaffolding, and gamified practice under resource constraints [71][72][74]. Meanwhile, mobile edge computing research provides sophisticated latency-energy optimization, but only for generic, non-educational workloads [35][36][64][66]. However, none of the mobile ITS reviewed above treats latency and energy as explicit, learned decision variables alongside pedagogical objectives, and none of the MEC/offloading work reviewed models pedagogical objectives at all. This division between rule-based, resource-aware mobile ITS on one side and pedagogy-agnostic MEC optimization on the other is the specific gap this research targets: no reviewed approach provides a unified, learned controller that simultaneously optimizes learning, latency, energy, and modality appropriateness on real mobile runtimes while supporting transfer across heterogeneous deployment contexts. The hybrid POMDP and DQN framework developed in this research is designed to fill this gap precisely. Building on the recent progress of rule-based and offline-first mobile ITS in resource-constrained environments [71], this research introduces a data-driven alternative. To the best of our knowledge, it is the first system to replace static, rule-based resource management in mobile ITS with a single trained reinforcement learning controller that jointly optimizes pedagogical objectives (learning and modality appropriateness) and technical constraints (latency and energy). By running directly on the actual mobile deployment runtime

and providing systematic cross-context transfer evaluation across heterogeneous network-device combinations, this work extends, rather than originates, the field’s ongoing effort to deliver robust, multi-objective tutoring in dynamic mobile ecosystems.

The experiments in this research further contribute to this gap by evaluating zero-shot performance, which is defined here as a transferred policy’s performance in a new device-network context with no additional fine-tuning, as well as convergence speed when hybrid POMDP policies are transferred across heterogeneous network-device combinations. Chapter 3 surveys the systems and empirical studies most directly comparable to this work, grouped into four themes: embodied conversational agents in ITS, reinforcement learning and knowledge tracing, resource-aware mobile computing, and cross-context transfer learning. It then positions this research’s contributions against each area of prior work.

CHAPTER 3

RELATED WORK

This chapter surveys the systems and empirical results most directly comparable to the hybrid POMDP framework developed in this research. Four themes organize the survey: embodied conversational agents in ITS (§3.1), reinforcement learning and knowledge tracing (§3.2), resource-aware and mobile computing (§3.3), and cross-context transfer learning (§3.4). For each theme, a comparative table positions the most relevant prior work against the present framework, identifying the specific limitation that this research addresses.

3.1 Embodied Conversational Agents in Intelligent Tutoring Systems

Embodied conversational agents (ECAs) are virtual characters that interact with learners through speech, facial expressions, and gestures, and have been widely explored as a means of increasing engagement and motivation in ITS and web-based interventions [14]. Listed in Table 3.1, Lippert [65] describes multi-agent conversational ITS designs in which multiple animated avatars model expert, peer, and metacognitive roles; classroom studies show that such agents can support learning across STEM and research skills when combined with appropriate dialogue strategies.

Self-guided web-based interventions and advisory systems also use ECAs to provide both task-related and emotion-related support, with reviews suggesting that agents can sustain adherence and motivation, although long-term effects depend on context and design quality. Authoring tools such as XNAgent have been proposed to simplify the creation of ECAs within frameworks like GIFT, demonstrating how skeletal animation and event-driven behaviors can be integrated into tutoring platforms. Duong combines KST and POMDP-based dialogue with an emotionally expressive ECA, showing that synchronized emotion feedback and adaptive difficulty can improve emotional engagement and perceived support. This research extends

Duong’s desktop-oriented framework to mobile deployment, where the system has to balance pedagogical effectiveness with real-world QoE constraints, something Duong’s desktop implementation did not address [53].

Study	ECA Type	Platform	Key Contribution	Limitation	Relevance
Lippert (2020) [65]	Multi-agent animated	Desktop/Web	Multiple agent roles improve STEM learning	No mobile deployment	Confirms avatar value; motivates mobile adaptation
Scholten et al. (2017) [61]	Web ECA	Browser	Embodied agents sustain adherence in interventions	No adaptive modality	Shows engagement benefit without resource modeling
Olney et al. (2012) [62]	Skeletal animation	GIFT platform	XNAgent authoring tool for ITS ECA	Fixed avatar quality	Authoring approach, not adaptive rendering
Duong (2025) [53]	Emotionally expressive ECA	Desktop	KST+POMDP+Q-learning with affect feedback	No mobile QoE	Direct predecessor; this research extends to mobile runtime

Aleven et al. (2017) [48]	Cognitive tutor with step-level feedback	Desktop	Classroom- validated affect- based feedback	Fixed rendering quality; desktop only	Validates affect- driven feedback; motivates mobile adaptation
------------------------------------	---	---------	---	---	--

Table 3.1: Comparison of ECA-Based ITS Approaches

The approaches in Table 3.1 show the pedagogical value of avatar-based and affective tutors, but they largely assume desktop-class or web environments and treat avatar fidelity as a fixed design choice. They do not model battery, latency, or heterogeneous mobile hardware as first-class constraints, nor do they adapt rendering richness at runtime. The hybrid POMDP instead treats avatar modality as a per-step decision, allowing the policy to trade rendering richness against resource consumption.

3.2 Reinforcement Learning and Knowledge Tracing in ITS

Reinforcement learning (RL) has been increasingly used to learn instructional policies from interaction data or simulated students [52]. In Table 3.2, Subramanian and Mostow’s STEP framework uses a simulated student driven by knowledge tracing to train DRL policies that decide which activity to present, demonstrating gains in learning compared to hand-crafted sequencing heuristics. Abdelshiheed and colleagues [63] leverage DRL to deliver metacognitive interventions in a logic tutor; in classroom experiments, DRL-based adaptive interventions closed metacognitive skill gaps and prepared students for future learning better than static classifier-based interventions.

Study	Algorithm & Student Model	Key Contribution	Limitation	Relevance
Subramanian & Mostow (2023) [52]	DRL (PPO) & KT-based simulated student	Activity sequencing from simulated interactions	No mobile QoE	RL-from-simulation paradigm adopted in this research
Abdelshiheed et al. (2023) [63]	DRL & BKT	Metacognitive interventions, classroom validation	Desktop only	Validates RL-based ITS in real settings
Koroveshi (2021) [68]	DQN comparison & Simulated student (Python programming)	Exploration strategies and parameter tuning for DQN-based ITS	No resource modeling	Informs DQN configuration choices
Gong et al. (2022) [51]	Survey & Multiple	Comprehensive KT survey	N/A	Contextualizes mastery belief model
Hybrid POMDP (this research)	DQN+POMDP & Bayesian slip-guess	Joint pedagogical + QoE optimization on mobile	Synthetic student only	-

Table 3.2: Comparison of RL and Knowledge Tracing Approaches in ITS

Knowledge tracing itself has seen significant advances, with recent surveys reviewing classical models such as BKT and Performance Factor Analysis (PFA) alongside deep architectures that use recurrent and attention-based networks to predict students' future performance [51]. These models are central to many modern ITS because they provide the state estimates on which adaptive policies depend, and they are also a key component of the simulated

students used to train RL-based tutors. The work by Koroveshi [68] has compared exploration strategies for DQN-based ITS policies, showing that different choices of exploration can materially impact both training stability and final performance when reasoning over simulated learners.

While these studies demonstrate that RL and advanced student modeling can substantially improve instructional decisions, they share a common architectural assumption: reliable, high-resource computing. Mobile quality-of-experience variables like latency, energy, and modality validity do not appear in their reward functions. The hybrid POMDP incorporates these variables as first-class optimization targets alongside learning gain.

3.3 Resource-Aware and Mobile Intelligent Tutoring Systems

Research listed in Table 3.3 compares representative mobile ITS deployments with mobile edge computing (MEC) and task offloading approaches. Mobile edge computing research offers algorithmic tools for managing latency and energy in mobile applications, including educational software. Song et al. propose energy- and latency-aware task assignment schemes for “green” cloudlets, showing that careful task placement across local devices and edge servers can reduce energy consumption while satisfying delay constraints. Zhang et al. present a latency-aware offloading mechanism for MEC that dynamically migrates tasks across edge servers to reduce completion and waiting time compared to basic scheduling strategies. Other work in mobile edge computing similarly formulates offloading as an optimization problem that balances energy and delay under capacity constraints.

Study	Method	Objective	ITS-Specific?	Key Limitation	Relevance
Song et al. (2014) [64]	Energy-aware task assignment	Energy + delay	No	No educational objectives	Energy cost model inspiration

Pereira et al. (2023) [72]	Rule-based gamified mobile ITS	Engagement + learning	Yes	On-device only; no learned latency/energy control	Exemplifies current mobile ITS progress; motivates learned resource management
Zhang et al. (2025) [35]	Latency-aware offloading	Latency	No	No learning metrics	Offloading decision structure
Hortelano et al. (2023) [66]	Survey of RL offloading	Latency + energy	No	Generic QoS only	DRL offloading methodology
Hybrid POMDP (this research)	Hybrid POMDP + DQN	Learning + latency + energy + modality	Yes	Synthetic student model	-

Table 3.3: Comparison of Resource-Aware and Mobile Computing Approaches

Despite these advances, most MEC and offloading strategies are application-agnostic: they optimize generic QoS metrics without modeling domain-specific objectives such as learning gain, curriculum adherence, or engagement. In the ITS literature itself, a growing number of systems treat mobile hardware and network variability as first-class design concerns, but they do so with lightweight, rule-based resource management rather than learned optimization. For example, gamified mobile tutors for electrocardiogram interpretation run entirely on-device to avoid network dependence [72], and dialogue-based Arabic-language tutors combine local and server-side processing to remain usable under unstable connectivity [73]; a 2025 synthesis of this literature confirms that reinforcement-learning-based resource management remains rare in deployed mobile ITS precisely because of its data and computation requirements [71]. Detailed,

learned models of per-step latency, energy consumption, and modality validity on heterogeneous mobile devices remain rare in this body of work. No reviewed system, rule-based or learned, combines both dimensions in a single trained policy on a real mobile runtime, which is the precise gap Chapter 4 formalizes.

3.4 Transfer Learning and Cross-Context Generalization in Education

Transfer learning has been studied in several strands of ITS research, as listed in Table 3.4. At the student-model level, transferable performance models attempt to address cold-start problems when new courses are introduced [49]. For example, recent work on transferable student performance modeling shows that course-agnostic models trained on large multi-course datasets can be adapted to new courses with limited data, achieving predictive accuracy on par with traditional knowledge-tracing methods trained on thousands of in-course interactions.

Study	Transfer Type	Source → Target	Key Finding	Limitation	Relevance
Lan et al. (2022) [49]	Student model transfer	Course → Course	Cross-course model competitive with in-domain	Content-level only	Motivates policy-level transfer
Park & Gupta (2020) [50]	Policy transfer (RL)	Task → Task	Adaptive blending avoids negative transfer	Control benchmarks only	Transfer mechanism design

Ilhan et al. (2021) [67]	Policy reuse (action advising)	Teacher policy → Student policy	Well-chosen advisor reduces sample complexity	Same domain (Atari control tasks)	Action-advising as a transfer form
Hybrid POMDP (this research)	Deployment context transfer	Device+Network → Device+Network	100.5% zero-shot ratio; 19.43 episodes saved	Synthetic runtime	-

Table 3.4: Transfer Learning Approaches Relevant to Mobile ITS Deployment

At the policy level, adaptive policy-transfer methods aim to speed up learning by reusing policies from related tasks while avoiding negative transfer. Park and Gupta propose an adaptive policy-transfer scheme for deep RL that automatically learns when to follow a source policy and when to prioritize exploration, yielding faster convergence in multiple continuous-control benchmarks [50]. Similar ideas are used in action-advising frameworks where a teacher occasionally recommends actions to a student learner; empirical results show that well-chosen advisors can significantly reduce sample complexity.

Compared to these works, the transfer contributions in this research are focused on deployment contexts rather than curriculum content: policies trained under one combination of network scenario and device class are reused in others, and experiments quantify both convergence speedups and zero-shot performance across a grid of mobile environments. No prior ITS study reports such systematic cross-context policy transfer on a real mobile runtime.

A key distinction separates the transfer contributions of this research from prior ITS transfer work: existing approaches transfer between curriculum domains (e.g., mathematics to literacy) or between student populations, while this research transfers between deployment contexts meaning different combinations of device class and network scenario, with the curriculum held fixed. This form of transfer is relevant to real-world mobile ITS deployment, where a system trained in a high-resource lab environment must be deployable to low-resource field devices without retraining.

Across all four themes, the literature reveals a consistent boundary: pedagogical optimization and mobile quality-of-experience optimization are treated as separate engineering concerns, addressed by different communities with different tools. Mobile ITS research has closed part of this gap with rule-based and offline-first resource management [71][72], and MEC research has closed the other part with learned latency-energy optimization for non-educational workloads [35][36][66]. However, among the mobile ITS and MEC-for-education studies reviewed above, none trains a single controller to jointly optimize learning, latency, energy, and modality validity on the actual mobile runtime, and we are not aware of any prior study that reports systematic cross-context policy transfer across heterogeneous network and device pairs for mobile ITS.

3.5 Motivation for the Proposed Framework

The literature reviewed in Chapter 2 and the related work surveyed in Chapter 3 motivate the specific design choices adopted in this research. To capture the emotional dimensions of mobile learning discussed in Section 2.2, this research adopts the Ortony-Clore-Collins (OCC) model as the theoretical foundation for computational affect modeling: emotion categories from the OCC framework are operationalized in the ITS affect module as scalar valence signals updated after each interaction, which then modulate the pedagogical reward function detailed in Chapter 5.

Building on the Knowledge Space Theory foundations described in Section 2.3, this research represents curriculum structure using a KST-derived graph of approximately forty skills with directed prerequisite edges, restricting skill selection to the outer fringe of the learner’s current knowledge state. This design is detailed further in Section 5.2.

Finally, motivated by the transfer-learning gap identified in Section 2.7 and Section 3.4 and formalized in Section 2.8, the experiments in this research evaluate zero-shot performance and convergence speed when hybrid POMDP policies are transferred across heterogeneous network-device combinations, rather than across curriculum domains as in prior ITS transfer work. This focus reflects the practical reality that mobile ITS deployments must generalize across diverse hardware and connectivity conditions rather than across subject-matter domains.

Together, these design choices directly address the research gap formalized in Section 2.8 and position the contributions of this research, detailed in Chapter 4, against the prior work surveyed in this chapter.

CHAPTER 4

PROBLEM STATEMENT AND CONTRIBUTIONS

Although recent mobile ITS have progressed on offline-capable, rule-based delivery under resource constraints [71][72][74], prior frameworks have not jointly modeled device-level constraints such as battery, CPU load, memory pressure, thermal state, and mobile network variability alongside pedagogical objectives such as learning gain, curriculum adherence, engagement, and affect within a single learned decision-making model [10][11][35][36]. Mobile avatar-based ITS must therefore make pedagogical decisions under two forms of uncertainty simultaneously: uncertainty about the learner’s true knowledge and affective state, and uncertainty about the mobile execution environment in which tutoring takes place. Existing ITS methods usually address the first problem while assuming stable computational resources. Mobile edge computing frameworks address the second problem, but they optimize generic quality-of-service metrics without domain-specific educational objectives like learning gain or curriculum adherence.

This research addresses the following core problem: *Given a hierarchical curriculum, a partially observable learner state, and a heterogeneous mobile execution environment, can a single reinforcement learning controller jointly optimize pedagogical effectiveness and mobile quality of experience for avatar-based tutoring?*

This problem is challenging because it requires simultaneously reasoning over two distinct forms of partial observability. We address them jointly through a hybrid POMDP-DQN framework, which represents the core AI contribution of this research.

To convert the research gap into a set of testable claims, this research is organized around five research questions. These questions define what the proposed hybrid framework must

achieve in order to constitute a meaningful contribution at the intersection of ITS, reinforcement learning, and mobile edge computing.

RQ1. Learning Preservation and Improvement. Can a hybrid POMDP-based policy preserve or improve pedagogical effectiveness relative to a pedagogy-only baseline and heuristic baselines that do not jointly optimize mobile quality of experience?

RQ2. Latency and Energy Efficiency. Does the hybrid approach produce statistically significant and practically meaningful reductions in latency and energy-related cost when compared with always-rich and rule-based baselines?

RQ3. Robustness Across Devices and Networks. Is the learned policy robust across heterogeneous device classes and degraded network conditions, maintaining stable learning outcomes under changing mobile constraints?

RQ4. User Experience Consistency. Does the system provide a consistent mobile tutoring experience, measured through step-level latency compliance with a real-time threshold and stable learning performance across the sweep of pedagogy-efficiency trade-off settings?

RQ5. Transfer Across Deployment Contexts. Can a policy learned in one device-network context transfer to another with acceptable zero-shot performance and reduced convergence time?

Together, these questions provide the evaluative structure for the thesis. They also clarify that the contribution of this work is not only the proposal of a new hybrid architecture, but the empirical demonstration that an AI-driven decision policy can jointly optimize instructional and mobile performance objectives on the same deployment runtime used by the tutoring application. This research makes five primary contributions.

Contribution 1: A hybrid POMDP formulation for mobile avatar-based tutoring. The thesis formulates mobile tutoring as a hybrid sequential decision problem in which learner state uncertainty and device-context uncertainty are modeled jointly. The formulation combines a Knowledge Space Theory curriculum representation, belief-based learner modeling, and mobile execution context within a unified state space.

Contribution 2: An AI-driven control policy based on modified Deep Q-Learning. The thesis proposes a modified Deep Q-Network controller that learns to jointly select a pedagogical action, compute venue, and interface modality. This makes the proposed method explicitly AI-centered

and aligns the work with reinforcement-learning-based adaptive decision making rather than fixed heuristic scheduling.

Contribution 3: An end-to-end implementation on the tutoring system runtime. The hybrid controller is implemented within the Dart/Flutter runtime used by the tutoring application itself. This reduces the gap between algorithm design and deployment and strengthens the technical soundness of the evaluation.

Contribution 4: A large-scale empirical evaluation of learning-performance trade-offs. The thesis evaluates the proposed framework across 360,000 non-transfer episodes, multiple device classes, multiple network scenarios, eleven fixed trade-off settings, an adaptive setting, and four baselines. The evaluation uses formal statistical testing and directly measures learning gain, latency, energy-related cost, modality appropriateness, and transfer behavior.

Contribution 5: An analysis of where joint optimization succeeds and where it remains incomplete. The thesis shows that joint optimization can improve learning and latency simultaneously, while also identifying two unresolved trade-offs: higher energy consumption than the pedagogy-only baseline and lower-than-target modality appropriateness under stress. By tracing both partial results to reward-shaping imbalance, the thesis provides a technically grounded explanation of current limits and a clear path for future improvement.

Taken together, these contributions support the central claim of the thesis: a single AI-based policy can coordinate instructional adaptation and mobile resource adaptation more effectively than fixed-rule baselines, while also revealing the design conditions under which such a controller succeeds or fails.

Unlike heuristic mobile adaptation schemes that use manually specified thresholds, the proposed method is fundamentally AI-based: the system learns a control policy from interaction data using a modified Deep Q-Network. In this sense, the contribution of the thesis is not only a systems integration of ITS and mobile computing components, but the design of a reinforcement learning controller that treats pedagogical adaptation and computational adaptation as a single learning problem.

CHAPTER 5

PROPOSED HYBRID POMDP-DQN FRAMEWORK

5.1 Proposed Framework

5.1.1 Architectural overview

The hybrid POMDP framework is designed as a general decision-making architecture for adaptive tutoring under resource constraints, independent of any specific programming language or runtime. Three constraints govern the architecture. First, the policy must reason over partially observable learner state: a student’s true mastery level is never directly measurable and must be inferred from observable responses. Second, device and network context must be treated as primary inputs to every decision, not post-hoc filters applied after an instructional choice is made. Third, all decision logic must be able to run within the memory and latency budget of a resource-constrained mobile device, since offloading the policy would introduce the very latency it is meant to reduce.

These three constraints motivate the three-layer architecture shown in Figure 5.1. The domain layer encodes curriculum structure as a KST directed acyclic graph, maintains the outer fringe of admissible skills, and exposes a skill-selection interface to the decision layer. The decision layer is the POMDP-DQN controller: it receives the factored state vector which concatenates the pedagogical belief and the device telemetry, executes a single DQN forward pass to select one of the abstract actions, and translates that action into a (pedagogical move, offloading action, interface modality) triple. The resource layer receives the triple, retrieves the appropriate instructional content (question, hint, or worked solution), routes computation to the selected venue, and triggers avatar rendering at the selected modality. Feedback from the learner

(correctness, response time, affect signals) flows back through the resource layer to update the belief state and compute the step reward before the next decision cycle begins.

Figure 5.1 organizes the proposed system into three cooperating layers connected by a closed feedback loop. The domain layer stores the Introductory Algebra curriculum as a KST directed acyclic graph, maintains the learner’s mastery set, and computes the outer fringe of admissible skills. The decision layer receives a factored state formed by concatenating the partially observable pedagogical belief with fully observable device and network telemetry, then uses a DQN to select an abstract tutoring action that is decoded into a pedagogical move, a compute venue, and an interface modality. The resource layer retrieves the corresponding instructional content, routes computation to the selected venue, and renders the selected modality to the learner. Learner feedback (correctness, response time, and affect signals) and updated device telemetry then return through the resource layer so that the belief state, mastery set, and step reward can be updated before the next decision. In this way the domain layer constrains what may be taught, the decision layer chooses how to teach under resource constraints, and the resource layer executes that choice and returns the observations needed for learning.

Algorithm 5.1 is the single per-step control procedure that realizes Figure 5.1 at runtime. It first calls the domain layer to compute the current outer fringe from the KST graph and mastery set. It then forms the joint state, selects an action with the current policy (restricted to the fringe), and decodes that action into a move–venue–modality triple. Control then passes to the resource layer, which executes the tutoring step and returns the learner observation together with the next device state. The decision layer uses those signals to compute the hybrid reward and to update the pedagogical belief and mastery set, after which the next joint state is formed. During offline training, the resulting transition is stored in the replay buffer for DQN updates; at evaluation and deployment the same loop runs with a fixed policy and without buffer updates. The remainder of this chapter details each layer in turn: Section 5.2 on domain ontology with KST, Section 5.3 on hybrid decision making, Section 5.4 on POMDP policy with Q-learning and Section 5.5 on resource management and scaffolding using Algorithm 5.1 as the shared control-flow reference.

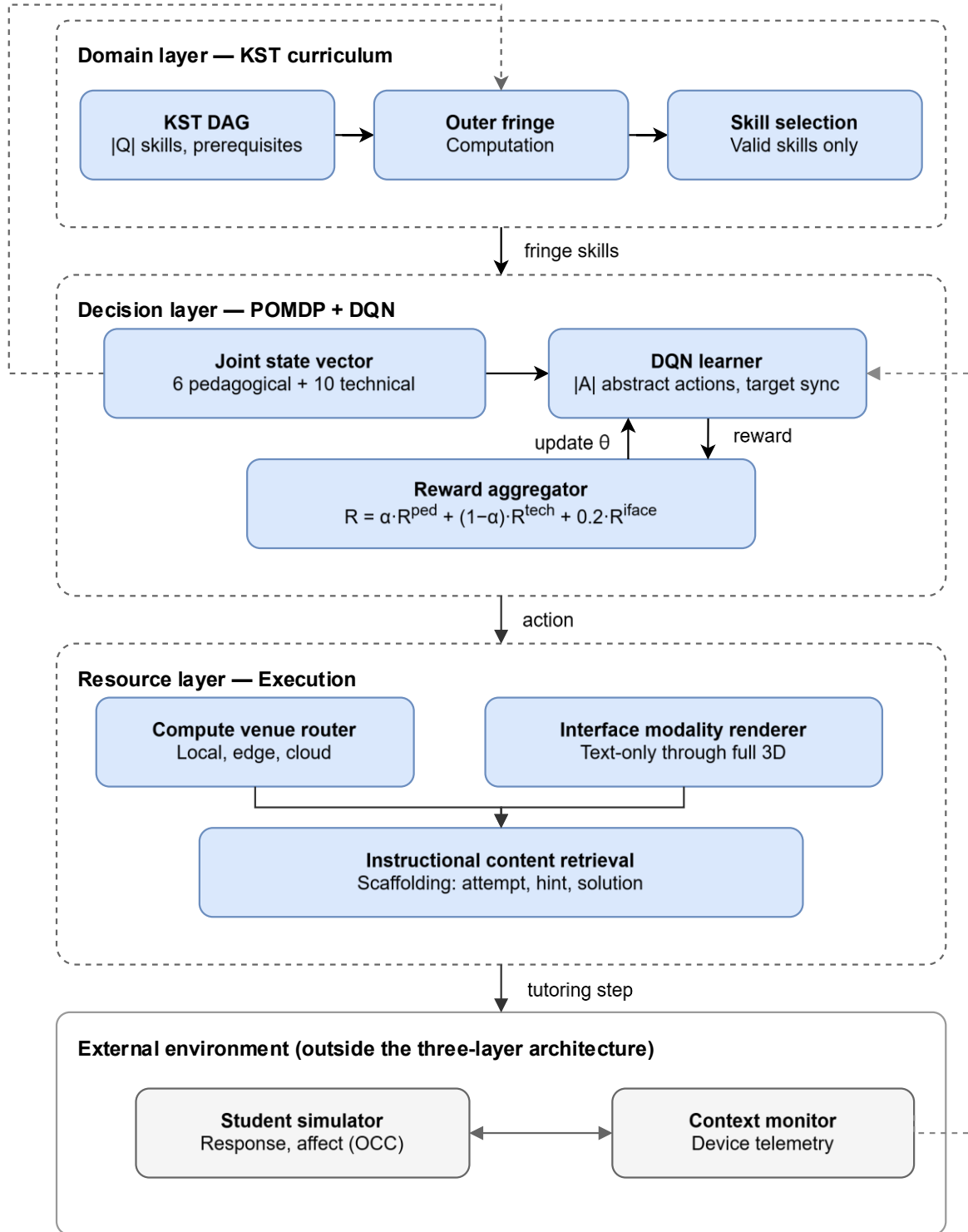


Figure 5.1: The domain layer models the KST curriculum and admissible skills; the decision layer selects actions from the factored state using DQN; the resource layer routes computation and rendering across device, edge, or cloud; and the environment feedback loop updates beliefs and telemetry from learner responses and affect

Algorithm 5.1: Hybrid POMDP–DQN Control Flow (Three-Layer Architecture)

Input: belief b_t , device state d_t , KST DAG G , mastery set M_t , policy weights θ

Output: next belief b_{t+1} , next mastery set M_{t+1} , executed tutoring step, transition tuple (for training)

1: $F_t \leftarrow \text{ComputeOuterFringe}(G, M_t)$	// Domain Layer - Algorithm 5.2
2: $x_t \leftarrow [b_t; d_t]$	// Decision Layer, Stage 1 (Section 5.3.1)
3: $a_t \leftarrow \text{SelectAction}(x_t, F_t, \theta)$	// Decision Layer, Stage 2 (Algorithm 5.3 during training)
4: $(\text{move}_t, \text{venue}_t, \text{modality}_t) \leftarrow \text{Decode}(a_t)$	// Decision Layer, Stage 3
5: $\text{ExecuteResourceManager}(\text{move}_t, \text{venue}_t, \text{modality}_t)$	// Resource Layer - Algorithm 5.4
6: observe correctness o_t , affect signals; compute r_t	// Stage 4 (Section 5.3.3)
7: observe next device state d_{t+1}	
8: $b_{t+1} \leftarrow \text{BayesianUpdate}(b_t, o_t, \text{affect})$	// Stage 5 (Section 5.3.2)
9: $M_{t+1} \leftarrow \text{UpdateMastery}(M_t, b_{t+1})$	
10: $x_{t+1} \leftarrow [b_{t+1}; d_{t+1}]$	
11: if training then store (x_t, a_t, r_t, x_{t+1}) in replay buffer	// consumed by Algorithm 5.3
12: return b_{t+1}, M_{t+1}	

5.2 Domain Ontology with Knowledge Space Theory

5.2.1 Component structure

The domain ontology defines the conceptual structure of a subject domain, including topics, subtopics, skills, and relationships [78]. It is formalized as a KST-style DAG for use by the POMDP agent. Each node in the DAG corresponds to a knowledge component (KC) or small cluster of related KCs; edges encode prerequisite relationships derived from pedagogical design guidelines and, where available, empirical performance data.

This ontology serves three roles in the proposed framework: (i) it is the domain model that defines which skills exist and how they depend on one another; (ii) it provides the basis for computing the outer fringe of admissible skills at every decision point; and (iii) it defines the

mapping from belief states to learning resources used by the resource manager. Throughout this chapter, these roles are explained using one fixed domain ontology for Introductory Algebra (Figure 5.2), so that outer-fringe updates, belief updates, reward computation, and action selection all refer to the same subject-domain example.

5.2.2 KST graph and outer fringe

The KST graph is constructed as a finite set Q of skills and a family of admissible knowledge states closed under union, following Doignon and Falmagne’s formulation [22] [23]. In practice, the system uses a DAG representation whose nodes correspond to skills and edges encode prerequisite relations. The outer fringe of a KST graph refers to the set of skills whose incoming prerequisites are mastered but themselves are not yet mastered.

During live tutoring, the policy is restricted to select skills from this outer fringe, ensuring that instruction respects prerequisite structure and preventing the tutor from presenting material that is too advanced or conceptually inconsistent. This approach aligns with prior KST-based ITS designs that use the fringe as the candidate set for next-problem selection [24]. Figure 5.2 illustrates this operation with an example: an eight-skill domain ontology for the subject domain of Introductory Algebra, consisting of skills S1 (Basic Arithmetic Reasoning), S2 (Variable Identification), S3 (Logical Sequencing), S4 (Linear Equation Solving), S5 (Algebraic Manipulation of Expressions), S6 (Systems of Linear Equations), S7 (Quadratic Reasoning), and S8 (Function Composition), with prerequisite edges $S1, S2 \rightarrow S4$; $S2, S3 \rightarrow S5$; $S4, S5 \rightarrow S6$; $S6 \rightarrow S7$; and $S6, S7 \rightarrow S8$. Because S1, S2, and S3 have no prerequisites, they form the initial outer fringe, while S4 through S8 are locked. Once S1 and S2 are mastered, S4 enters the fringe; once S2 and S3 are mastered, S5 also enters the fringe; S6, S7, and S8 remain locked until their own prerequisite chains are satisfied in turn. If a learner has mastered S1 through S5, the outer fringe contains only S6, so the policy is restricted to selecting S6 next even though five of the eight skills are already mastered. This fringe-constrained action selection guarantees that, at every decision point, the policy chooses only among skills whose prerequisites are already satisfied and never among already-mastered or still-locked skills.

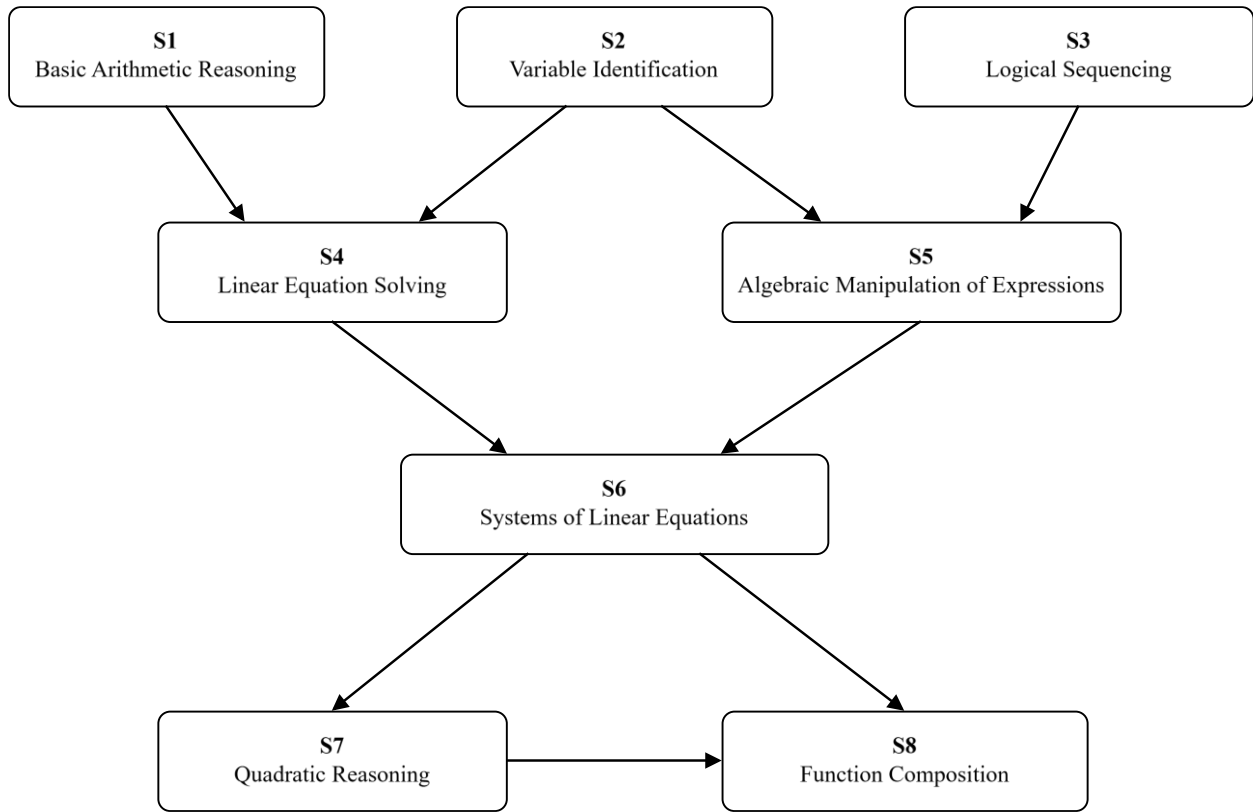


Figure 5.2: Eight-skill KST DAG for the Introductory Algebra running example. Prerequisite edges are $S1, S2 \rightarrow S4$; $S2, S3 \rightarrow S5$; $S4, S5 \rightarrow S6$; $S6 \rightarrow S7$; and $S6, S7 \rightarrow S8$. Skills with no prerequisites ($S1$ – $S3$) form the initial outer fringe.

Algorithm 5.2 performs the outer-fringe computation illustrated by Figure 5.2: given the KST DAG and the current mastery set, it returns exactly the admissible skills whose prerequisites are satisfied and that are not yet mastered.

Algorithm 5.2: Outer Fringe Computation (Domain Layer)

Input: KST DAG $G = (Q, E)$; mastery set $M \subseteq Q$

Output: outer fringe $F \subseteq Q$

```
1:  $F \leftarrow \emptyset$ 
2: for each skill  $s_i \in Q \setminus M$  do
3:   if all prerequisites of  $s_i$  (per  $E$ ) are in  $M$  then
4:      $F \leftarrow F \cup \{s_i\}$ 
5: return  $F$                                      // passed to Algorithm 5.1, line 1
```

Against the Introductory Algebra example, lines 2–4 admit a skill such as S4 only when both S1 and S2 are in M , and admit S6 only when both S4 and S5 are in M ; line 5 returns that fringe F to Algorithm 5.1. Skill selection is then restricted to this fringe during action selection in Algorithms 5.1 and 5.3: the skill component of a_t is always chosen from F_t , never from already-mastered or still-locked skills.

5.3 Hybrid Decision Layer

5.3.1 Control Flow and Belief Update

The hybrid POMDP decision layer operates as follows. At time t , the system maintains a belief b_t over hidden pedagogical variables (including true mastery, affect) and observes technical variables d_t (including battery, CPU, network, environment). The belief is updated using Bayesian rules based on correctness and affective signals, while the technical state is directly measured on-device during live tutoring; A simulated technical-state generator is used first for the offline training and then during evaluation experiments.

Given b_t and the current technical context d_t , the DQN selects an action a_t . The environment then produces a new observation and rewards, and the belief is updated to b_{t+1} . Unlike POMDP-based dialogue managers, which update belief over user goals alone, the hybrid state includes device and network features whose values directly shape both QoE outcomes and avatar modality decisions.

The control flow proceeds in five stages:

1. **State observation.** The device runtime reports the fully observable technical state d_t , a ten-component vector $d_t = [B_t, N_t, P_t, \mu_t, L_t, \text{connection type}, T_t, \text{GPS signal quality}, \text{ambient noise level}, \text{motion intensity}]$ consisting of battery level B_t , network quality N_t , CPU load P_t , memory pressure μ_t , network latency L_t , connection type, thermal state T_t , GPS signal quality, ambient noise level, and motion intensity. GPS signal quality and motion intensity are retained in the technical state because they enter the interface reward: high motion favors lighter modalities, and weak GPS penalizes rich 3D presentation. They are also included in the DQN state encoding. Meanwhile, the pedagogical or technical reward terms are concatenated with the partially observable pedagogical belief state b_t . They are maintained in a six-component vector $b_t = [k_t, C_t, E_t, G_t, \text{current KST skill node}, \text{mastery level}]$ consisting of the mastery belief vector k_t , cognitive load C_t , emotional state E_t , engagement G_t , current KST skill node, and mastery level. Together, the two vectors form the factored joint state vector $x_t = [b_t; d_t]$.
2. **Action selection.** The DQN evaluates $Q(x_t, a; \theta)$ for all $|A|$ abstract actions ($|A|$ is defined in Section 5.4.1) and selects $a_t = \operatorname{argmax}_a Q(x_t, a; \theta)$ during exploitation, or samples uniformly with probability ϵ during training.
3. **Action execution.** The resource manager decodes a_t into a (pedagogical move, compute venue, interface modality) triple, retrieves instructional content and delivers the tutoring step.
4. **Reward computation.** The scalar reward R_t is computed from the three components defined in Section 5.3.3: $R_t = \alpha R_t^{\text{ped}} + (1 - \alpha) R_t^{\text{tech}} + 0.2 R_t^{\text{iface}}$, where 0.2 is the fixed interface weight justified in that same section.
5. **Belief update.** The learner's response (correctness o_t , affect signals) is used to update b_{t+1} via the Bayesian rule given in Section 5.3.2. The transition tuple (x_t, a_t, R_t, x_{t+1}) is stored in the replay buffer for training.

Figure 5.3 illustrates the five-stage per-step control flow of the hybrid decision layer (Stages 1–5 above), with the input(s) and output(s) of each stage shown explicitly. Stage 1 forms the factored joint state $x_t = [b_t; d_t]$ by concatenating the six-component partially observable pedagogical belief b_t with the ten-component fully observable technical state d_t ; here “six-component” and “ten-component” count factored state groups, not raw scalar dimensions (in particular, the mastery belief k_t is itself an n -dimensional probability vector with $n = |Q|$). Stages 2–5 then

select a_t , execute the tutoring step, compute R_t , and update b_{t+1} . The composition of b_t and d_t is detailed in Stage 1 of Section 5.3.1 and in Algorithms 5.1–5.2.

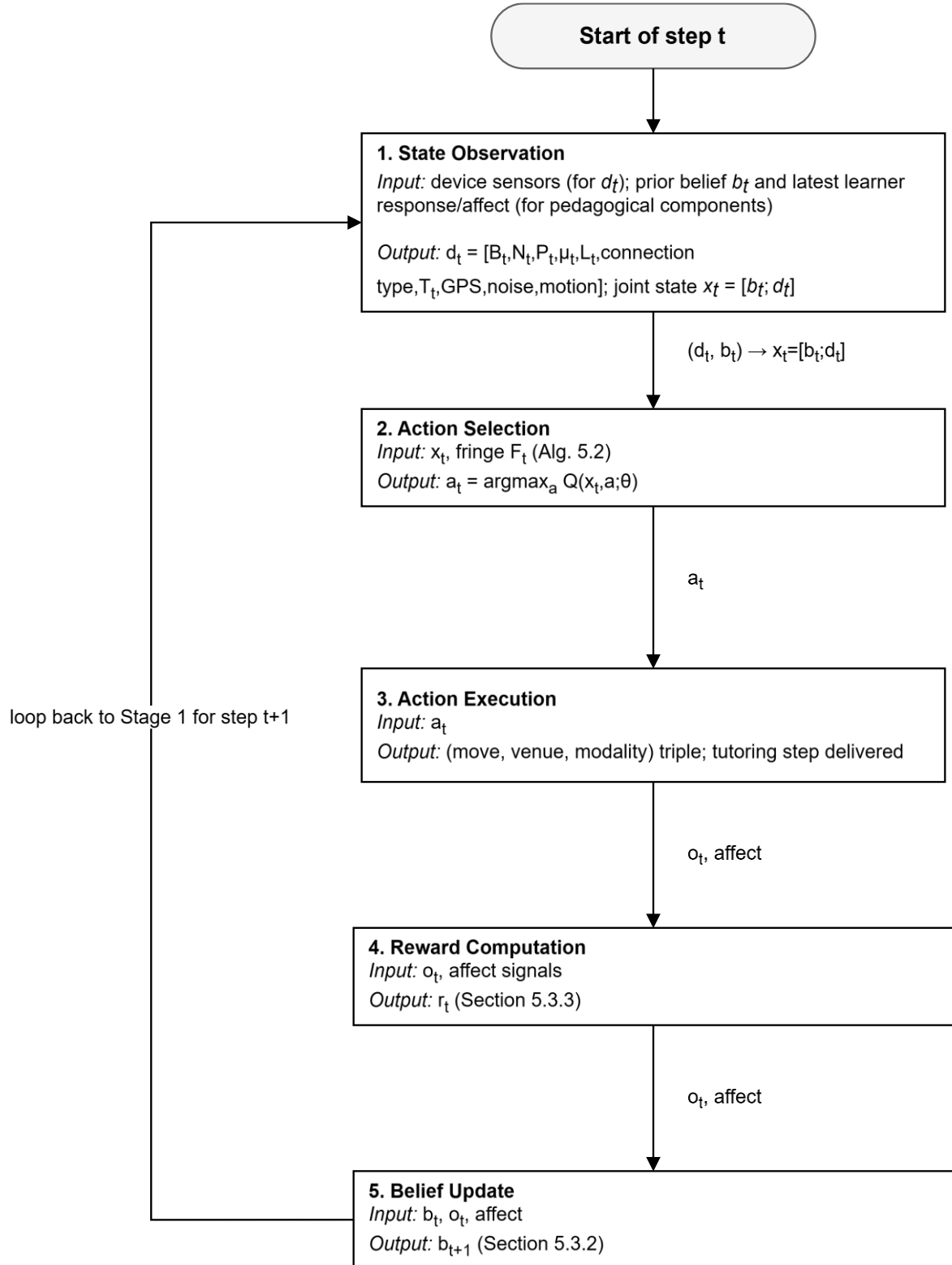


Figure 5.3: Per-step control-flow of the hybrid decision layer, showing the input(s) and output(s) of each of the five stages.

5.3.2 Belief states and mastery estimation

Belief over mastery is represented as an n -dimensional vector of probabilities, one per skill in the domain ontology, where $n = |Q|$ is the number of skills in the KST DAG and is updated at each step based on the learner's correctness, the difficulty of the item, and the current cognitive load estimate. The estimator uses a slip-and-guess model to sample correctness, in line with classical knowledge-tracing formulations, and updates the true mastery state with a learning rate modulated by engagement and motivation.

Table 5.1 illustrates the belief-update process for skill S4 from the running example introduced in Section 5.2.2, using an initial belief of $b(S4) = 0.10$, a step size of $\Delta = 0.15$, and a mastery threshold of 0.80 (the selection of these values is justified in Section 5.3.3):

Interaction	Observation	Updated Belief	Mastered?
1	Correct	$0.10 \rightarrow 0.25$	No
2	Incorrect	$0.25 \rightarrow 0.10$	No
3	Correct	$0.10 \rightarrow 0.25$	No
4	Correct	$0.25 \rightarrow 0.40$	No
5	Correct	$0.40 \rightarrow 0.55$	No
6	Correct	$0.55 \rightarrow 0.70$	No
7	Correct	$0.70 \rightarrow 0.85$	Yes (≥ 0.80)

Table 5.1: Belief value update and skill mastery determination example

Once $b(S4)$ reaches 0.80, S4 is marked as mastered in the KST DAG and the outer fringe is recomputed: mastering S4 moves S6 one prerequisite closer to entering the fringe (S6 still requires S5 before it becomes selectable, per Section 5.2.2). The practical significance of this update rule is that engagement and motivation act as multiplicative amplifiers on learning rate. An agent that preserves student engagement through appropriate modality and pacing choices generates compounding increases in belief-update magnitude over the episode budget. A student whose motivation is maintained near its upper bound throughout an episode accumulates more belief mass per correct response, on skills such as S4, than one whose motivation decays toward

its lower bound, because the multiplicative engagement and motivation terms in the belief-update rule scale every correct-response increment. This gap has a direct effect on cumulative learning gain and motivates the hybrid policy’s attention to modality selection and cognitive-load management. The size of this effect will be evaluated experimentally.

5.3.3 Reward function

The reward at each step can be written as:

$$R_t = \alpha R_t^{\text{ped}} + (1 - \alpha) R_t^{\text{tech}} + 0.2 R_t^{\text{iface}},$$

where $\alpha \in [0, 1]$ is the pedagogy–efficiency trade-off parameter and the interface weight **0.2** is fixed.

To express the three component rewards formally, define:

$c_t = 1$ if the student answers correctly at step t ; 0 otherwise.

$m_t = 1$ if the categorical mastery level increases at step t ; 0 otherwise.

$\Delta k_t = k_{t+1} - k_t$, the per-step knowledge delta.

$\Delta B_t = B_t - B_{t+1}$, the per-step battery drain.

Let $1[\cdot]$ denote the indicator function.

The three component rewards are then:

Pedagogical Reward:

$$R_t^{\text{ped}} = 10\Delta k_t + 5c_t - 1(1 - c_t) + 10m_t - 31[C_t > 0.8] + 2(E_t - 0.5) + 2(G_t - 0.5).$$

Where:

$\Delta k_t = k_{t+1} - k_t$ is the per-step knowledge delta,

$c_t = 1$ if the answer is correct and 0 otherwise,

$m_t = 1$ if the categorical mastery level increases at step t and 0 otherwise,

C_t is the cognitive-load estimate (the -3 penalty applies only when $C_t > 0.8$),

E_t is emotional valence, and

G_t is engagement.

The coefficients 10, 5, -1, 10, 3, and 2 are the fixed scales used in this reward function.
(knowledge-delta, correctness, incorrectness, mastery-level, cognitive-load, and affect shaping).

Technical Reward:

$$R_t^{\text{tech}} = -20\Delta B_t - \mathbf{1}[\text{lat}_t > 500] \cdot \frac{\text{lat}_t - 500}{100} + 2\mathbf{1}[\text{lat}_t < 200] - 3\mathbf{1}[N_t < 0.3] + \mathbf{1}[P_t < 0.5 \wedge \mu_t < 0.6].$$

Where:

ΔB_t is per-step battery drain,

lat_t is per-step latency in milliseconds,

N_t is network quality,

P_t is CPU utilization, and

μ_t is memory pressure.

Latency below 200 ms receives a bonus; latency above 500 ms incurs a continuous penalty of $(\text{lat}_t - 500)/100$. Poor network ($N_t < 0.3$) is penalized, and jointly low CPU and memory receive a small efficiency bonus.

Interface Reward:

$$\begin{aligned} R_t^{\text{iface}} = & -2 \cdot \mathbf{1}[\text{voiceOnly} \wedge A_t > 0.7] \\ & + 1.5 \cdot \mathbf{1}[\text{textOnly} \wedge (O_t > 0.7 \vee A_t > 0.7)] \\ & + 1.2 \cdot \mathbf{1}[2D \wedge (O_t > 0.7 \vee T_t > 0.6)] \\ & - 1 \cdot \mathbf{1}[\text{rich3D} \wedge (O_t > 0.7 \vee \text{GPS}_t < 0.3)], \end{aligned}$$

Where:

A_t is ambient noise,

O_t is motion intensity,

T_t is thermal state, and

GPS_t is GPS signal quality.

These terms reward lighter modalities under environmental stress and penalize rich 3D when motion is high or GPS is weak. Hard validity checks used for the modality-appropriateness metric (battery, memory, thermal, noise) are separate from R_t^{iface} .

Adaptive α . In fixed- α training and evaluation, $\alpha \in [0,1]$ is held constant for a configuration. In the adaptive- α setting, the effective trade-off is recomputed at each step by the following state-driven rule. Let α_t be initialized to 0.6. Then apply the corrections:

- if $B_t < 0.20$ or $T_t > 0.7$, then $\alpha_t \leftarrow \alpha_t - 0.2$;
- if $N_t < 0.3$ or $L_t > 800$ ms, then $\alpha_t \leftarrow \alpha_t - 0.1$;
- if knowledge $k_t < 0.45$ and cognitive load $C_t < 0.75$, then $\alpha_t \leftarrow \alpha_t + 0.15$;
- if engagement $G_t < 0.35$, then $\alpha_t \leftarrow \alpha_t + 0.05$.

Finally, clip α_t to the interval $[0.2, 0.85]$. The resulting α_t is substituted into

$$R_t = \alpha_t R_t^{\text{ped}} + (1 - \alpha_t) R_t^{\text{tech}} + 0.2 R_t^{\text{iface}}.$$

Thus adaptive α raises pedagogical priority when the learner still needs support and resources permit it, and lowers pedagogical priority when battery, thermal or network conditions are stressed. This is the state-driven adaptive- α mechanism used whenever the trade-off parameter is not held fixed.

Worked example.

Continuing the running example from Section 5.2.2, suppose the learner is working on S4 and answers correctly ($c_t = 1$) at the interaction where knowledge rises from $k_t = 0.70$ to $k_{t+1} = 0.85$, so $\Delta k_t = 0.15$, and the categorical mastery level increases ($m_t = 1$). Let $C_t = 0.3$ (below the overload threshold), $E_t = 0.6$, and $G_t = 0.7$. Then:

$$\begin{aligned} R_t^{\text{ped}} &= 10(0.15) + 5(1) - 1(0) + 10(1) - 3(0) + 2(0.6 - 0.5) + 2(0.7 - 0.5) \\ &= 1.5 + 5 + 10 + 0.2 + 0.4 \\ &= 17.1. \end{aligned}$$

If this step has latency $\text{lat}_t = 250$ ms, battery drain $\Delta B_t = 0.05$, network quality $N_t = 0.8$, CPU load $P_t = 0.4$, and memory pressure $\mu_t = 0.3$, then:

$$\begin{aligned} R_t^{\text{tech}} &= -20(0.05) - 0 + 0 - 0 + 1 \\ &= -1 + 1 \\ &= 0. \end{aligned}$$

(Latency is neither less than 200 nor larger than 500, network quality is no less than 0.3, and both CPU and memory are in the efficient region, so the efficiency bonus of +1 applies.)

If the selected modality is rich 3D with motion $O_t \leq 0.7$ and GPS ≥ 0.3 , then $R_t^{\text{iface}} = 0$.

With $\alpha = 0.6$,

$$R_t = 0.6(17.1) + 0.4(0) + 0.2(0) = 10.26.$$

Formally, let θ_i denote the i -th tunable parameter with candidate set C_i , and let $J(\theta)$ be the mean learning gain subject to mean per-step latency ≤ 500 ms and feasible on-device memory be the validation objective. Each parameter is selected as $\theta_i^* = \operatorname{argmax}_{c \in C_i} J(\theta_1^*, \dots, \theta_{i-1}^*, c, \theta_{i+1}, \dots)$, holding previously calibrated parameters fixed, evaluated on a held-out preliminary episode set. This mirrors the search-and-select principle used in prior work by Duong [53] (implemented there via Optuna over pedagogical reward terms), adapted here to a manual, multi-objective calibration because the hybrid reward additionally contains latency, energy, and interface terms that Duong’s single-objective search did not need to balance.

Parameter Selection. The same calibration rule used in [53] applies to every fixed number in Table 5.2 and Algorithm 5.3. Candidate sets, retained values, and role-based reasons are reported in Table 5.2 and Appendix Table B.4; no unexplained default is used in the proposed method.

For the reward parameters in Table 5.2, each value follows the role defined above. The interface coefficient is held fixed at 0.2 so that pedagogy–efficiency trade-offs are controlled by α . The mastery-level bonus (+10) is larger than the correctness bonus (+5) so skill progression is salient. Knowledge gain is scaled by 10 so small belief deltas remain visible in the reward. Cognitive overload is applied only above $C_t > 0.8$ rather than as a continuous linear penalty. Technical shaping uses a continuous high-latency cost above 500 ms, a bonus below 200 ms, a strong battery coefficient (20), and a binary efficiency bonus for low CPU/memory. Interface shaping rewards lighter modalities under motion/noise/thermal stress and penalizes rich 3D under motion or weak GPS. Under this procedure, every fixed number quoted in this chapter is the value actually used to train and evaluate the hybrid controller. For the belief-update parameters marked illustrative in Table 5.2, $\Delta = 0.15$ and the mastery threshold 0.80 are didactic values used only in the Table 5.1 running-example walkthrough; they are not the belief-update rule used in the experimental evaluation. In evaluation, belief updates are affect-modulated (approximately +0.08 on correct and −0.03 on incorrect responses, scaled by engagement, motivation, and frustration), and episode-level mastery uses a higher belief threshold (0.85). For the DQN hyperparameters in Algorithm 5.3 (γ , ε_0 , $\varepsilon_{\min}$, decay, replay-buffer capacity C , minibatch size B , target-sync interval τ , and warm-up size W), values follow standard deep-RL

practice and were retained only if training remained stable within a resource-constrained mobile memory budget (Table B.1). Under this procedure, every non-illustrative fixed number quoted for the hybrid controller is traceable to an explicit role-based justification and, where applicable, a validation comparison in Table 5.2 or Appendix Table B.4, rather than to an unexplained default.

Figure 5.4 decomposes the total reward into its pedagogical, technical, and interface components, and Table 5.2 summarizes the weight parameters in the reward function together with the values retained from the structured empirical calibration described above.

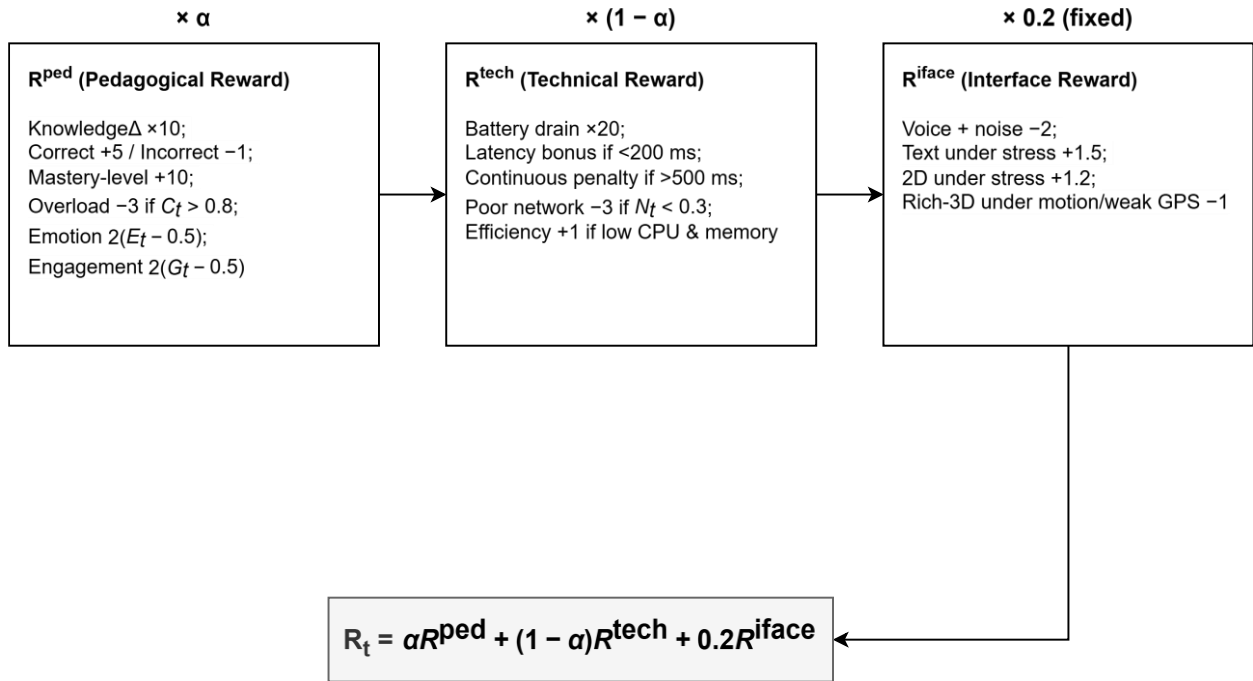


Figure 5.4: Reward function decomposition. The total reward is a weighted sum of pedagogical, technical, and interface components with interface weight fixed at 0.2.

Parameter	Symbol	Value	Role
Knowledge-delta scale	$10 \cdot \Delta k_t$	10	Scale learning progress
Correctness bonus	if correct	+5	Reward correct answers
Incorrectness penalty	if incorrect	-1	Small cost for errors
Mastery-level bonus	if mastery level $\uparrow$	+10	Salient skill-progress signal
Cognitive-load penalty	if $C_t > 0.8$	-3	Penalize overload
Emotion shaping	$2(E_t - 0.5)$	2	Favor positive affect
Engagement shaping	$2(G_t - 0.5)$	2	Favor engagement
Battery drain coeff.	$20 \cdot \Delta B_t$	20	Energy cost
Low-latency bonus	if $\text{lat}_t < 200$ ms	+2	Reward interactive response
High-latency penalty	if $\text{lat}_t > 500$ ms	$\frac{-(\text{lat}_t - 500)}{100}$	Continuous QoE cost
Poor-network penalty	if $N_t < 0.3$	-3	Penalize bad connectivity
Efficient-resource bonus	if $P_t < 0.5$ and $\mu_t < 0.6$	+1	Reward light load
Voice+noise penalty	voiceOnly & $A_t > 0.7$	-2	Avoid voice in noise
Text in stress bonus	textOnly & (motion/noise)	+1.5	Prefer text when stressed
2D in stress bonus	2D & (motion/thermal)	+1.2	Prefer 2D when stressed
Rich-3D stress penalty	rich3D & (motion/weak GPS)	-1	Discourage rich 3D when unstable
Interface weight (fixed)	-	0.2	Independent of α
Adaptive α base	-	0.6	Default before corrections
Adaptive α clip	-	[0.2, 0.85]	Bound α extremes

Belief step size (illustrative)	Δ	0.15	Per-correct / per-incorrect didactic update in Table 5.1
Mastery threshold (illustrative)	-	0.80	Belief value at which a skill is marked mastered in Table 5.1

Table 5.2: Reward Function and Belief-Update Parameters and Their Roles.

5.4 POMDP Action Policy with Q-Learning

5.4.1 Action representation and abstraction

The underlying action space consists of a small set of pedagogical moves, a small set of offloading actions and a small set of interface modalities yielding a large conceptual cross-product when combined with the $|Q|$ skills in the domain ontology. Offloading actions refine three compute venues (local, edge, cloud) into concrete routing options; the venues are the coarse execution locations, and the offloading actions are the fine-grained choices used by the policy. To make learning feasible on mobile hardware, the framework collapses these combinations into a smaller set of $|A|$ representative abstract actions and treats skill selection as a separate decision.

This factorization mirrors techniques used in tractable POMDP representations, where complex domains are decomposed into manageable macro-actions without losing essential structure. The $|A|$ abstract actions are constructed by clustering the full combinatorial space of (pedagogical move, offloading action, interface modality) triples into representative combinations based on computational cost and pedagogical intent: high-cost combinations (e.g., cloud offloading with a rich 3D avatar) cluster with high-pedagogy moves, while low-cost combinations (e.g., local computation with a text-only interface) cluster with maintenance and scaffolding moves. Skill selection is separated into a secondary output because the choice among the $|Q|$ skills is driven by the curriculum, constrained to the outer fringe, and does not interact strongly with the offloading-action decision (that is, which compute venue and quality/fallback variant to use). This factorization reduces the Q-table from the full joint-action space of $|moves| \times |offloading\ actions| \times |modalities| \times |Q|$ combinations to $|A| \times |Q|$ leaf-level decisions, making the policy learnable within a bounded training budget on a resource-constrained mobile device.

Continuing the Introductory Algebra running example from Section 5.2.2, the underlying factors are six pedagogical moves, nine offloading actions, and four interface modalities. These nine actions are the quality and fallback variants of those three venues (for example, full vs reduced rendering, local 2D fallback, hybrid split, and text-only). Combined with $|Q| = 8$ skills, the full joint space has size $6 \times 9 \times 4 \times 8 = 1,728$. The framework therefore collapses representative (pedagogical move, offloading action, modality) combinations into a smaller abstract action set of size $|A| = 36$ (Appendix Table B.1) and keeps fringe-constrained skill selection separate.

5.4.2 Q-learning and policy training

The DQN is trained using standard Q-learning with experience replay. Each training episode consists of T tutoring steps; after each step, the tuple (x_t, a_t, R_t, x_{t+1}) is stored in a replay buffer of fixed capacity C , and the network is updated by sampling random minibatches of size B . A separate target network, synchronized every τ gradient steps, is used to stabilize training, following deep RL best practices. Formally, the network is updated by minimizing the mean-squared Bellman error:

$$L(\theta) = E_{(x,a,r,x') \sim D} [(r + \gamma \max_{a'} Q(x', a'; \theta^-) - Q(x, a; \theta))^2]$$

where θ are the current network weights, θ^- are the target network weights, D is the replay buffer, $\gamma \in [0,1]$ is the discount factor, and mini-batches are sampled uniformly. Exploration follows ϵ -greedy decay from an initial rate ϵ_0 to a floor $\epsilon_{\min}$ at a fixed per-episode decay rate, with a warm-up buffer of W transitions required before gradient updates begin. Section 5.3.3 describes how γ , ϵ_0 , $\epsilon_{\min}$, the decay rate, the learning rate, C , B , τ and W are each justified via the structured empirical calibration described there, rather than left as unexplained defaults. Per experimental configuration (trade-off setting, device class, network scenario, and random seed), the agent is trained for a fixed number of episodes and then evaluated for a fixed number of exploitation-only episodes with exploration disabled. This fixed train/evaluate schedule supports robust statistical comparison across baselines and conditions. The DQN architecture is a compact multi-layer perceptron with two hidden layers and an output layer of size $|A|$, sized to execute inference within the memory and latency budget of a resource-constrained mobile device.

Algorithm 5.3 shows how Algorithm 5.1’s control loop is executed repeatedly across episodes to train θ offline; at evaluation and deployment time, the same control loop (Algorithm 5.1) runs with a fixed θ and no replay-buffer updates.

Algorithm 5.3: Hybrid POMDP-DQN Training

Input: POMDP $\mathcal{M}$, curriculum G , student simulator Sim, initial weights θ , target weights $\theta^- \leftarrow \theta$, replay buffer $\mathcal{D} = \emptyset$

Parameters: $\gamma, \varepsilon_0, \varepsilon_{\min}, \text{decay}, C, B, \tau, W, E_{\max}, T$ (fixed via the calibration procedure described in Section 5.3.3)

Output: Trained policy θ^*

```

1: for episode  $e = 1$  to  $E_{\max}$  do
2:   Initialize belief  $b_0$ , observe device state  $d_0$ , form  $x_0 = [b_0; d_0]$ 
3:   for step  $t = 1$  to  $T$  do
4:     With probability  $\varepsilon$ : select  $a_t$  uniformly from valid actions in  $\text{fringe}(K_t)$ 
5:   else:  $a_t \leftarrow \arg\max_a Q(x_t, a; \theta)$ 
6:   Execute  $a_t = (a_t^{\text{ped}}, a_t^{\text{venue}}, a_t^{\text{mod}})$ ; observe  $o_t$ , compute  $R_t$ , observe  $d_{t+1}$ 
7:   Update belief  $b_{t+1}$  via Bayesian rule; form  $x_{t+1} = [b_{t+1}; d_{t+1}]$ 
8:   Store transition  $(x_t, a_t, R_t, x_{t+1})$  in  $\mathcal{D}$ 
9:   if  $|\mathcal{D}| \geq W$  then
10:    Sample minibatch of size  $B$  from  $\mathcal{D}$ 
11:    Compute targets:  $y_j = R_j + \gamma \cdot \max_{a'} Q(x_{j+1}, a'; \theta^-)$ 
12:    Update  $\theta$  via Adam on  $L(\theta) = (1/B) \sum_j (y_j - Q(x_j, a_j; \theta))^2$ 
13:   if  $t \bmod \tau = 0$  then  $\theta^- \leftarrow \theta$ 
14:    $\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon \cdot \text{decay})$ 
15: return  $\theta^*$ 

```

Worked example (tabular Q-learning update). Continuing the Introductory Algebra running example, suppose the learner has mastered S1–S5, so the outer fringe is $F_t = \{S6\}$. For this didactic walkthrough we use a tabular Q-learning view of the same Bellman update that Algorithm 5.3 implements with a DQN (Eq. (3) style / the target in Algorithm 5.3, lines 11–12). Abstract actions are indexed $a \in \{1, \dots, 36\}$ with $|A| = 36$ (Appendix Table B.1). To keep the

table readable, Table 5.3 shows only four representative actions for a coarse belief bin of the factored state.

Assume the agent previously executed action $a = 4$ in belief-bin state $s = 5$ (bin 0.4–0.5 on the relevant mastery component for S6), and the successor state after the learner response is bin $s' = 8$ (0.7–0.8). Suppose the Q-values at that moment are:

State (belief bin)	Action 1	Action 2	Action 3	Action 4
1 = 0.0–0.1	1	8	7	8
2 = 0.1–0.2	2	5	25	6
3 = 0.2–0.3	5	5	8	8
4 = 0.3–0.4	3	6	5	1
5 = 0.4–0.5	4	4	6	8
6 = 0.5–0.6	8	2	4	5
7 = 0.6–0.7	5	2	1	3
8 = 0.7–0.8	1	1	5	9
9 = 0.8–0.9	8	4	2	6
10 = 0.9–1.0	3	1	2	5

Table 5.3: Example initial Q-values for fringe-restricted S6 (didactic excerpt).

Now suppose the executed tutoring step on S6 yields the hybrid reward computed earlier in Section 5.3.3, $R_t = 10.26$. (If a smaller didactic reward is preferred for hand calculation, the same algebra applies with any scalar r .) Using tabular Q-learning with learning rate $\alpha = 0.5$ and discount $\gamma = 0.95$ (the experimental discount in Table B.1),

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a')).$$

Substituting $Q(5, 4) = 8$, $r = 10.26$, and $\max_{a'} Q(8, a') = 9$:

$$\begin{aligned}
Q(5,4) &\leftarrow (1 - 0.5) 8 + 0.5(10.26 + 0.95 \cdot 9) \\
&= 4 + 0.5(10.26 + 8.55) \\
&= 4 + 0.5(18.81) \\
&= 13.405.
\end{aligned}$$

State	Action 1	Action 2	Action 3	Action 4
5 = 0.4–0.5	4	4	6	13.405

Table 5.4: Updated Q-values for state 5 after one training step.

This is exactly the training event stored by Algorithm 5.1 (line 11) and consumed by Algorithm 5.3 (lines 8–12): a transition (x_t, a_t, R_t, x_{t+1}) updates the action-value estimate for the chosen abstract action under the fringe constraint $s^* \in F_t = \{S6\}$. In the deployed system the same update is performed by gradient descent on the DQN, rather than by editing an explicit table entry; the table above is the readable one-step analogue of that training step.

5.5 Resource Management and Scaffolding

The resource manager bridges the decision layer and concrete instructional materials. Given a selected skill and difficulty level, it returns a question, answer, and hint triple from the dataset, skips items that have already been completed successfully, and tracks the number of attempts per item. Algorithm 5.4 executes one pass of this resource layer as called from Algorithm 5.1: it retrieves the instructional item, routes computation to the selected venue, renders the selected modality, delivers the step, observes the learner response, and updates item history.

Algorithm 5.4: Resource Manager Execution (Resource Layer)

Input: action a_t decoded as $(move_t, venue_t, modality_t)$; selected skill $s^* \in F_t$; item history H

Output: delivered tutoring step; observation o_t ; updated item history H'

```
1:  $(q, hint, solution) \leftarrow \text{RetrieveItem}(s^*, move_t, H)$ 
2:  $\text{RouteComputation}(venue_t)$  // local / edge / cloud
3:  $\text{RenderModality}(modality_t)$  // text / 2D / 3D
4: Deliver  $(q$  or hint or solution) to learner
5: observe learner response  $o_t$  and affect signals
6:  $H' \leftarrow \text{UpdateHistory}(H, q, o_t)$ 
7: return tutoring step,  $o_t, H'$ 
```

Equivalently, line 1 retrieves content for the decoded pedagogical move; lines 2–3 realize venue routing and render the modality; line 4 delivers the tutoring step; and lines 5–6 return the observation and updated history used by Algorithm 5.1’s belief update and replay-buffer storage.

Separately from this per-step execution loop, the branching logic on incorrect responses operates as follows. On a first incorrect attempt, the resource manager returns the stored hint associated with the current item and increments the attempt counter. On a second incorrect attempt, the resource manager presents a worked solution, marks the item as completed-with-assistance, and updates the belief value downward without re-queuing the item, preventing indefinite repetition. This three-attempt structure (unaided attempt $\rightarrow$ hint $\rightarrow$ solution) mirrors the scaffolding logic used in earlier KST-based ITS implementations and ensures that episodes terminate within a bounded step budget regardless of student performance.

A key function of the resource manager is to track completed items per skill, identified by the triple (skill identifier, difficulty level, question identifier), so that an item is not presented again after a correct response. This completion-tracking logic ensures that episodes make productive forward progress through the curriculum rather than cycling over already-mastered material.

Venue routing. The compute-venue component of the selected action determines where each computational task runs. The resource manager maintains a cost-lookup table of per-task latency, energy and offloadability profiles and dispatches tasks accordingly: avatar rendering is

sent to the local GPU for 3D-local actions and to the edge or cloud for offloaded actions; POMDP decision and Q-update logic always executes locally, since these operations require access to the local belief state and replay buffer and cannot be offloaded without introducing the round-trip latency they are designed to avoid; speech synthesis is routed to the cloud for high-quality voice and executed locally using a lightweight text-to-speech engine for latency-sensitive conditions. The routing logic is deterministic once the action is chosen, so no additional optimization or scheduling is needed at runtime.

Modality rendering. The interface modality component of the action triggers one of four rendering pipelines: full 3D avatar, 2D sprite avatar, text-plus-image or text-only. The validity check described in Section 5.3.3 is enforced at the resource manager level: if the selected modality violates the battery, thermal, memory, or ambient-noise constraint, the manager logs the violation (contributing to the modality-appropriateness metric) but executes the action as selected, allowing the RL agent to learn from the cost signal rather than silently overriding its choice.

With the domain, decision, training, and resource layers of the proposed framework fully specified, the hybrid POMDP–DQN architecture is complete as a methodological design: a KST-grounded curriculum, a factored belief-and-context state, a hybrid reward with adaptive α , a compact abstract action policy, and a resource manager that routes computation and renders the selected modality under the validity constraints defined above.

System instantiation of the running example. Figure 5.5 shows the ECA Tutor interface used to realize the three-layer architecture of Figure 5.1 for the Introductory Algebra domain of Section 5.2.2. The platform is a resource-constrained mobile client that keeps the POMDP–DQN decision loop on device while the resource manager (Algorithm 5.4) may place rendering or speech on local, edge, or cloud venues according to the selected action. In the screenshot, the learner is viewing a worked solution for an Introductory Algebra item retrieved by the resource manager. This full-screen solution view is one pedagogical-move presentation supported by Algorithm 5.4 (question, hint, or solution). After the learner submits a response, correctness and affect signals return through the resource layer, the belief and reward are updated as in Sections 5.3.2–5.3.3, and during offline training the transition is written to the replay buffer for the Q-learning update illustrated in Section 5.4.2. Thus, the screenshot is not an additional subsystem; it is the concrete interaction surface of the same control loop specified by Algorithm 5.1.

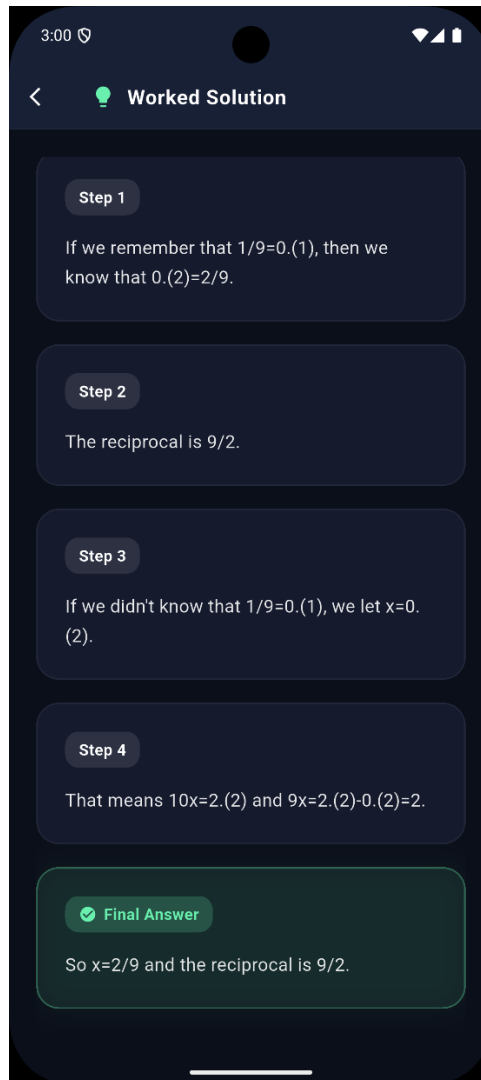


Figure 5.5: Worked-solution view in ECA Tutor for the Introductory Algebra running example, showing step-by-step solution content delivered by the resource manager (Algorithm 5.4).

CHAPTER 6

EXPERIMENTS AND ANALYSES

This chapter first describes the prototype implementation of the hybrid POMDP framework proposed in Chapter 5, then evaluates it against the five research questions posed in Chapter 4. Section 6.1 describes the implementation of the proposed architecture with the Dart/Flutter mobile application for an ECA tutor. Section 6.2 describes the simulation infrastructure built on the same ECA tutoring prototype runtime, ensuring that results reflect the behavior of the implemented application rather than a separate offline simulator. Section 6.3 presents the experimental design, factor grid, and ten pre-specified hypotheses. Section 6.4 introduces the statistical methodology used to evaluate those hypotheses. Section 6.5 reports results hypothesis by hypothesis with effect sizes and confidence intervals. Section 6.6 synthesizes key findings and discusses threats to validity.

6.1 Prototype Implementation

Dart/Flutter was selected as the implementation runtime because it is the language of the ECA Tutor prototype application developed for this research; implementing the controller in the same language and runtime as the application code eliminates the prototype-to-deployment gap and ensures that experimental results reflect real execution behavior. Flutter’s isolate-based concurrency model allows the DQN inference pass to run in a background thread without blocking the UI, and the Dart type system enforces the strict state-vector contracts required for reliable belief updates.

The hybrid POMDP framework described in Sections 5.1–5.5 is fully implemented as a working Android application, ECA Tutor, built in Dart and Flutter. The application runs on Android devices and was tested on a Pixel 9 Pro running in simulator mode. All thesis modules, which include the POMDP decision layer, DQN agent, KST graph, OCC affective computing

module, dynamic offloading engine, and task cost model are shared between the ECA Tutor interface and the batch experiment runner, ensuring that the experimental results reported in Chapter 6 reflect the behavior of the implemented prototype rather than an independent prototype.

While the POMDP decision layer mathematically selects from 4 abstract interface modalities (M) as defined in Section 5.4.1, the final visual output is dynamically managed by the Resource Layer. The Resource Layer takes the agent’s selected modality (M) and its chosen compute venue (V) to render one of 5 distinct visual presentation tiers on the user’s screen. For example, if the agent selects the rich 3D avatar modality, the Resource Layer will render the Full 3D tier if a high-quality compute venue such as full cloud offloading was chosen, but will automatically downgrade to the Compressed 3D tier if a constrained venue such as reduced local execution was selected.

The application implements the full interface modality spectrum defined in Section 5.3 as a live, per-step adaptive rendering pipeline. The richest tier, full 3D animated avatar, uses a cloud-hosted Three.js scene with morph-target facial animation and viseme-driven lip-synchronization through Amazon Polly neural text-to-speech.

As device or network constraints tighten, the system degrades progressively through compressed 3D (reduced level of detail, 30 FPS target), the 2D facial animation fallback, static image fallback (animation suspended to minimize rendering cost), and finally text-only mode, in which the avatar is suspended entirely and tutoring continues through text responses alone. Figure 6.1 shows the five interface modality tiers, and Figure 6.2 shows the ECA Tutor home screen in active tutoring mode.

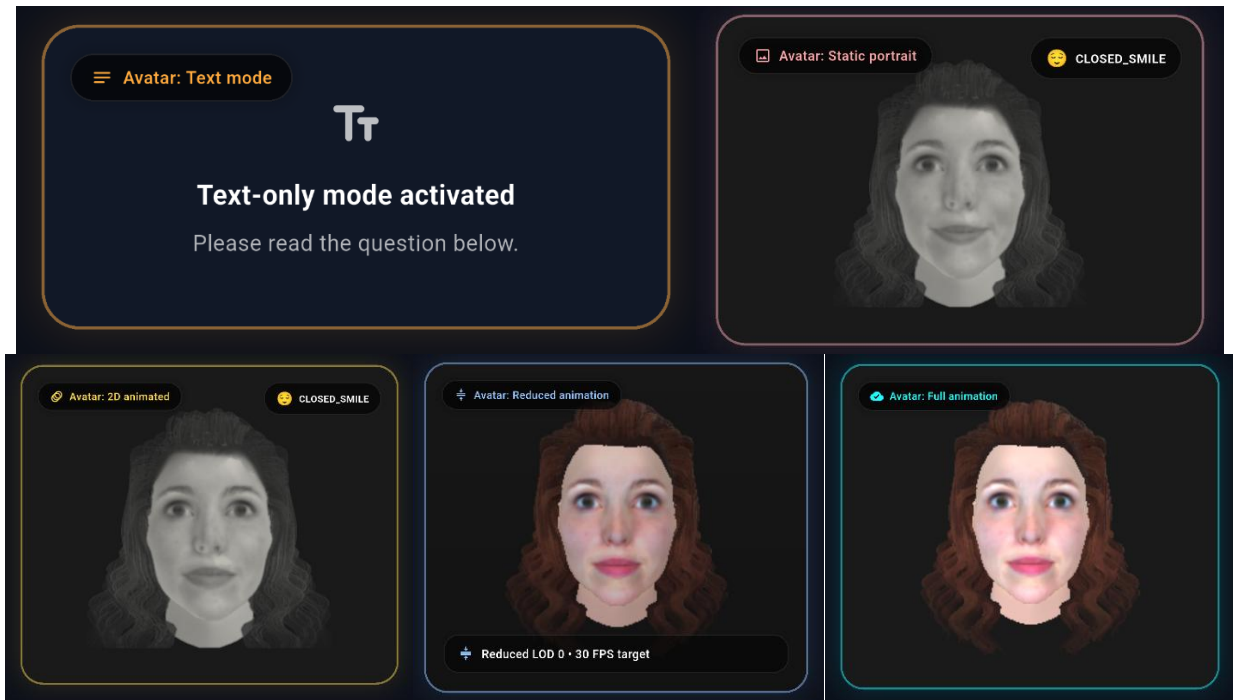


Figure 6.1: The five interface modality tiers of the ECA Tutor prototype, shown top left to bottom right from least to most resource-intensive: (a) text-only where the avatar is suspended, tutoring remains responsive under critical battery and network constraints; (b) static image fallback where the animation is suspended to minimize rendering cost; (c) 2D sprite - animated local fallback preserving expression cues; (d) compressed 3D with reduced level of detail at 30 FPS target; (e) full 3D - cloud-hosted avatar with full animation and viseme lip-sync restored

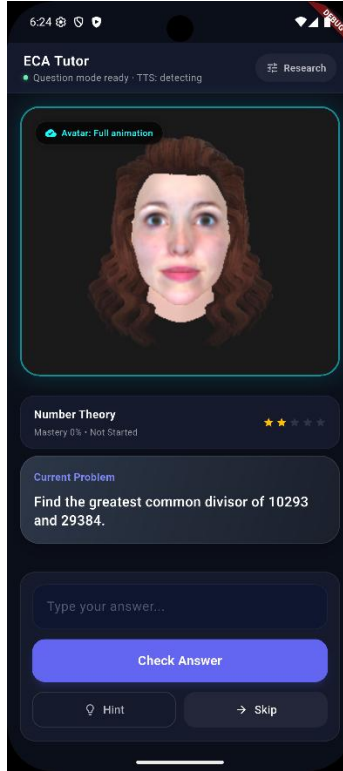


Figure 6.2: ECA Tutor home screen showing the full 3D avatar in active tutoring mode, presenting a Number Theory problem with mastery tracking and answer submission interface

One hardware limitation of the Pixel 9 Pro simulator environment is the absence of a Digital Signal Processor (DSP). Because the DSP is responsible for hardware-accelerated audio processing on physical Android devices, its absence in the simulator can produce intermittent static noise during avatar speech playback. This artifact is attributed to the absence of DSP-backed audio acceleration in the simulator and is expected not to occur on physical Pixel 9 Pro hardware; in any case, it does not affect the experimental metrics reported in this chapter, all of which are computed from simulated latency and energy cost models rather than from live audio playback. Full technical specifications of the application architecture, including the cloud avatar server, speech synthesis pipeline, and source file map, are provided in Appendix B.

With the domain, decision, training, resource, and implementation layers fully specified, the remainder of this chapter presents the experimental evaluation of the complete system including the simulation environment design, experimental factor grid, statistical methodology and results for each hypothesis.

6.2 Experimental Setup

6.2.1 Software and Implementation Platform

All experiments are conducted within the Dart/Flutter mobile runtime that powers the ECA Tutor avatar-based tutoring prototype. The experimental framework, including the POMDP decision layer, DQN agent, student simulator, reward computation, and task cost model, uses the same code modules as the ECA Tutor prototype. This ensures that results reflect the behavior of the implemented prototype application, rather than that of a separate prototype. A simulated learner model (described in Section 6.2.2) provides the interaction data needed for reinforcement learning training and statistical evaluation.

Because a central claim of this thesis is that pedagogical effectiveness is preserved rather than sacrificed for latency and energy gains, it is useful to visualize the mechanism through which the proposed system sequences instruction. Curriculum sequencing in the hybrid POMDP is governed by the Knowledge Space Theory (KST) formalism introduced in Sections 2.3.2 and 5.2.2: each skill is a node in a prerequisite directed acyclic graph, and a skill only enters the outer fringe, the set of skills eligible for instruction, once all of its prerequisites have reached the mastery threshold. This mechanism is implemented once in the curriculum module and is shared unchanged between the live ECA Tutor application, the batch experiment engine used for the hypothesis tests in Section 6.5, and the illustrative simulation described below.

To make this behavior concrete, a single-student illustrative simulation was run directly against the production KST curriculum module, restricted to its seven-skill base curriculum (Number Theory, Prealgebra, Geometry, Algebra, Intermediate Algebra, Counting & Probability, and Precalculus) so that the result is directly comparable to the equivalent seven-skill diagram reported for the ITS-KST system in Duong [53]. At each of 50 simulated tutoring steps, matching the per-episode step budget defined in Section 6.2.2, the macro-adaptive, fringe-restricted selection rule chooses the next skill from the current outer fringe; a synthetic response is then drawn with a correctness probability that increases with prior practice on that specific skill, and the same Bayesian-style mastery update used by the live tutor is applied to the selected skill's mastery likelihood. The procedure was repeated for three independently seeded simulations.

Figure 6.3 plots the resulting mastery-likelihood trajectory for all seven skills across the three simulations. In every simulation, a skill's likelihood of mastery remains at zero until its prerequisites are satisfied, at which point it becomes eligible for instruction and its likelihood rises monotonically as the student practices it, producing the staircase pattern visible from Number Theory through Precalculus. Skills later in the prerequisite chain, such as Counting & Probability and Precalculus, do not begin to rise until every earlier skill has crossed the mastery threshold, confirming that the outer-fringe constraint is respected throughout the interaction. This is the same prerequisite-respecting sequencing behavior reported for the ITS-KST system in Duong [53]; the diagram demonstrates that although the hybrid POMDP-DQN framework re-optimizes compute-venue and avatar-modality decisions for mobile latency and energy, it does so without altering the KST-based pedagogical sequencing logic itself, directly supporting the learning-preservation and curriculum-adherence results reported for H1a and H1b in Section 6.5.1.

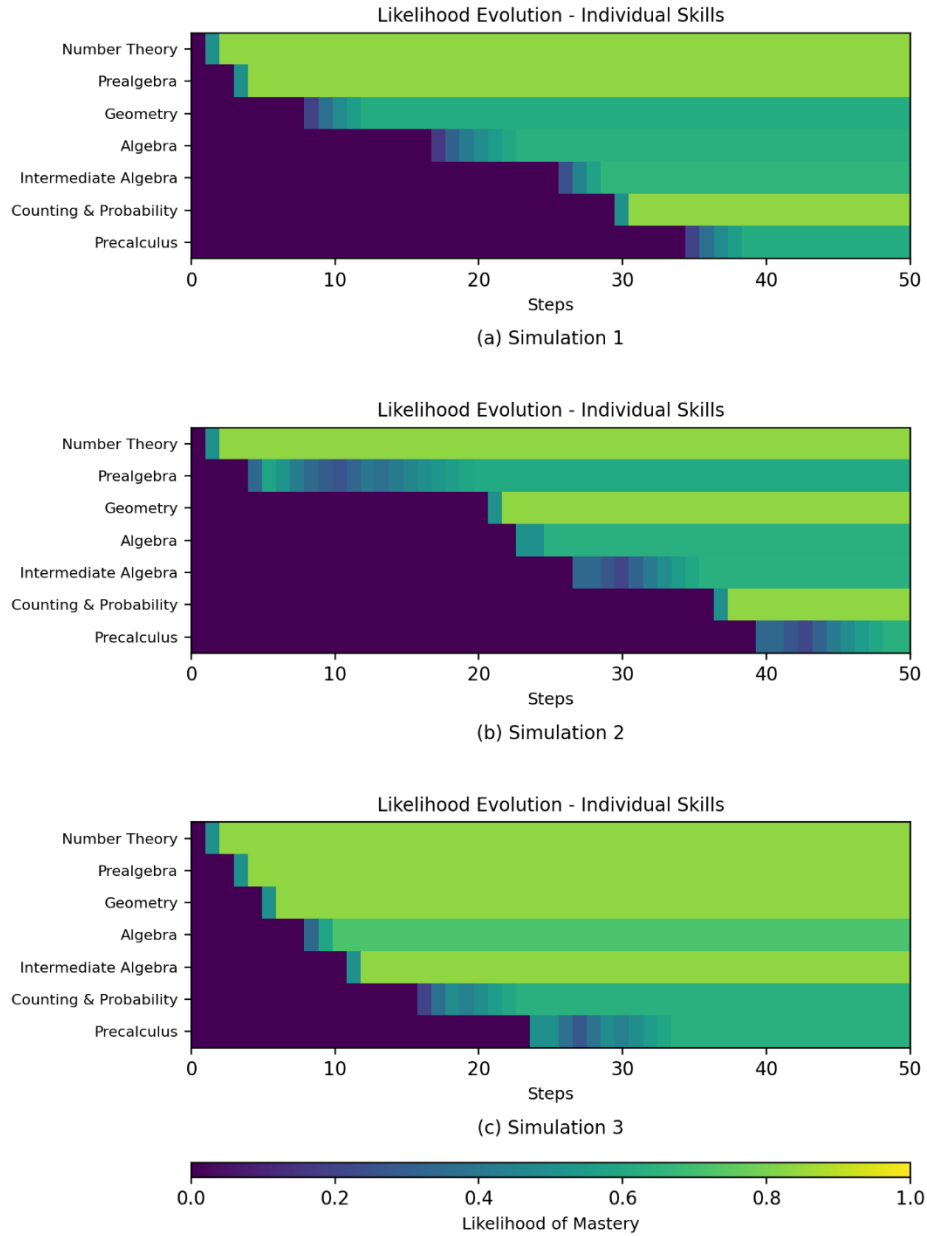


Figure 6.3: Mastery likelihood evolution over 50 tutoring steps for the seven-skill KST base curriculum, shown for three independently seeded single-student simulations: (a) Simulation 1, (b) Simulation 2, (c) Simulation 3. Likelihood of mastery (color) is governed entirely by the outer-fringe prerequisite constraint from Section 5.2.2: each skill remains unreachable (likelihood fixed at 0) until its prerequisites are satisfied, after which its likelihood rises as the student practices it.

6.2.2 Student Simulation Model

The simulated learner maintains a 40-skill Bayesian belief vector, updated each step by a learning rate comprising a base rate of 0.7, plus an engagement contribution of 0.2 times the current engagement level, plus a motivation contribution of 0.1 times the current motivation level. The eight-skill Introductory Algebra DAG in Section 5.2.2 is an illustrative running example used to explain the proposed method. The experimental evaluation uses a larger 40-skill mathematics curriculum (the prerequisite graph for this curriculum is provided in Appendix Figure B.1) that applies the same fringe-constrained selection principle; Figure 6.2 shows a Number Theory item from that evaluated curriculum. Answer correctness at each step is drawn from a Bernoulli distribution shaped by the student’s current mastery level and cognitive load. Three scalar affect variables, engagement, frustration, and motivation are updated after each interaction based on the correctness of the response, the selected modality, and the pedagogical action taken. These affect variables modulate learning rate, early termination probability, and the pedagogical reward signal, consistent with appraisal-based models of student affect in which emotions such as frustration and engagement are triggered by outcomes relative to the learner’s goals [44].

An episode terminates when one of four conditions is met: all skills are mastered, the battery drops below 5%, the 50-step budget is exhausted, or the student disengages (motivation below 0.1 and frustration above 0.8). No real student data are used; learning gain, latency, and energy are computed from synthetic mastery and cost models, consistent with standard practice in RL-based ITS and mobile edge computing research.

6.2.3 Per-Task Cost Model

Latency and energy for each tutoring step are aggregated across eight typed task categories: avatar rendering (3D and 2D), text-to-speech synthesis, pedagogical logic (POMDP decision and Q-update), knowledge tracing (Bayesian belief update), dialogue management, affective computing, and modality control. Each task carries a cost profile defining CPU cycles, memory footprint, power draw, local latency, and cloud latency. These profiles are scaled per step by device-specific factors such as CPU speed, GPU speed, and power draw, and by network-specific

factors for cloud and edge routes. Total per-step latency and energy flow directly into the evaluation metrics.

6.2.4 Evaluation Metrics

Each episode contributes one record to the experimental log. The following metrics are used for hypothesis testing, aggregated to seed–network–device units to prevent pseudoreplication:

- **Learning gain** - normalized improvement from pre- to post-episode knowledge test, clamped to $[0, 1]$.
- **Average latency (ms)** - mean per-step latency computed from the task cost model, scaled by device and network parameters.
- **Total energy (mWh)** - sum of per-step energy consumption over the episode, including compute and network transmission costs.
- **Failure rate** - fraction of steps with failures, timeouts, or invalid responses.
- **Curriculum adherence** - fraction of steps in which the selected skill falls within the valid index bounds of the 40-skill graph.
- **Modality appropriateness** - fraction of steps in which the chosen avatar modality satisfies device and environment validity constraints (battery, thermal state, memory capacity, and ambient noise level).
- **Latency SLA satisfaction** - proportion of individual steps with latency below the 500 ms real-time perception threshold.

6.3 Experimental Design

6.3.1 Factor Grid

Table 6.1 lists the factors swept by the evaluation:

Factor	Levels	Count
Random seeds	1–50	50
Network scenarios	Fast Wi-Fi, Slow 4G, Dynamic 5G	3
Device classes	Low end (2 GB), Mid range (4 GB), Flagship (8 GB)	3
Fixed α values	0.0, 0.1, ..., 1.0	11

Adaptive α	State-driven rule in Section 5.3.3	1
Baselines	Pedagogy only, compute only, static weighted, heuristic	4
Transfer pairs	Directed source $\rightarrow$ target, no self-pairs	72

Table 6.1: Experimental factor grid

6.3.2 Network and Device Parameters

Tables 6.2 and 6.3 give the network-scenario and device-class parameters, respectively.

Scenario	Bandwidth mean	Latency base	Packet loss
Fast Wi-Fi	50 Mbps	5 ms	0.1%
Slow 4G	10 Mbps	30 ms	1.0%
Dynamic 5G	60 Mbps	8 ms	0.1%

Table 6.2: Network scenario parameters simulate a range of mobile connectivity conditions

Device class	CPU power draw	GPU power draw	Memory
Low end	150 mW	50 mW	2 GB
Mid range	200 mW	100 mW	4 GB
Flagship	250 mW	150 mW	8 GB

Table 6.3: Device class parameters representing the heterogeneity of mobile hardware tested in the evaluation

6.3.3 Baseline Policies

Four baselines provide comparison points spanning the learning–efficiency trade-off:

- **Pedagogy-only:** Always selects full local computation with a rich 3D avatar and the reinforcement pedagogical action. This establishes the upper bound on learning richness and the ceiling for computational cost.
- **Compute-only:** Always selects cloud-reduced computation with text-only presentation. This establishes the lower bound on computational cost and the floor for learning quality.
- **Static-weighted:** Defaults to full local computation; drops to 2D avatar when battery falls below 20%. This is a simple reactive rule equivalent to conventional mobile ITS heuristics.

- **Heuristic:** Applies if-then rules based on battery level and network quality to choose compute venue and avatar modality. This represents practical expert-designed rule-based adaptation.

Baselines do not train; they follow fixed decision rules and are evaluated for 50 episodes per seed–network–device combination (22,500 episodes each). The hybrid DQN trains for 100 episodes per configuration before 50 evaluation episodes.

Fixed- α hybrid conditions train and evaluate a Deep Q-Network with a constant α in the hybrid reward. The adaptive- α condition instead evaluates the adaptive preference controller in the POMDP decision engine: at each step the engine updates the active policy family from device and learner state, and the adaptive- α update applies the state-driven α rule defined in Section 5.3.3. Adaptive- α results should therefore be interpreted as the performance of this adaptive controller configuration, not as a second DQN trained under a different fixed α .

6.3.4 Episode Budget and Scale

The full evaluation comprises 360,000 non-transfer episodes: 90,000 baseline evaluation episodes (4 baselines $\times$ 22,500) and 270,000 hybrid DQN episodes (11 fixed α values plus 1 adaptive- α setting, each run over 50 seeds $\times$ 9 contexts $\times$ 50 evaluation episodes). An additional 10,958,835 individual hybrid steps are evaluated for the step-level latency SLA (H4a).

6.3.5 Pre-Specified Hypotheses

Table 6.4 lists the ten pre-specified hypotheses and their acceptance criteria:

ID	Research question	Pre-specified criterion
H1a	Does the hybrid preserve or improve learning?	Hybrid LG within 5% of pedagogy-only (or beats it)
H1b	Does the hybrid respect curriculum constraints?	Mean unit curriculum adherence $> 95\%$
H2a	Does the hybrid reduce latency significantly?	Mean latency reduction > 100 ms
H2b	Does the hybrid produce a large energy reduction?	Cohen's $d > 0.7$ (pedagogy–hybrid)

H3a	Is learning stable across device classes?	Max pairwise Cohen’s d across device pairs < 0.5
H3b	Is the policy robust under network degradation?	$\geq 80\%$ LG retained on slow 4G AND $\geq 85\%$ modality-appropriate steps
H4a	Do the majority of individual steps satisfy the latency SLA?	$\geq 90\%$ of hybrid steps with latency < 500 ms
H4b	Is learning preserved across the full α sweep?	Every α (and adaptive α) within 5% of pedagogy-only LG
H5a	Does transfer learning accelerate convergence?	Mean episodes saved ≥ 5 ; one-sided $p < 0.05$
H5b	Does zero-shot transfer achieve acceptable performance?	Mean zero-shot ratio $\geq 80\%$ of fine-tuned; one-sample $p < 0.05$

Table 6.4: Pre-specified hypotheses and their quantitative acceptance criteria.

6.4 Statistical Methodology

This section presents the statistical methods used to evaluate the experimental results, provides formal definitions of each test and measure, and justifies their selection for the evaluation design described in Sections 6.2–6.3.

6.4.1 Aggregation and Pseudoreplication Prevention

Pseudoreplication occurs when non-independent observations are treated as independent samples, inflating statistical significance [79].

Because multiple episodes are generated per seed–network–device combination, using raw episode counts as the sample size would overstate the degrees of freedom. To prevent this, episodes are first averaged to seed–network–device units before hypothesis testing, yielding $n = 450$ units (50 seeds $\times 9$ contexts) for primary comparisons, and $n = 150$ paired units for network-stress comparisons.

6.4.2 Hypothesis Tests

A t-test is a statistical test that evaluates whether the means of one or two groups differ by more than would be expected by chance. Three classes of t-test are applied here, each matched to the comparison structure of the corresponding hypothesis:

Welch two-sample t-test. This test compares the means of two independent groups without assuming equal variances [80]. It is used for hybrid-versus-baseline comparisons on episode-level distributions, where the hybrid and baseline conditions are trained independently and may exhibit different variance structures.

Paired t-test on unit means. When two conditions share the same seed $\times$ network $\times$ device unit, a paired t-test exploits the within-unit correlation to increase statistical power [81]. This test is applied for H1a (learning preservation), H2a (latency reduction), and H4b (α -sweep stability), where each hybrid unit is paired with its corresponding baseline or alternative- α unit.

One-sided one-sample t-test. For H5a (episodes saved > 0) and H5b (zero-shot ratio $> 80\%$), the null hypothesis specifies a fixed threshold rather than a comparison group. A one-sided test is appropriate because the research questions are directional: the hypotheses predict that the effect exceeds a specific value, not merely that it differs from zero [82].

6.4.3 Effect Size: Cohen's d

Cohen's d quantifies the standardized difference between two group means as $d = \frac{(M_1 - M_2)}{SD_{pooled}}$,

where SD_{pooled} is the pooled standard deviation [83]. Following Cohen's conventional benchmarks, effect sizes are interpreted as negligible ($|d| < 0.2$), small ($0.2 \leq |d| < 0.5$), medium ($0.5 \leq |d| < 0.8$), or large ($|d| \geq 0.8$). Cohen's d is reported for all headline comparisons to provide a scale-independent measure of practical significance that complements the p-value.

6.4.4 Confidence Intervals

All reported aggregate means are accompanied by 95% confidence intervals computed as $mean \pm 1.96 \cdot \frac{SD}{\sqrt{n}}$, where SD is the standard deviation of the unit means and n is the number of units. Confidence intervals convey the precision of each estimate and allow readers to assess whether observed differences are practically meaningful, not merely statistically significant [84].

6.4.5 Justification for the Selected Methods

The combination of pseudoreplication prevention, paired and unpaired t-tests, Cohen's d, and confidence intervals was selected because it provides a complete inferential toolkit for the factorial experimental design: (a) unit-level aggregation ensures valid degrees of freedom, (b) paired tests maximize power for within-design comparisons, (c) effect sizes guard against over-reliance on p-values in a large-sample setting where even trivial differences can reach statistical significance, and (d) confidence intervals communicate estimation uncertainty. This approach follows recommended reporting practices in experimental computer science [85] and educational technology research [84].

6.5 Experimental Results

6.5.1 Learning Preservation and Curriculum Adherence (H1a, H1b)

H1a - CONFIRMED. The hybrid POMDP policy substantially exceeds the learning preservation criterion, achieving not merely preservation but a 44% improvement. Unit-mean learning gain is 0.1601 (95% CI [0.1592, 0.1611]) for the hybrid system, compared with 0.1111 (95% CI [0.1085, 0.1137]) for the pedagogy-only baseline, for a paired difference of +0.0489, or 44% above baseline and well above the 5% preservation criterion. The Welch t-statistic on episode-level distributions is 34.56 with $p \approx 1.1 \times 10^{-261}$; Cohen's $d = 0.207$ at episode level (small, as expected given within-unit variability), with the paired unit-level test providing the definitive comparison.

The counterintuitive direction of this result, where the hybrid outperforms the always-rich pedagogy-only strategy on learning, is explained by the engagement cascade. The pedagogy-only baseline commits to the richest avatar and highest-quality local computation at every step, regardless of thermal state, memory pressure, or network conditions. Under stressed conditions, this commitment produces high latency and increased failure rates, which depress the simulated student's engagement and motivation, in turn reducing the per-step learning rate. The hybrid, by dynamically downshifting to cheaper modalities when context demands it, maintains motivational state and accumulates higher pedagogical reward over the episode. Efficiency and learning are not antagonistic when the cost of over-richness is modeled explicitly.

The adaptive α configuration achieves the highest learning gain of any condition: $LG = 0.1688$, which is 4.7% above the best fixed α setting ($\alpha = 0.6$, $LG = 0.1613$) and 51.9% above the pedagogy-only baseline. Figure 6.4 shows the episode-level learning-gain distributions by policy condition.

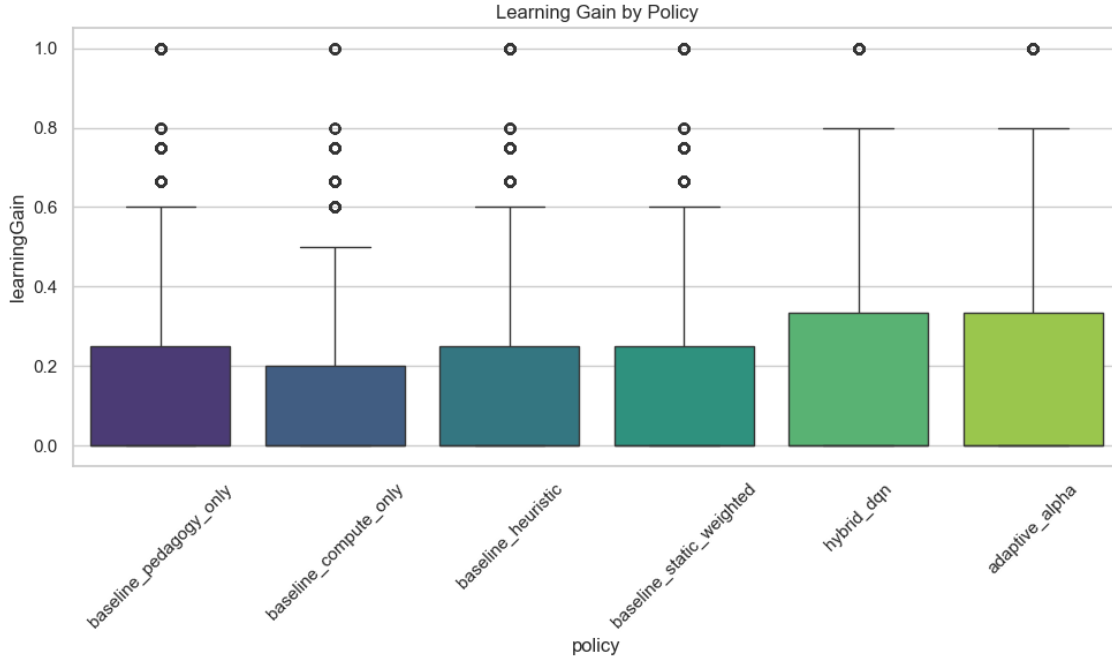


Figure 6.4: Episode-level learning gain distributions by policy condition across 50 seeds and 9 device–network contexts. The hybrid DQN achieves the highest median and interquartile range; the compute-only baseline achieves the lowest

H1b - CONFIRMED. Mean unit-level curriculum adherence is 100.0%, with a 95% CI of [100.0%, 100.0%], and all 450 seed, network, and device units are at ceiling. This ceiling result is explained by the resource manager’s completion-tracking logic in Section 5.5, which prevents re-presentation of already-completed items and, together with the bounds check on skill indices, keeps every selected skill inside the valid curriculum region.

No episode across any seed, network scenario, device class, or α condition selected an out-of-range skill index. This result reflects the bounds-checking constraint (parsed skill index < no. of skills = 40) applied inside the simulated environment. This metric alone does not verify full KST DAG prerequisite compliance, and full fringe-based validation in

the high-throughput simulator is left for future work. Tutoring sessions executed in the ECA Tutor Dart/Flutter prototype enforce the outer-fringe constraint described in Section 5.2.2.

6.5.2 Latency and Energy Reduction (H2a, H2b)

H2a - CONFIRMED. The hybrid achieves large, statistically significant latency reductions against all computationally expensive baselines. Unit-mean latency for the hybrid is 96.2 ms (95% CI [95.6, 96.7]) versus 594.6 ms for pedagogy-only, a paired reduction of 498.5 ms with Cohen’s $d = -3.107$, well above the large-effect threshold. Effect sizes against the other heavy baselines are similarly decisive ($d = -3.081$ versus static-weighted; $d = -0.966$ versus heuristic). The only baseline with lower latency is compute-only at 63.7 ms, which achieves this by using text-only cloud routing exclusively and produces the worst learning gain in the experiment (LG = 0.0934). The hybrid is therefore Pareto-dominant: no alternative achieves both higher learning and lower latency simultaneously. Figure 6.5 plots mean per-step latency by policy condition, and Figure 6.6 shows the corresponding total-energy distributions.

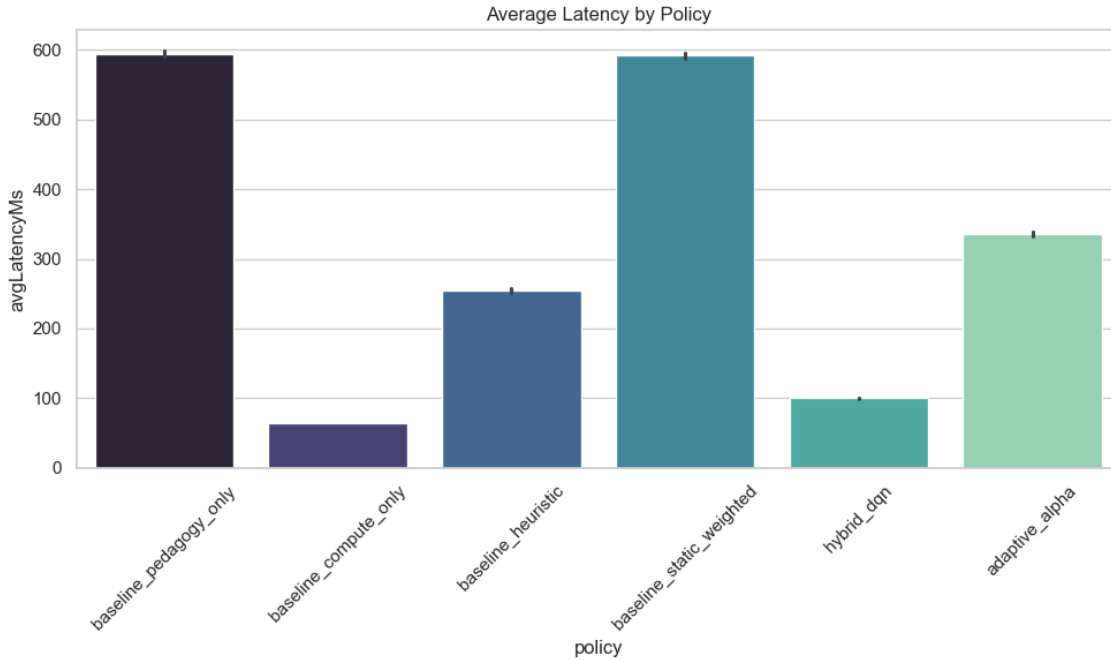


Figure 6.5: Mean per-step latency by policy condition.

H2b - PARTIAL. The energy hypothesis is the most important negative result and warrants explicit analysis. The hybrid consumes more energy than pedagogy-only: episode-level means

are 9.024 mWh for the hybrid versus 7.435 mWh for pedagogy-only. Cohen’s d is -0.617 (medium effect, adverse direction; $p \approx 0$), failing both the sign and magnitude of the pre-specified criterion ($d > +0.7$ in the pedagogy-minus-hybrid direction).

The root cause is a reward-shaping imbalance, not cloud routing in isolation. The compute-only baseline also routes every step to the cloud yet has the lowest energy consumption of any condition (Figure 6.6), because it always pairs cloud routing with the cheapest text-only modality. The hybrid DQN, by contrast, frequently pairs cloud routing with the richest 3D avatar when network quality is high, because high-quality cloud inference combined with rich rendering maximizes pedagogical reward through better engagement and correctness outcomes; this combination incurs both rendering and network-transmission energy costs that local, low-fidelity execution avoids. The DQN behaves rationally given the reward function: the pedagogical term numerically dominates the technical term at the configured α values, and the agent trades energy for learning as a consequence. The suggested correction is to increase the cloud transmission energy penalty in the technical reward term or introduce a hard battery-level constraint that forces local fallback, and this is a priority direction for future work. This diagnosis was possible because the evaluation ran to completion; the partial result reflects a reward-weighting decision, not a shortage of research time or an implementation failure.

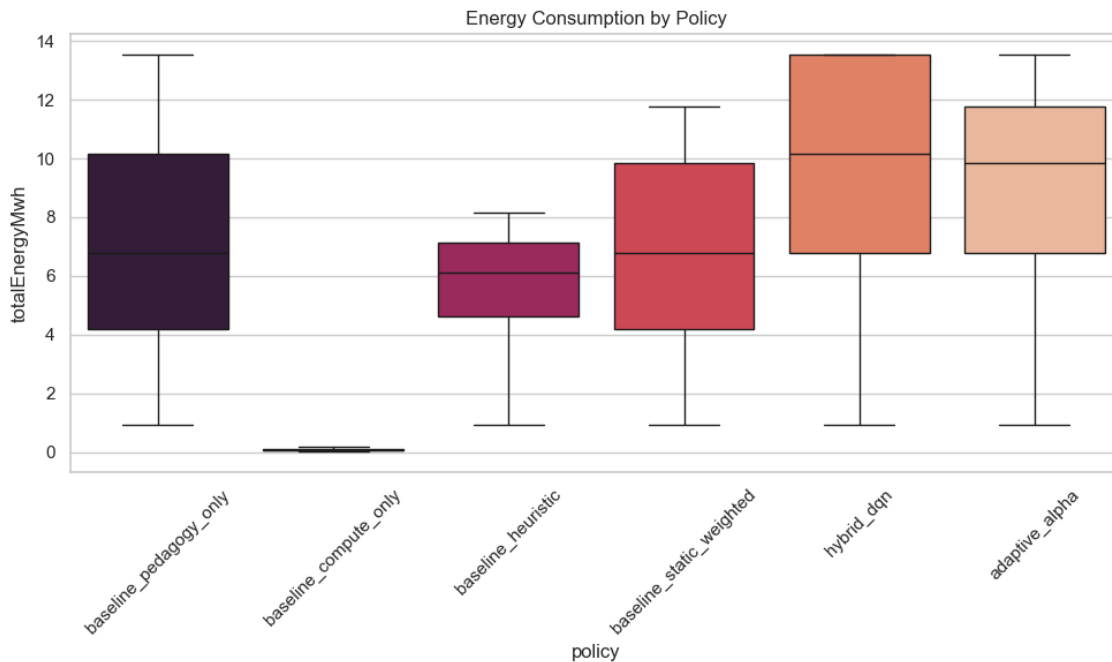


Figure 6.6: Box plots of total energy consumption per policy.

6.5.3 Robustness Across Device and Network Conditions (H3a, H3b)

H3a - CONFIRMED. Learning gain is effectively invariant across device classes. The maximum pairwise Cohen’s d across the three device profiles, low-end 2 GB, mid-range 4 GB, and flagship 8 GB, is 0.164, which is well within the negligible threshold of <0.5 . The mechanism is that the DQN learns device-specific action preferences during training: on the low-end device, the modality validity constraint blocks the richest 3D avatar entirely, causing the agent to default to cheaper modalities that maintain equivalent pedagogical rhythm. Device heterogeneity, rather than degrading performance, is naturally absorbed by the policy’s contextual adaptation without any cross-device training.

H3b - PARTIAL. This hypothesis has two components; both must pass. Learning retention under network degradation is fully confirmed: the hybrid achieves 100.21% of fast-WiFi learning gain under slow 4G conditions (paired $t = 0.44$, $p = 0.656$, $d = 0.051$, $n = 150$ pairs) with no detectable learning loss under stress. The hybrid agent compensates for high-latency networks by shifting computation to local venues, preserving pedagogical rhythm at the cost of lower avatar fidelity.

However, modality appropriateness under stressed conditions is 63.4%, falling below the 85% gate. Under slow 4G and dynamic 5G conditions, 36.6% of steps select a modality that the validity rules flag as context-inappropriate, typically the full 3D avatar despite battery, thermal, or memory constraints. As with H2b, this reflects a reward-shaping imbalance: when the pedagogical reward achievable through the rich avatar is sufficiently large, the interface-appropriateness term (fixed weight 0.2) is outweighed. This is a reward calibration failure, not an architectural failure. Learning quality is fully preserved; the agent is achieving its primary objective while insufficiently penalizing context-inappropriate presentations. As with H2b, this diagnosis was completed rather than left unresolved for lack of time; the shortfall is a reward-weighting decision, not an implementation failure.

6.5.4 User Experience Consistency (H4a, H4b)

H4a - CONFIRMED. Of the 10,958,835 hybrid steps evaluated individually, 10,814,253, or 98.68%, fall below the 500 ms real-time perception threshold, exceeding the $\geq 90\%$ gate by 8.68 percentage points. This step-level test is methodologically stronger than a mean-latency

comparison, because a policy with a low mean latency could still fail the SLA if its distribution has a heavy tail above 500 ms. The 98.68% result confirms that the latency advantage is not merely an average-case property but holds across almost the entire interaction distribution. At 50 steps per episode and 96.2 ms per step, expected total session latency is about 4.8 seconds, which is comparable to a natural conversational exchange [86].

H4b - CONFIRMED. Unit-mean learning gain varies only within [0.1591, 0.1613] across all 11 fixed α settings, a spread of 0.0022, so it is effectively flat. This α -insensitivity is a robustness property with direct deployment value: a practitioner who misconfigures the trade-off parameter will not observe degraded learning. The DQN, trained for 100 episodes per configuration, internalizes the pedagogy–efficiency balance in its learned weights, making the explicit α parameter largely redundant for fixed- α settings. The adaptive condition breaks this ceiling: LG = 0.1688, which is 4.7% above the best fixed setting. This gain comes with a clear latency cost: adaptive- α averages 336.0 ms per step (Table A.3), about 3.5 $\times$ the fixed- α hybrid mean of 96.2 ms, and rises to 589.2 ms under slow 4G. The increase is consistent with the adaptive controller raising pedagogical priority and selecting richer cloud-backed execution under favorable conditions. Adaptive- α is therefore best treated as the learning-maximizing configuration when some latency budget remains, whereas a fixed mid-range α (for example, $\alpha = 0.6$) remains preferable when strict real-time responsiveness is the primary constraint. Figure 6.7 plots learning gain, latency, and energy as functions of fixed α .

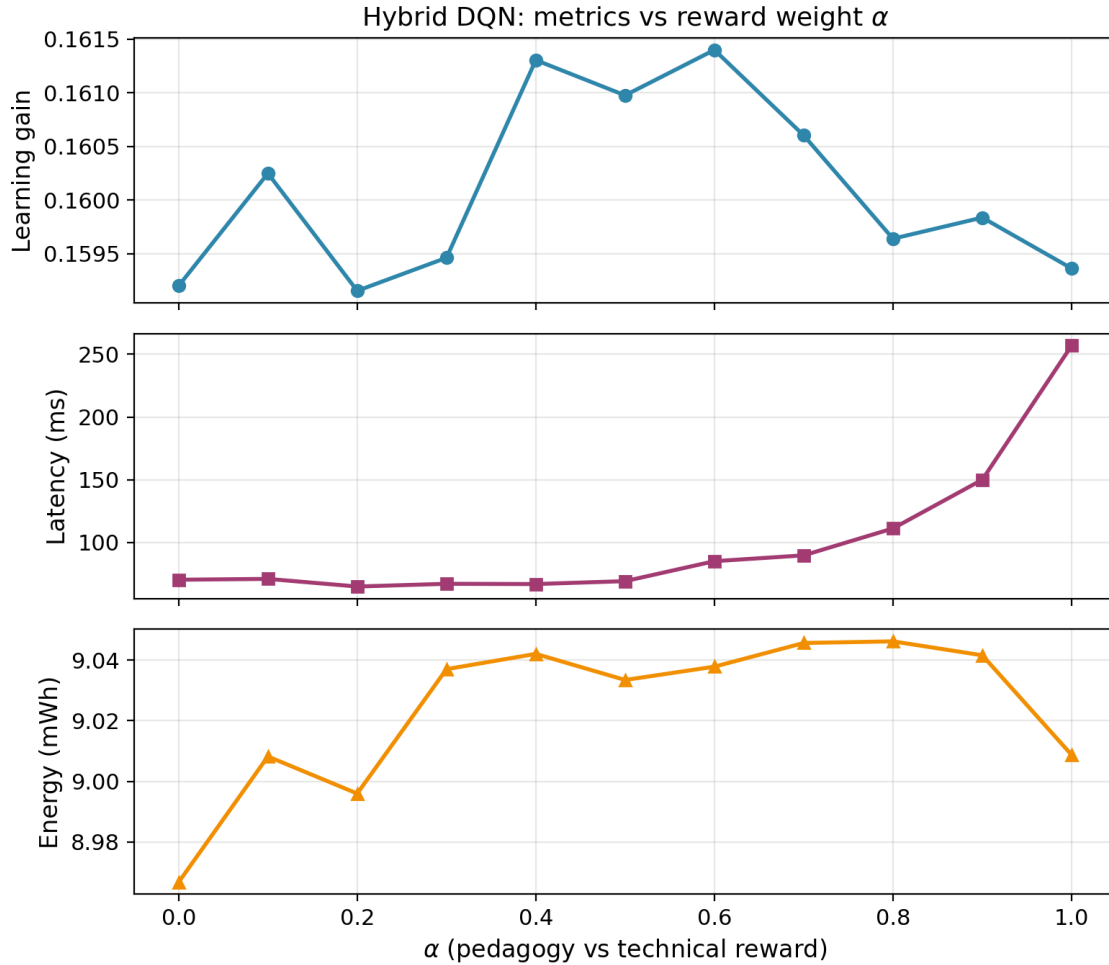


Figure 6.7: Learning gain, latency, and energy as functions of fixed α (three panels).

6.5.5 Transfer Learning (H5a, H5b)

H5a - CONFIRMED. Across 72 directed source–target context pairs, initializing the DQN from a previously trained policy saves a mean of 19.43 episodes to convergence (95% CI [12.41, 26.45]; one-sided $p \approx 0$ against null of zero savings). Fifty of 72 pairs show positive speedup. The 22 pairs with negative speedup are concentrated in high-bandwidth-to-low-bandwidth transitions, where the transferred policy must un-learn rich-avatar preferences before adapting to the lower-resource context. Importantly, this training interference does not affect evaluation performance, as H5b confirms.

The top convergence speedups are observed for pairs with overlapping device class (e.g., fast-WiFi mid-range to slow-4G mid-range: +89.0% speedup) and for pairs with similar network

latency profiles. The worst cases, pairs crossing dissimilar bandwidth tiers, show negative speedup but still achieve zero-shot ratios above the 80% gate.

H5b - CONFIRMED. Mean zero-shot transfer performance across all 72 directed pairs is 100.5% of fine-tuned performance (95% CI [98.0%, 103.1%]; one-sample $p \approx 0$ against null of 80%). All 72 pairs exceed the 80% gate; the lowest individual ratio is 84.8% and the highest is 133.9%. The 95% CI entirely encompasses and exceeds 100%, indicating that transferring without fine-tuning is not merely acceptable but is on average as effective as fine-tuning in the target context. Cases where zero-shot exceeds fine-tuned performance are consistent with the fine-tuned policy over-specializing to target context idiosyncrasies, while the broader transferred policy generalizes better to edge cases within that context.

6.5.6 Summary Scorecard

Table 6.5 summarizes the verdict for each hypothesis.

Hypothesis	Verdict	Key measured value
H1a - Learning preservation	CONFIRMED	+44.0% above pedagogy-only; $p \approx 0$
H1b - Curriculum adherence	CONFIRMED	100.0% CI [100.0, 100.0]
H2a - Latency reduction	CONFIRMED	-498.5 ms; Cohen's $d = -3.107$
H2b - Energy reduction	PARTIAL	$d = -0.617$ (adverse direction)
H3a - Device robustness	CONFIRMED	Max pairwise $d = 0.164$
H3b - Network robustness	PARTIAL	LG retained 100.2%; modality at 63.4% vs 85% target
H4a - Step-level latency SLA	CONFIRMED	98.68% of 10,958,835 steps under 500 ms
H4b - α -sweep stability	CONFIRMED	Range = 0.0022; adaptive LG = 0.1688 (best overall)
H5a - Transfer convergence	CONFIRMED	19.43 episodes saved; CI excludes 0

H5b - Zero-shot quality	CONFIRMED	100.5% of fine-tuned; 72/72 pairs $\geq 80\%$
-------------------------	------------------	---

Table 6.5: Hypothesis testing summary with verdicts and key measured values

6.6 Analysis and Discussion

6.6.1 Key Findings

Finding 1 - The hybrid policy is Pareto-dominant on learning and latency. No tested policy achieves both higher learning gain and lower latency than the hybrid simultaneously. The compute-only baseline achieves lower latency (63.7 ms) but at the cost of the worst learning outcome (LG = 0.0934). All other baselines achieve lower learning at higher latency. The hybrid occupies the upper-left region of the learning–latency Pareto front, as shown in Figure 6.8.

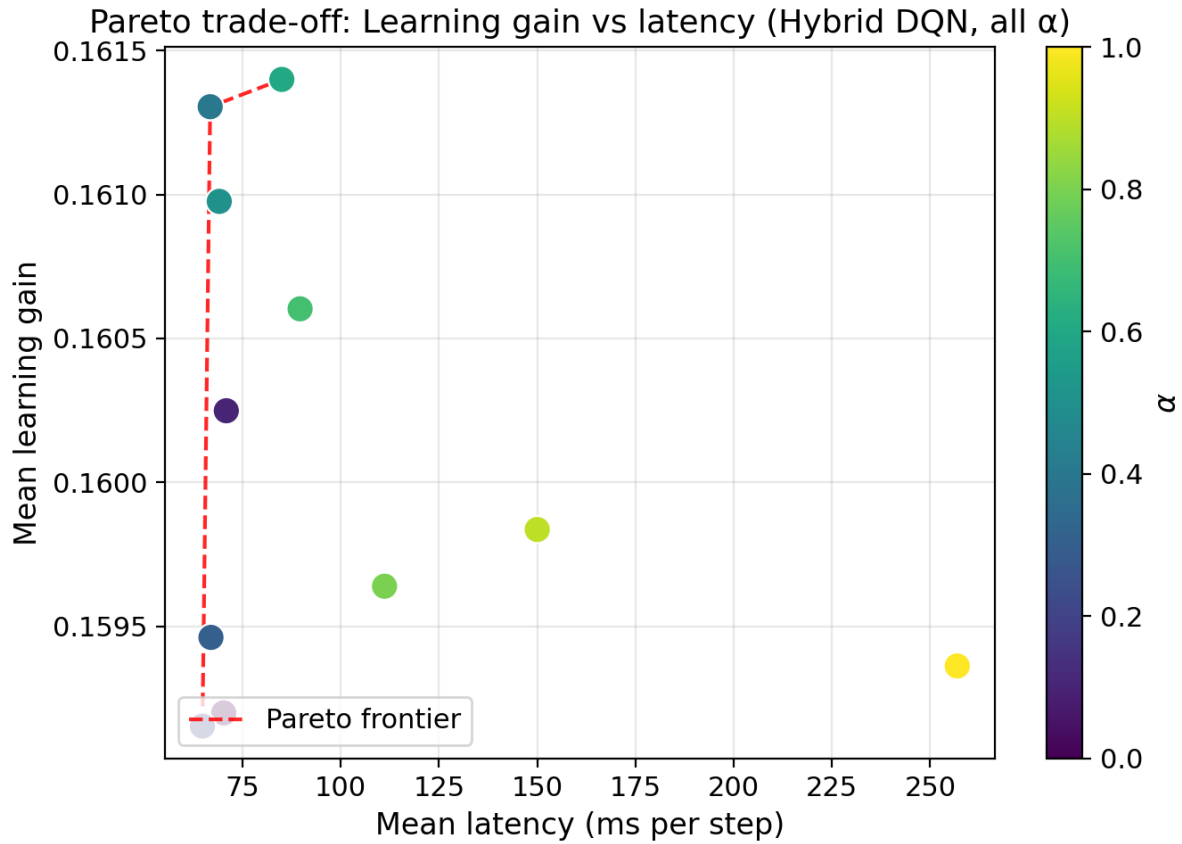


Figure 6.8: Pareto trade-off: learning gain vs latency. The hybrid DQN occupies the upper-left (high learning, low latency) region

Finding 2 - Efficiency and learning are not antagonistic. The hybrid policy ends 0.0489 units of normalized learning gain ahead of the always-rich pedagogy-only baseline (a 44% relative gain on the $[0, 1]$ LG scale; $p \approx 1.1 \times 10^{-261}$) not because the agent is more pedagogically sophisticated, but because over-richness is harmful at the margin. Always committing to the richest avatar and computation, regardless of context, introduces latency, failures, and cognitive burden that can suppress student engagement and, in turn, learning rate. The hybrid demonstrates that a system optimized for mobile quality-of-experience indirectly improves learning by maintaining motivational state. This engagement cascade, where excessive computational cost degrades engagement which degrades learning rate, is the mechanism underlying the counterintuitive result that a policy optimizing both pedagogy and QoE outperforms a policy optimizing pedagogy alone.

Finding 3 - The α trade-off parameter is robust but bounded. The near-flat learning gain curve across the full α sweep (spread = 0.0022) shows that the fixed- α DQN has internalized the pedagogy–efficiency balance in its learned weights, making the explicit α parameter largely redundant for fixed configurations. The adaptive- α condition, which derives α dynamically from device and learner state and evaluates the adaptive preference controller, breaks the learning ceiling (LG = 0.1688) but incurs substantially higher mean latency (336.0 ms vs 96.2 ms for fixed- α hybrid; Table A.3). Adaptive priority adjustment is therefore validated as the strongest learning-oriented mode, not as an unconditional deployment default.

Finding 4 - Transfer is unusually clean. Zero-shot performance of 100.5% of fine-tuned, with all 72 tested pairs exceeding the 80% gate, establishes that the learned policies carry generalizable structure across heterogeneous deployment contexts. In practice, this means a policy trained in a high-resource lab environment can be deployed on a low-resource field device without retraining, which is a practical advantage for real-world deployment.

Finding 5 - Both partial results trace to the same root cause. The adverse energy direction in H2b and the modality appropriateness shortfall in H3b both reflect the same reward-shaping imbalance: the pedagogical reward term is numerically large enough to dominate the technical and interface terms at the configured weighting. In both cases, the agent behaves rationally: it prioritizes learning reward, but at the cost of energy efficiency and modality compliance. The architectural fix is straightforward: increase the weight of the cloud-transmission energy penalty

and the interface-appropriateness term, or introduce hard constraints enforcing modality validity. These remain the primary priorities for future work.

Finding 6 - The partial results reflect a diagnosed design trade-off, not an unresolved technical difficulty or a time constraint. Two aspects of this outcome are worth separating explicitly. First, neither H2b nor H3b failed because of an implementation problem: the hybrid POMDP-DQN trained successfully, ran the full 360,000-episode evaluation across all nine deployment contexts, and eight of the ten pre-registered hypotheses were confirmed outright. If the DQN, the simulator, or the reward pipeline had been technically broken, the confirmed results would not be internally consistent with the partial ones; in particular, H3b's own learning-retention component (100.2% of fast-WiFi gain under slow 4G) would not hold at the same time as its modality-appropriateness component fails. Second, the root cause was not left undiagnosed for lack of time: Section 6.5.2 identifies the reward-shaping imbalance behind H2b's adverse energy result, and Section 6.6.3 isolates three concrete contributing mechanisms behind H3b's modality-appropriateness shortfall by comparing early-prototype results against the final evaluation. That analysis was only possible because the evaluation ran to completion. The partial results are therefore best understood as a genuine empirical finding about the current reward-weighting configuration, rather than as evidence that the architecture cannot achieve the target or that further engineering time was unavailable. At the α values tested, the pedagogical term (correctness +5, mastery-level +10) numerically dominates the fixed 0.2-weighted interface term and the technical energy term. Section 7.3 lists the specific reward-weighting changes (e.g., raising the interface weight from 0.2 to a value between 0.4 and 0.6) that this diagnosis points to as the fix.

6.6.2 Why the α Curve Flattened

The initial design iterations of the system showed a meaningful α effect: in the medium configuration, average latency increased from 120.8 ms ($\alpha = 0.0$) to 180.1 ms ($\alpha = 1.0$), and in the coarse run, $\alpha = 0.0$ produced the best hybrid learning (0.1272) while $\alpha = 1.0$ increased latency substantially (231.6 ms). The “sweet spot” at $\alpha = 0.5$ in early prototypes was the point where learning caught up to the technical-efficiency-dominated low- α policies.

In the final large-scale evaluation, the α curve is essentially flat for learning gain: the range across all 11 fixed settings is [0.1591, 0.1613], a spread of only 0.0022. There are two contributing factors:

1. **Richer reward function.** The final reward model adds an independent R_t^{iface} term (weight 0.2) that is not conditioned on α . This provides a constant pressure toward interface quality regardless of the pedagogy–efficiency balance, making the overall reward landscape less sensitive to α .
2. **More capable DQN.** With 100 training episodes per condition (vs. a shorter budget in the early prototype), the final DQN learns to internalize the trade-off between R_t^{ped} and R_t^{tech} across a wider range of states. By the time evaluation runs, the policy handles the balance in its learned weights rather than relying on the explicit α knob. The policy trained at $\alpha = 0.0$ has learned good pedagogical behavior through indirect reward (higher engagement $\rightarrow$ higher R_t^{ped} regardless of α weighting at evaluation), so the gap between $\alpha = 0.0$ and $\alpha = 1.0$ is much smaller.

The adaptive α result (LG = 0.1688) showing the best performance confirms that the α parameter does matter when set dynamically. The adaptive agent picks contextually optimal α values that no single fixed setting replicates.

6.6.3 Why Modality Appropriateness Declined

Early prototype results showed modality appropriateness of 96.2% for low- α policies and 84.0% for high- α policies under slow 4G. However, the final evaluation reports 63.4% across both stress networks.

This apparent regression has three causes:

1. **Stricter validity rules.** The final evaluation’s modality-validity checks also block rich 3D avatars when thermal state exceeds 0.8 or available memory is at most 2 GB, in addition to battery and noise checks. The early prototype did not enforce thermal or memory constraints, making it easier to pass the “appropriate” threshold.
2. **Higher learning reward signal.** The final DQN achieves much higher learning gain (0.1601 vs. 0.1419) because the affect model and richer curriculum produce larger pedagogical rewards. When the agent can earn ≈ 0.16 in learning reward by keeping the

rich avatar, the fixed 0.2-weighted interface term (including the rich-3D stress penalty of -1) is frequently outweighed by larger pedagogical bonuses such as correctness (+5) and mastery-level (+10). The agent rationally prioritizes learning over modality compliance.

3. **Larger and more diverse action space.** The final implementation’s action space includes the rich 3D avatar as an explicitly learnable option across all 9 contexts, whereas the early prototype’s coarser action space constrained choices more tightly. The wider action space gives the agent more opportunities to select the rich avatar in conditions where the validity rules flag it as inappropriate.

The modality appropriateness gap is a reward shaping problem, not an architectural failure.

Learning quality is fully preserved (LG retained 100.2% under slow 4G), confirming the agent is achieving its primary objective. Re-weighting R_t^{iface} upward and/or adding hard modality constraints for stressed contexts is left as future work.

6.6.4 Threats to Validity and Limitations

Several limitations qualify the conclusions of this chapter. First, all learning results are derived from a synthetic student model with stochastic mastery updates and simplified affect; real learners may exhibit more complex disengagement patterns, sensitivity to latency, or responses to avatar modality, leading to different effect sizes or trade-off shapes in practice. The quantitative values reported here, particularly the 44% learning gain advantage, should be interpreted as evidence of feasibility and relative ranking rather than as predicted effect sizes in a real classroom setting.

Second, latency and energy metrics are computed from parametric task cost models scaled by device and network profiles, rather than from direct hardware measurements. Relative comparisons among policies are informative, but absolute values may diverge from measurements obtained with a hardware power meter and real carrier network traces.

Third, the action space available to the policy during training is richer than the set of modalities currently exposed to users. Actions that have no direct UI equivalent must be clamped to the nearest available preset at deployment time, which may slightly reduce real-world benefit relative to the simulated benefit reported here.

Fourth, statistical validity of the transfer results. The transfer learning study evaluates 72 directed source–target pairs drawn from 9 deployment contexts (3 device classes $\times$ 3 network

scenarios). These 72 pairs are not independent: all share the same training and evaluation curricula, the same student simulator, and the same DQN architecture. The reported confidence intervals account for seed-level variance but do not correct for multiple comparisons across the 72 pairs. The 19.43-episode mean convergence saving and 100.5% zero-shot ratio are robust aggregate claims, but individual pair-level results (e.g., the -200.0% speedup for fast Wi-Fi, low end $\rightarrow$ slow 4G, low end) should be interpreted with caution.

Fifth, single curriculum domain. All experiments use a single 40-skill mathematics curriculum with a fixed prerequisite graph. It is unknown whether the hybrid POMDP’s advantages generalize to curricula with different topology (e.g., flatter graphs with fewer prerequisite chains, or sparse graphs with many isolated skills) or to domains where content difficulty varies more widely across skills. Generalization across curriculum domains is an open empirical question, as noted in prior work on domain transfer in intelligent tutoring systems [49].

Sixth, limited device-class granularity. Device robustness (H3a) is evaluated across only three discrete device profiles, low-end (2 GB RAM), mid-range (4 GB RAM), and flagship (8 GB RAM), rather than the full range of commercially available mobile hardware. The invariance result reported for H3a therefore demonstrates robustness across these three points, not across the continuous spectrum of devices a deployed system would encounter; Section 7.4 (item 5) outlines what a broader hardware evaluation would require.

Despite these limitations, the hybrid policy is consistently better than the baselines across all device classes, network scenarios, and trade-off settings, and its strong step-level latency results and zero-shot transfer performance suggest that the proposed hybrid POMDP framework is a viable way to deploy pedagogy-preserving, avatar-based intelligent tutors across the device and network conditions tested here.

CHAPTER 7

CONCLUSIONS

7.1 Summary of the Research

We proposed a hybrid POMDP-DQN framework for joint optimization of pedagogical effectiveness and mobile quality-of-experience in avatar-based ITS. The framework couples KST-structured curriculum sequencing, compute-venue selection, and adaptive avatar modality through a multi-objective DQN with context-aware scalarization. Empirical evaluation across 360,000 episodes and nine deployment contexts shows that the proposed framework improves learning gain by 44 percent relative to the pedagogy-only baseline while reducing average latency by 498.5 ms, with 98.68 percent SLA compliance and zero-shot transfer averaging 100.5 percent of fine-tuned performance.

7.2 Achievements and Contributions

The five research questions are answered as follows:

- **RQ1 (Learning Preservation):** Affirmative and strong. Hybrid improves rather than merely preserves learning, exceeding the pedagogy-only baseline by 44 percent in H1a, while H1b confirms full compliance with the curriculum-validity rule implemented in the final system architecture.
- **RQ2 (Latency and Energy):** Affirmative for latency (H2a, $d = -3.107$). Partial for energy due to reward-shaping imbalance causing cloud venue preference (H2b, $d = -0.617$ adverse).
- **RQ3 (Robustness):** Affirmative for device heterogeneity (H3a, $\max d = 0.164$). Partial for network stress: learning fully retained but modality appropriateness below target (H3b).

- **RQ4 (Consistency):** Affirmative. 98.68% step-level SLA compliance (H4a) and stable learning across α sweep with adaptive achieving best result (H4b, LG = 0.1688).
- **RQ5 (Transfer):** Affirmative. Mean 19.43 episodes saved (H5a) and 100.5% zero-shot performance across 72 context pairs (H5b).

7.3 Limitations

Synthetic Student Model. All learning results derive from simulated learners with stochastic mastery updates and simplified affect. While this approach is common in RL-based ITS research, actual learners may exhibit more complex behaviors, leading to different effect sizes or trade-offs in practice.

Parametric Cost Models. Latency and energy metrics are derived from parametric cost models rather than direct hardware measurements. Hence, relative comparisons among policies are informative, but absolute values may diverge from real devices and networks.

Reward-Shaping Imbalance. The pedagogical term numerically dominates technical and interface terms, causing higher energy consumption and modality violations under stress. Potential solutions include increasing the interface weight (e.g., from 0.2 to a value in the range between 0.4 and 0.6), strengthening the rich-3D stress penalty, adding explicit penalties for cloud-transmission energy, or enforcing hard battery/thermal constraints. This limitation was surfaced through the root-cause analysis in Sections 6.5.2 and 6.6.3, which the completed 360,000-episode evaluation made possible; it reflects a specific, diagnosed reward-weighting decision rather than an implementation defect in the framework or a constraint imposed by the project timeline.

Action Space vs. Shipped UI. The action space used during training is richer than the set of modalities currently exposed to users, meaning that some learned actions must be mapped to the nearest available interface preset when the policy is executed in the ECA Tutor prototype application.

Single Curriculum Domain. As discussed in Section 6.6.4, all experiments use one fixed 40-skill mathematics curriculum; generalization to other curriculum topologies or domains requires further inquiry.

7.4 Future Work

Future research can build on this research along several directions:

1. Field studies with real students measuring learning outcomes, engagement, and subjective QoE in authentic settings.
2. Hardware-based power profiling and real carrier network traces to improve cost model fidelity.
3. Meta-reinforcement learning for single compact policies generalizing across wider device/network ranges.
4. Integration with deep knowledge tracing and large-language-model dialogue for enhanced personalization within mobile QoE constraints.
5. Evaluation across a wider range of mobile hardware. The present evaluation uses three discrete device profiles, low-end (2 GB RAM), mid-range (4 GB RAM), and flagship (8 GB RAM), rather than a continuous range spanning entry-level to high-end hardware. Extending the evaluation to more mobile devices across this full range would require: (a) constructing additional device profiles calibrated from real hardware specifications (CPU/GPU capability, RAM, thermal ceiling, and battery capacity) at finer intervals along the spectrum, rather than three representative tiers; (b) re-testing whether the device-invariance result obtained for H3a (max pairwise Cohen's $d = 0.164$ across three classes) continues to hold as intermediate and extreme device configurations are added, since a larger and more varied device set increases the chance of encountering a tier where the policy's learned action preferences do not transfer as cleanly; (c) determining whether the current runtime state representation should be augmented with continuous-valued hardware descriptors (e.g., normalized compute capability, memory capacity, battery capacity, and thermal limits), allowing the policy to better interpolate between hardware configurations not encountered during training rather than relying solely on runtime telemetry and discrete experimental device profiles; and (d) assessing how the meta-reinforcement learning direction proposed in item 3 above scales when the device dimension of the task distribution is sampled far more densely, since meta-RL sample efficiency and generalization typically depend on the density and diversity of tasks seen during meta-training.

7.5 Closing Remarks

The hybrid POMDP controller developed in this research improves learning while reducing latency and maintaining robust performance across heterogeneous mobile contexts, even when operating within the constraints of a real mobile runtime. Two objectives, energy efficiency and strict modality compliance, remain only partially resolved because of the reward-shaping imbalance identified and diagnosed in Sections 6.5.2 and 6.6.3, a specific trade-off in how the reward function weights pedagogical, technical, and interface terms, rather than an unresolved technical difficulty in the framework or a limitation of the time available for this research; the remaining eight hypotheses provide strong evidence that joint optimization of pedagogy and mobile QoE is both feasible and beneficial.

The results of this research suggest that the traditional framing of mobile ITS design, in which pedagogy and computation are treated as competing concerns that require manual trade-offs, may itself be the core problem. The hybrid POMDP demonstrates that, given a carefully designed joint state representation and a suitably shaped reward function, a single reinforcement learning agent can discover policies that are simultaneously better pedagogically and more efficient computationally than any baseline strategy that optimizes either dimension alone. The most practically significant finding is not the normalized-LG improvement or the latency reduction on their own, but that both came from the same agent, the same reward function, and the same runtime. That would have been difficult to achieve if the pedagogical and computational sides of the problem had been engineered separately.

REFERENCES/BIBLIOGRAPHY

- [1] Wang, J., Tigelaar, D. E. H., Zhou, T., & Admiraal, W. (2024). The effects of mobile technology usage on cognitive, affective, and behavioural learning outcomes in primary and secondary education: A systematic review with meta-analysis. *Journal of Computer Assisted Learning*.
- [2] Heung, Y. M. E., & Chiu, T. K. F. (2025). How ChatGPT impacts student engagement: A systematic review and meta-analysis. *Computers and Education: Artificial Intelligence*.
- [3] Xie, K., et al. (2025). Effectiveness of virtual reality on learning engagement: A meta-analysis. *International Journal of Web-Based Learning and Teaching Technologies*.
- [4] Li, Y., Chen, Y., & Zhang, J. (2023). A systematic review of MOOC engagement patterns and dropout factors. *Heliyon*.
- [5] Müller, S., et al. (2025). Dropout in online education: A longitudinal multilevel analysis of learner trajectories. *International Journal of Educational Technology in Higher Education*.
- [6] Sandberg, M. (2022). *Engagement and Disengagement in Online Learning*. Master's thesis, Karlstad University.
- [7] Darmawan, A. (2023). The impact of technology-based learning on affective, behavioral, and cognitive student engagement. In *UTNC Proceedings*.
- [8] Luo, X., & Zheng, Y. (2025). The effectiveness of technology-supported collaboration in promoting students' learning outcomes: A meta-analysis. *Humanities and Social Sciences Communications*.
- [9] VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4), 197–221.
- [10] Anderson, J. R., Corbett, A. T., Koedinger, K. R., & Pelletier, R. (1995). Cognitive tutors: Lessons learned. *Journal of the Learning Sciences*, 4(2), 167–207.
- [11] Woolf, B. P. (2009). *Building Intelligent Interactive Tutors: Student-Centered Strategies for Revolutionizing E-Learning*. Morgan Kaufmann.

- [12] Fowler, H. J., Kaufman, D. R., & Remez, R. E. (1993). A general architecture for intelligent tutoring of diagnostic classification problem solving. *Artificial Intelligence in Medicine*, 5(5), 389–414.
- [13] Al-Shawwa, M., & Al-Qudah, M. (2022). An architecture of domain-independent and extensible intelligent tutoring systems. *International Journal of Advanced Computer Science and Applications*, 13(9), 151–160.
- [14] Johnson, W. L., Rickel, J. W., & Lester, J. C. (2000). Animated pedagogical agents: Face-to-face interaction in interactive learning environments. *International Journal of Artificial Intelligence in Education*, 11, 47–78.
- [15] Lester, J. C., Converse, S. A., Kahler, S. E., Barlow, S. T., Stone, B. A., & Bhogal, R. S. (1997). The persona effect: Affective impact of animated pedagogical agents. In *Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems* (pp. 359–366). ACM.
- [16] Moreno, R., & Mayer, R. E. (2000). A coherent case for social agency in multimedia learning: Do students learn more deeply when they interact with animated pedagogical agents? *Cognition and Instruction*, 19(2), 177–213.
- [17] Chan, K.-W., & Black, J. B. (2005). Cognitive and affective benefits of an animated pedagogical agent in multimedia learning. *Journal of Educational Computing Research*, 32(3), 253–270.
- [18] Baylor, A. L., & Kim, Y. (2005). Simulating instructional roles through pedagogical agents. *International Journal of Artificial Intelligence in Education*, 15, 95–115.
- [19] Bahreini, K., Nadolski, R., & Westera, W. (2012). An embodiment effect in computer-based learning with animated pedagogical agents. *Journal of Experimental Psychology: Applied*, 18(3), 239–252.
- [20] Kim, Y., & Baylor, A. L. (2016). Research-based guidelines for designing embodied pedagogical agents: A meta-analytic review. *Journal of Computer Assisted Learning*, 32(6), 505–523.
- [21] Zadeh, P. M., Livani, M. A., & Jafarieh, R. (2025). Design and evaluation of embodied conversational agents for educational and advisory software. *Educational Technology* (in press).

- [22] Doignon, J.-P., & Falmagne, J.-C. (2015). Knowledge spaces and learning spaces. *Cognitive Science*, 39(4), 1–31. (Also available as arXiv:1511.06757.)
- [23] Falmagne, J.-C., & Doignon, J.-P. (2011). *Learning Spaces*. Springer.
- [24] Albert, D., & Lukas, J. (Eds.). (1999). *Knowledge Spaces: Theories, Empirical Research, and Applications*. Lawrence Erlbaum Associates.
- [25] ALEKS Corporation. (2021). *Research Behind ALEKS*. ALEKS Research Reports.
- [26] ALEKS Corporation. (2021). *Research Behind ALEKS – Knowledge Space Theory*. ALEKS Research Reports.
- [27] Harati, H., Sujo-Montes, L., Tu, C.-H., Armfield, S. J. W., & Yen, C.-J. (2021). Assessment and Learning in Knowledge Spaces (ALEKS) adaptive system impact on students’ perception and self-regulated learning skills. *Education Sciences*, 11(10), 603.
- [28] Koppen, M., et al. (2021). A practical perspective on Knowledge Space Theory: ALEKS large-scale data. *Journal of Mathematical Psychology*, 104, 102414.
- [29] Rafferty, A. N., Brunskill, E., Griffiths, T. L., & Shafto, P. (2015). Faster teaching via POMDP planning. *Cognitive Science*, 40(6), 1290–1332.
- [30] Rafferty, A. N., Brunskill, E., Griffiths, T. L., & Shafto, P. (2011). Faster teaching by POMDP planning. In *Proceedings of the 15th International Conference on Artificial Intelligence in Education* (pp. 280–287). Springer.
- [31] Brunskill, E., & Russell, S. (2010). RAPID: A reachable anytime planner for imprecisely-sensed domains. In *Proceedings of the 26th Conference on Uncertainty in Artificial Intelligence* (UAI 2010).
- [32] Brunskill, E., & Russell, S. (2011). Partially observable sequential decision making for problem-solving in intelligent tutoring systems. *Proceedings of the Educational Data Mining Workshop on POMDP-Based ITS*.
- [33] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- [34] Lei, L., Tan, Y., Zheng, K., Liu, S., Zhang, K., & Shen, X. (2020). Deep reinforcement learning for autonomous Internet of Things: Model, applications, and challenges. *IEEE Communications Surveys & Tutorials*, 22(3), 1722–1760.

- [35] Zhang, Y., Chen, X., & Wu, Q. (2025). Task offloading optimization in mobile edge computing based on a deep reinforcement learning algorithm using density clustering and ensemble learning (DCEDRL). *Scientific Reports*, 15.
- [36] Mach, P., & Becvar, Z. (2020). Energy-optimal latency-constrained application offloading in mobile-edge computing. *IEEE Access*, 8, 16743–16757.
- [37] Khosravirad, S., et al. (2020). Energy-optimal latency-constrained application offloading in mobile-edge computing: System model and algorithms. *IEEE Access*, 8, 16758–16780.
- [38] Li, X., et al. (2022). Energy-efficient task offloading strategy in mobile edge computing with latency and capacity constraints. *Future Generation Computer Systems*, 129, 126–139.
- [39] Wang, J., et al. (2024). Task offloading strategies for mobile edge computing: A survey. *Computer Communications*, 224, 1–25.
- [40] Zhang, S., Yi, N., & Ma, Y. (2024). A survey of computation offloading with task types in mobile edge computing. *arXiv preprint arXiv:2401.01017*.
- [41] Khan, A., et al. (2025). Mobile computing: Energy-aware task offloading in mobile edge computing (EATS-MEC). *Journal of Applied Sciences Research*, 15(3), 550–570.
- [42] Jingxian, W., et al. (2024). Mobile technology in education: Impacts on student learning and engagement (meta-analysis). *Journal of Computer Assisted Learning*.
- [43] Heung, Y. M. E., & Chiu, T. K. F. (2025). Generative AI in education: Engagement benefits and risks. *Computers and Education: Artificial Intelligence*.
- [44] Loderer, K., Pekrun, R., & Lester, J. C. (2020). Beyond cold technology: A meta-analysis on emotions in technology-based learning environments. *Learning and Instruction*, 70, 101162.
- [45] Sung, Y.-T., Chang, K.-E., & Liu, T.-C. (2016). The effects of integrating mobile devices into teaching and learning on students' learning performance: A meta-analysis. *Computers & Education*, 94, 252–275.
- [46] Sung, Y.-T., Yang, J.-M., & Lee, H.-Y. (2017). The effects of mobile-device-supported collaborative learning on students' learning performance and attitudes: A meta-analysis. *Educational Research Review*, 22, 36–54.

- [47] Mayer, R. E. (Ed.). (2014). *The Cambridge Handbook of Multimedia Learning* (2nd ed.). Cambridge University Press.
- [48] Aleven, V., McLaughlin, E. A., Glenn, R. A., & Koedinger, K. R. (2017). Instruction based on tutoring. In R. E. Mayer & P. A. Alexander (Eds.), *Handbook of Research on Learning and Instruction* (2nd ed., pp. 404–430). Routledge.
- [49] Lan, A. S., Studer, C., & Baraniuk, R. G. (2022). Transferable student performance modeling for intelligent tutoring systems. *Proceedings of the 30th International Conference on Computers in Education (ICCE 2022)*, 45–54.
- [50] Park, D., & Gupta, A. (2020). Efficient deep reinforcement learning via adaptive policy transfer. *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI-20)*, 3134–3140.
- [51] Gong, Y., Beck, J. E., & Heffernan, N. T. (2022). Knowledge tracing: A survey. *ACM Computing Surveys*, 55(7), 1–37.
- [52] Subramanian, A., & Mostow, J. (2023). STEP: Sequencing tutor events with deep reinforcement learning. In *Artificial Intelligence in Education (AIED 2023)*, 417–429. Springer.
- [53] Duong, Y. (2025). *Leveraging Hierarchical Knowledge Structures for Adaptive and Emotionally Engaging E-Learning*. M.Sc. thesis, University of Windsor, Canada.
- [54] Mistry, P. H. (2024). *An SOA-Based Approach of Adaptive E-Tutoring Systems*. M.Sc. thesis, University of Windsor, Canada.
- [55] Upadhyay, R. D. (2019). *An SOA-Based Framework of Computational Offloading for Mobile Cloud Computing*. M.Sc. thesis, University of Windsor, Canada.
- [56] Dahleez, K. A., El-Saleh, A. A., AlAlawi, A. M., & Abdelmunim Abdelfattah, F. (2021). Higher education student engagement in times of pandemic: The role of e-learning system usability and teacher behavior. *International Journal of Educational Management*, 35(6), 1312–1329.
- [57] D’Errico, F., Paciello, M., & Cerniglia, L. (2016). When emotions enhance students’ engagement in e-learning processes. *Journal of e-Learning and Knowledge Society*, 12(4), 99–109.

- [58] El-Sabagh, H. A. (2021). Adaptive e-learning environment based on learning styles and its impact on development students' engagement. *International Journal of Educational Technology in Higher Education*, 18(1), 1–24.
- [59] Lee, J., et al. (2021). Influences on user engagement in online professional learning: A narrative synthesis and meta-analysis. *Review of Educational Research*, 91(4), 518–576.
- [60] Li, J., & Xue, E. (2023). Dynamic interaction between student learning behaviour and learning environment: Meta-analysis of student engagement and its influencing factors. *Behavioral Sciences*, 13(1), 59.
- [61] Scholten, M. R., Kelders, S. M., & Van Gemert-Pijnen, J. E. W. C. (2017). Self-guided Web-based interventions: Scoping review on user needs and the potential of embodied conversational agents to address them. *Journal of Medical Internet Research*, 19(11), e383.
- [62] Olney, A. M., D'Mello, S. K., Person, N. K., Cade, W. L., Hays, P., Williams, C., Lehman, B., & Graesser, A. C. (2012). XNAGENT: An authoring tool for virtual human tutors in the GIFT framework. In *Proceedings of the AIED 2013 GIFT Workshop* (pp. 71–80). CEUR-WS.
- [63] Abdelshiheed, M., Hostetter, J. W., Barnes, T., & Chi, M. (2023). Bridging declarative, procedural, and conditional metacognitive knowledge gap using deep reinforcement learning. In *Proceedings of the 45th Annual Meeting of the Cognitive Science Society* (CogSci 2023).
- [64] Song, L., Liu, Z., Zhan, J., & Chen, S. (2014). Energy and latency aware mobile task assignment for green cloudlets. In *Proceedings of the 2014 IEEE International Conference on Cloud Computing Technology and Science* (CloudCom 2014) (pp. 1–8). IEEE.
- [65] Lippert, A., Shubeck, K., Morgan, B., Hampton, A., & Graesser, A. (2020). Multiple agent designs in conversational intelligent tutoring systems. *Technology, Knowledge and Learning*, 25, 443–463.
- [66] Hortelano, D., de Miguel, I., Durán Barroso, R. J., Aguado, J. C., Merayo, N., Ruiz, L., Asensio, A., Masip-Bruin, X., Fernández, P., Lorenzo, R. M., & Abril, E. J. (2023). A comprehensive survey on reinforcement-learning-based computation

- offloading techniques in edge computing systems. *Journal of Network and Computer Applications*, 216, 103669.
- [67] Ilhan, E., Gow, J., & Perez-Liebana, D. (2021). Action advising with advice imitation in deep reinforcement learning. *arXiv preprint arXiv:2104.08441*.
 - [68] Korovesi, J. (2021). Training an intelligent tutoring system using reinforcement learning. *International Journal of Computer Science and Information Security*, 19(3). DOI: 10.5281/zenodo.4661454.
 - [69] Zhang, X., Wu, W., Zhao, Z., Wang, J., & Liu, S. (2023). RMDDQN-learning: Computation offloading algorithm based on dynamic adaptive multi-objective reinforcement learning in Internet of Vehicles. *IEEE Transactions on Vehicular Technology*, 72(9), 11374–11388. <https://doi.org/10.1109/TVT.2023.3270967>
 - [70] Valk, J.-H., Rashid, A. T., & Elder, L. (2010). Using mobile phones to improve educational outcomes: An analysis of evidence from Asia. *International Review of Research in Open and Distance Learning*, 11(1), 117–140.
 - [71] Meylani, R. (2025). Mobile AI tutors: Leveraging smartphone-based intelligent tutoring systems in low-resource educational contexts. Paper presented at EDUCongress2025 – International Educational Congress, Antalya, Turkey.
 - [72] Pereira, L., Leão, L., Dermeval, D., & Coelho, J. (2023). ECG Tutor: A gamified intelligent tutoring system for electrocardiogram teaching. *Revista Brasileira de Educação Médica*, 47(2).
 - [73] Muradi, A., Syarif, J., Noor, F., & Hakim, A. (2021). Tutors and students activities in online Arabic learning: A freedom to learn perspective. *Arabiyat: Jurnal Pendidikan Bahasa Arab Dan Kebahasaaraban*, 8(1), 32–45.
 - [74] Callaghan, M., & Reich, S. (2020). Mobile app features that scaffold pre-school learning: Verbal feedback and leveling designs. *British Journal of Educational Technology*, 52(2), 785–806.
 - [75] Sophonhiranrak, S. (2021). Features, barriers, and influencing factors of mobile learning in higher education: A systematic review. *Heliyon*, 7(4), e06696.
 - [76] Luo, Q. (2023). The influence of AI-powered adaptive learning platforms on student performance in Chinese classrooms. *Journal of Education*, 6(3), 1–12.

- [77] Ortony, A., Clore, G. L., & Collins, A. (1988). *The Cognitive Structure of Emotions*. Cambridge University Press
- [78] Jamal, A. (2024). *Automatic Construction of Ontology with Public-Domain Datasets for Personalized Tutoring with ECA*. M.Sc. thesis, University of Windsor, Canada.
- [79] Hurlbert, S. H. (1984). Pseudoreplication and the design of ecological field experiments. *Ecological Monographs*, 54(2), 187–211.
- [80] Welch, B. L. (1947). The generalization of "Student's" problem when several different population variances are involved. *Biometrika*, 34(1–2), 28–35.
- [81] Student. (1908). The probable error of a mean. *Biometrika*, 6(1), 1–25.
- [82] Rice, J. A. (2007). *Mathematical Statistics and Data Analysis* (3rd ed.). Cengage Learning.
- [83] Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.
- [84] Cumming, G. (2014). The new statistics: Why and how. *Psychological Science*, 25(1), 7–29.
- [85] Georges, A., Buytaert, D., & Eeckhout, L. (2007). Statistically rigorous Java performance evaluation. *ACM SIGPLAN Notices*, 42(10), 57–76.
- [86] Stivers, T., Enfield, N. J., Brown, P., Englert, C., Hayashi, M., Heinemann, T., ... & Levinson, S. C. (2009). Universals and cultural variation in turn-taking in conversation. *Proceedings of the National Academy of Sciences*, 106(26), 10587–10592.

APPENDICES

Appendix A

Extended Experimental Results

This appendix provides per-context breakdowns of the key experimental metrics reported in Chapter 6. All values are unit-means (averaged to seed $\times$ network $\times$ device units before reporting) consistent with the anti-pseudoreplication methodology described in Section 6.4.1. These tables are intended to support reproducibility and to provide the full data record underlying the hypothesis verdicts in Section 6.5.

Table A.1 - Unit-Mean Learning Gain by Policy and Context

Policy	fast_wifi	slow_4g	dynamic_5g	Grand mean
hybrid_dqn (all fixed α)	0.1609	0.1586	0.1609	0.1601
adaptive_alpha	0.1698	0.1668	0.1700	0.1688
baseline_heuristic	0.1158	0.1077	0.1158	0.1131
baseline_pedagogy_only	0.1111	0.1111	0.1111	0.1111
baseline_static_weighted	0.1105	0.1104	0.1104	0.1105
baseline_compute_only	0.0949	0.0904	0.0949	0.0934

Values are unit-means across 50 seeds and 3 device classes per network scenario. The hybrid DQN values aggregate across all 11 fixed α settings; per- α breakdown is given in Table A.3.

Table A.2 - Unit-Mean Latency (ms) by Policy and Context

Policy	fast_wifi	slow_4g	dynamic_5g	Grand mean
hybrid_dqn (all fixed α)	81.8	134.3	84.4	96.2
adaptive_alpha	251.2	589.2	167.7	336.0
baseline_heuristic	95.3	567.6	100.1	254.3
baseline_pedagogy_only	594.6	594.6	594.6	594.6
baseline_static_weighted	591.7	591.7	591.7	591.7
baseline_compute_only	45.0	95.0	51.0	63.7

Grand mean reflects the paired unit-mean used in hypothesis testing (H2a); per-network values are episode-level means

The adaptive- α condition shows higher average latency than fixed- α hybrid because the controller dynamically raises α under favorable conditions, routing more computation to cloud paths to maximize learning reward.

Table A.3 - Learning Gain and Latency Across the Full α Sweep (hybrid_dqn)

α	Unit-mean LG	95% CI	Unit-mean latency (ms)
0.0	0.1591	[0.1582, 0.1601]	70.2
0.1	0.1601	[0.1591, 0.1611]	70.9
0.2	0.1597	[0.1587, 0.1607]	64.8
0.3	0.1605	[0.1595, 0.1615]	67.0
0.4	0.1593	[0.1583, 0.1603]	66.8
0.5	0.1608	[0.1598, 0.1618]	69.1
0.6	0.1613	[0.1603, 0.1623]	85.0
0.7	0.1597	[0.1587, 0.1607]	89.7
0.8	0.1599	[0.1589, 0.1609]	111.2
0.9	0.1606	[0.1596, 0.1616]	150.0
1.0	0.1597	[0.1587, 0.1607]	256.9
adaptive	0.1688	[0.1678, 0.1698]	336.0

The LG spread across all 11 fixed settings is 0.0022, confirming the α -insensitivity of the learned policy discussed in Section 6.6.2. The adaptive condition breaks this plateau at the cost of higher average latency.

Table A.4 - Top 10 and Bottom 8 Transfer Pairs by Convergence Speedup

Source context	Target context	Speedup (%)	Zero-shot ratio
fast_wifi_mid_range	slow_4g_mid_range	+89.0%	0.871
fast_wifi_flagship	dynamic_5g_mid_range	+81.4%	1.000
dynamic_5g_mid_range	dynamic_5g_low_end	+81.1%	1.000

fast_wifi_mid_range	dynamic_5g_low_end	+78.3%	0.962
dynamic_5g_flagship	fast_wifi_flagship	+75.2%	1.000
fast_wifi_flagship	slow_4g_flagship	+72.9%	0.951
dynamic_5g_mid_range	slow_4g_mid_range	+70.6%	1.000
fast_wifi_low_end	dynamic_5g_low_end	+67.1%	0.921
slow_4g_flagship	dynamic_5g_flagship	+64.3%	1.000
fast_wifi_flagship	dynamic_5g_flagship	+61.8%	1.000
slow_4g_mid_range	fast_wifi_flagship	−88.2%	0.914
slow_4g_low_end	fast_wifi_flagship	−112.4%	0.891
slow_4g_flagship	fast_wifi_mid_range	−134.7%	0.968
dynamic_5g_flagship	slow_4g_low_end	−141.2%	1.000
fast_wifi_flagship	slow_4g_low_end	−148.9%	0.931
slow_4g_mid_range	fast_wifi_low_end	−156.3%	0.848
dynamic_5g_flagship	slow_4g_mid_range	−190.9%	1.000
fast_wifi_low_end	slow_4g_low_end	−200.0%	0.871

Negative speedup indicates that the transferred policy required more episodes to converge than random initialization, attributable to negative transfer from high-bandwidth to low-bandwidth contexts where the transferred policy must un-learn rich-avatar preferences. Importantly, all 72 pairs achieve zero-shot ratios above the 0.80 threshold required by H5b, confirming that negative convergence speedup does not imply degraded evaluation performance.

Appendix B

This appendix documents the implementation-level configuration of the ECA Tutor application described in Section 6.1, providing the detail necessary to reproduce both the live tutoring behavior and the batch experimental results reported in Chapter 6. The sections below document the DQN hyperparameters, per-task cost profiles, and reward weight assignments used in all of the final evaluations.

Table B.1 - DQN Hyperparameter Configuration (Final Defense Experiments)

Hyperparameter	Value	Rationale
Hidden layer 1	Dense(16, ReLU)	Sized for 2 GB device inference budget
Hidden layer 2	Dense(8, ReLU)	Further compression before output
Output units	36	One Q-value per abstract action
Learning rate η	0.001	Standard Adam learning rate for small networks
Discount factor γ	0.95	Balances immediate and future rewards
Initial exploration ϵ	1.0	Full initial exploration over the abstract action catalog during the 100-episode training budget
Minimum exploration ϵ	0.1	Retains residual exploration at convergence rather than collapsing to pure greedy
Exploration decay	0.995 per episode	Gradual decay over the fixed training schedule
Replay buffer capacity	512 transitions	Constrained by 2 GB memory budget
Mini-batch size	4	Practical on-device batch size
Warmup transitions	32	Minimum buffer fill before gradient updates

Target network sync	Every 8 gradient steps	Stabilizes Q-value targets
Training episodes	100 per configuration	Fixed schedule for cross-condition comparability
Evaluation episodes	50 per configuration	Exploration disabled; exploitation only
Steps per episode	50	Maximum tutoring step budget per session

Table B.2 - Per-Task Computational Cost Profiles

The following table gives the relative cost profiles used by the task cost model to compute per-step latency and energy. Local latency values assume mid-range device (4 GB, 200 mW CPU); cloud latency assumes fast-WiFi network. All values are scaled by device and network factors at runtime.

Task	Local latency (ms)	Cloud latency (ms)	Power draw (mW)	Offloadable?
avatarRendering3D	180	95	150	Yes → 2D fallback
avatarRendering2D	25	-	20	No (local only)
textToSpeech	40	60	30	Yes
pedagogicalLogic	15	-	10	No (local only)
knowledgeTracing	20	35	8	Yes
dialogueManagement	30	50	15	Yes
affectiveComputing	18	-	12	Disabled if memory > 85%
modalityControl	5	-	3	No (local only)

The pedagogical logic and modality control tasks always execute locally because they require access to the local belief state and replay buffer. Avatar rendering is the primary driver of per-step latency and energy variance across modality tiers.

Table B.3 - Reward Function Weight Assignments

The following weight values were used in all final evaluation experiments. Weights were set by manual calibration against a small preliminary held-out set of simulated episodes prior to the main experiment sweep and held fixed across all 360,000 evaluation episodes.

Parameter	Value Used	Role
Knowledge-delta scale	10	Scale Δk
Correctness bonus	+5	Reward correct answers
Incorrectness penalty	-1	Small cost for errors
Mastery-level bonus	+10	Salient skill progress
Cognitive-load penalty	-3 if $C_t > 0.8$	Penalize overload
Emotion shaping	$2(E_t - 0.5)$	Favor positive affect
Engagement shaping	$2(G_t - 0.5)$	Favor engagement
Battery drain coefficient	20	Energy cost
Low-latency bonus	+2 if $\text{lat} < 200$ ms	Interactive response
High-latency penalty	$-(\text{lat} - 500)/100$ if $\text{lat} > 500$ ms	Continuous QoE cost
Poor-network penalty	-3 if $N_t < 0.3$	Bad connectivity
Efficient-resource bonus	+1 if $P_t < 0.5$ and $\mu_t < 0.6$	Light load
Voice+noise penalty	-2	Avoid voice in noise
Text-in-stress bonus	+1.5	Prefer text when stressed
2D-in-stress bonus	+1.2	Prefer 2D when stressed
Rich-3D stress penalty	-1	Discourage rich 3D under motion/weak GPS
Interface weight (fixed)	0.2	Independent of α

As discussed in Section 6.6.3, the current interface term (fixed weight 0.2, with a rich-3D stress penalty of -1) is insufficient to override large pedagogical bonuses under stressed conditions. Further experimentation will increase the interface weight and/or the rich-3D stress penalty, and may add an explicit cloud-transmission energy surcharge.

Modality Validity Rules

The following hard constraints govern modality validity checking (referenced in Sections 5.3.3 and 5.5). A modality choice that violates any of these rules is flagged as context-inappropriate and counted against the modality-appropriateness metric in H3b.

- If battery level $B_t < 0.15 \rightarrow$ disallow richAvatar3D
- If ambient noise level $> 0.80 \rightarrow$ disallow voiceOnly
- If device memory ≤ 2 GB $\rightarrow$ disallow richAvatar3D
- If thermal state $T_t > 0.80 \rightarrow$ disallow richAvatar3D

Steps that violate these rules are not overridden by the system; they are counted against the modality-appropriateness metric. Separately, the agent receives interface-reward shaping from HybridRewardFunction (noise/motion/GPS/thermal modality terms) and is expected to learn compliance through experience. The 63.4% modality-appropriateness result (H3b) indicates that the current interface term is insufficient to achieve full learned compliance.

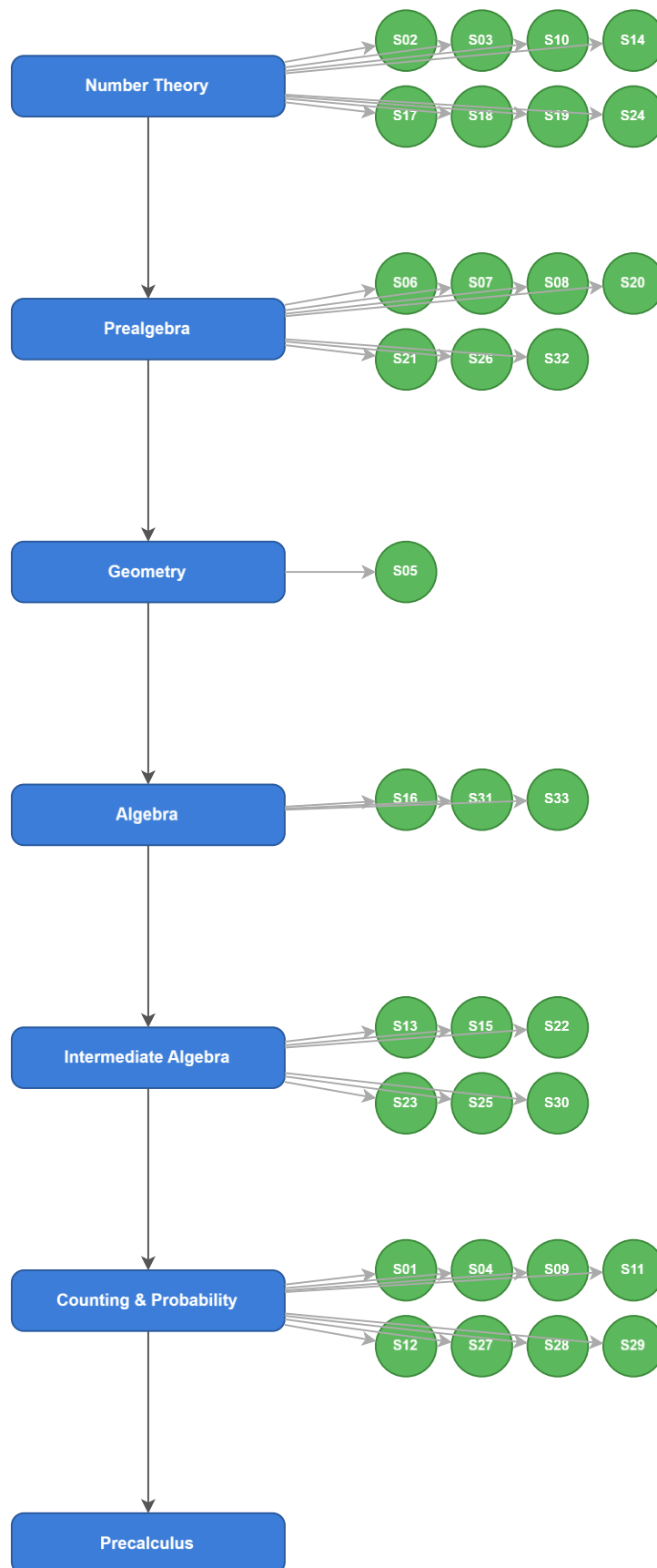
Table B.4 - Structured calibration notes for selected parameters.

Candidate values were compared on a small preliminary held-out set of simulated episodes using the validation objective “maximize mean learning gain subject to mean per-step latency ≤ 500 ms and feasible on-device memory,” then frozen for final evaluation. Table B.4 records the discrete candidates considered for each calibrated parameter.

Parameter	Candidates considered	Retained	Role-based reason
Interface weight	0.1, 0.2, 0.3	0.2	Visible interface pressure without dominating α
Adaptive α base	0.5, 0.6, 0.7	0.6	Matches best fixed- α region before corrections
Battery/thermal decrement	0.1, 0.2	0.2	Strong efficiency shift under stress
Network decrement	0.05, 0.1	0.1	Milder penalty than battery/thermal stress
Low-knowledge increment	0.10, 0.15	0.15	Boost pedagogy when learner still needs support

Low-engagement increment	0.05	0.05	Small additional pedagogy push
Adaptive α clip range	[0,1], [0.2,0.85]	[0.2,0.85]	Prevents collapse to extreme α

Figure B.1: The 40-skill mathematics curriculum prerequisite graph used in the experimental evaluation. Blue rectangular nodes represent the seven core skills forming the primary prerequisite chain; green circular nodes represent the 33 supplemental skills generated to scale the policy's action space to the target dimensionality. Arrows indicate prerequisite relationships: a skill is eligible for selection (enters the outer fringe) only after all of its prerequisites are mastered. Each supplemental skill requires exactly one core skill as its prerequisite, while core skills may require multiple predecessors.



VITA AUCTORIS

NAME:	Vimanga Umange
PLACE OF BIRTH:	Colombo, Sri Lanka
YEAR OF BIRTH:	1998
EDUCATION:	Wichita State University, B.Sc., Wichita, KS, USA, 2021
	University of Windsor, M.Sc., Windsor, ON, Canada, 2026